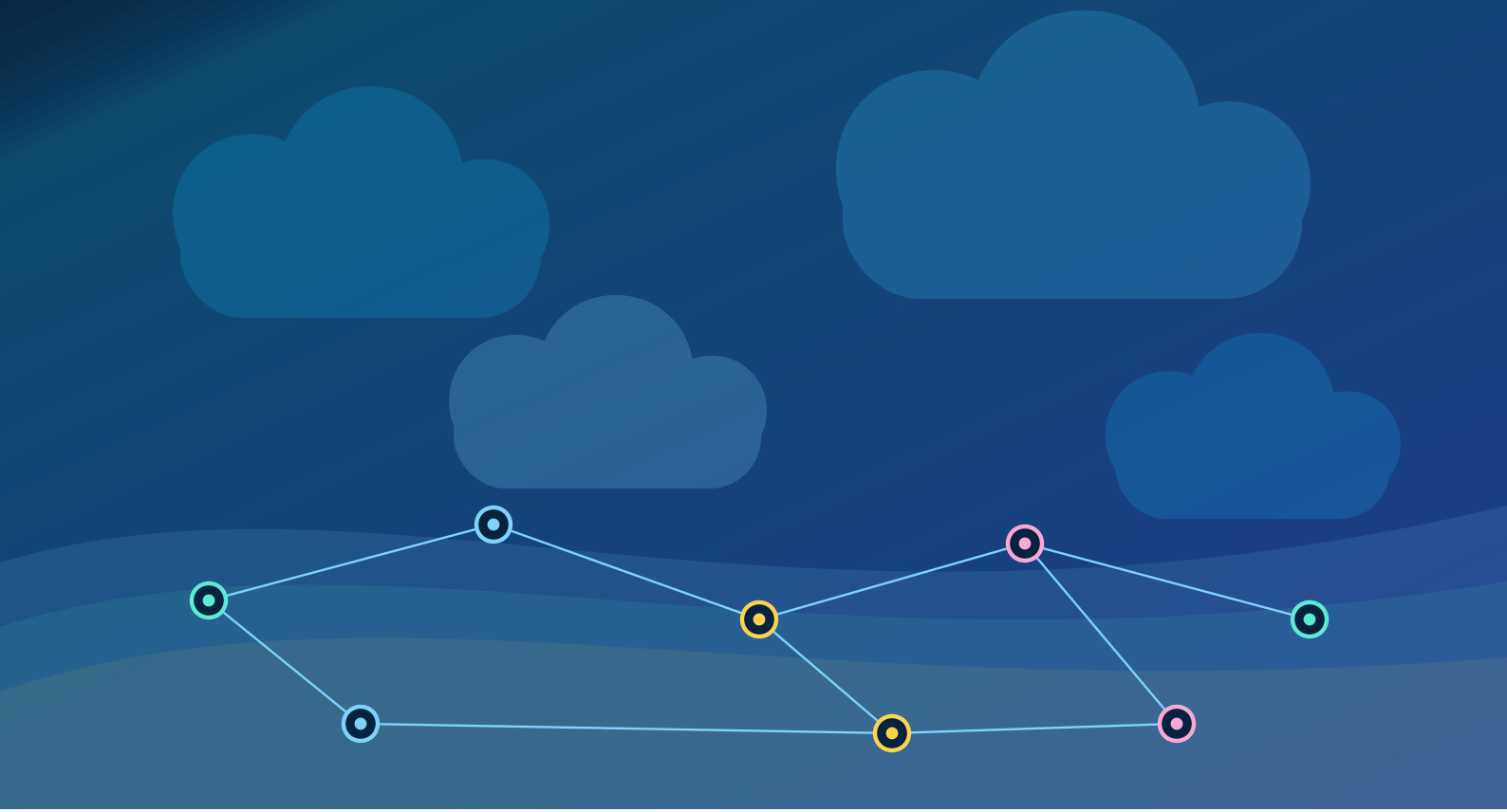
INTERVIEW HANDBOOK · 2026 EDITION

Cloud Concepts Explained

From servers and virtualization to VPCs, IAM, disaster recovery, IaC and cost optimization: every cloud concept for the interview, in learning order, with diagrams.

63 topics | 70+ diagrams | Interview Q&A | AWS · Azure · GCP



Contents

Topics are ordered the way you should learn them. Stars show how often a topic comes up in interviews.

PART 1 — CLOUD FOUNDATIONS

1.1 Server vs Client 4
1.2 Cloud Computing 5
1.3 On-Premises vs Cloud 6
1.4 CapEx vs OpEx 7
1.5 Pay-as-you-go Pricing 8
1.6 Public vs Private vs Hybrid Cloud 9
1.7 IaaS vs PaaS vs SaaS 10
1.8 Cloud Service Provider (CSP) 11
1.9 Shared Responsibility Model 12

PART 2 — COMPUTE & SCALING

2.1 Virtualization 15
2.2 Hypervisor 16
2.3 Virtual Machine (VM) 17
2.4 Container vs Virtual Machine 18
2.5 Container Orchestration & Kubernetes 19
2.6 Serverless Computing & Serverless Architecture 20
2.7 Scalability vs Elasticity 21
2.8 Vertical vs Horizontal Scaling 22
2.9 Auto Scaling 23

PART 3 — STORAGE & DATABASES

3.1 Object Storage 25
3.2 Block Storage 26
3.3 File Storage 27
3.4 Object vs Block vs File Storage 28
3.5 Storage Bucket 29
3.6 Data Durability vs Data Availability 30
3.7 Backup vs Replication 31
3.8 Managed Databases in the Cloud 32

PART 4 — NETWORKING

4.1 Public IP vs Private IP 34
4.2 DNS (Domain Name System) 35
4.3 HTTP vs HTTPS 36
4.4 Ports and Protocols 37
4.5 Firewall 38
4.6 Virtual Network and Subnet 39
4.7 Connecting Networks: VPN, Peering & Private Links 40
4.8 Load Balancer 41

PART 5 — RELIABILITY & DISASTER RECOVERY

5.1 Region vs Availability Zone 44
5.2 High Availability 45
5.3 Fault Tolerance 46
5.4 High Availability vs Fault Tolerance 47
5.5 SLA, SLO & SLI 48
5.6 Disaster Recovery 49
5.7 RTO vs RPO 50

PART 6 — SECURITY

6.1 IAM (Identity and Access Management) 52
6.2 Authentication vs Authorization 53
6.3 Principle of Least Privilege 54
6.4 Encryption at Rest vs Encryption in Transit 55
6.5 Secrets, Key Management & Compliance 56

PART 7 — ARCHITECTURE & DEVOPS

7.1 Microservices vs Monolithic Architecture 58
7.2 API (Application Programming Interface) 59
7.3 Event-Driven Architecture & Message Queues 60
7.4 Monitoring vs Logging 61
7.5 Infrastructure as Code (IaC) 62
7.6 CI/CD 63
7.7 Edge Computing 64
7.8 CDN (Content Delivery Network) 65

PART 8 — COST, STRATEGY & INTERVIEW TOOLKIT

8.1 Cloud Cost Optimization 68
8.2 Vendor Lock-in 69
8.3 Well-Architected Framework 70
8.4 Cloud Migration Strategies (the 6 Rs) 71
8.5 Cloud for Data Engineering: Putting It Together 72
8.6 Multi-Cloud Service Cheat Sheet 73
8.7 How to Answer a Cloud Architecture Question 74
8.8 Rapid-Fire Interview Q&A 75
8.9 Glossary: Every Term in One Line 78

How to use this handbook

Read parts 1–5 in order: each builds on the last. For every topic read the **In one line** box, study the diagram, then the **Interview angle** callouts. Finish with Part 8: cheat sheet, design template, Q&A and glossary.

Beyond your list

Added for freshers: Kubernetes, SLA/SLO/SLI, managed databases, connecting networks (VPN, peering), secrets & compliance, queues & events, Well-Architected, migration 6 Rs, cloud for data engineers, multi-cloud cheat sheet, and an architecture-answer template.

01

Part 1 · Cloud Foundations

What the cloud is, why companies move to it, how it is paid for, and who is responsible for what.

- Server vs Client
- Cloud Computing
- On-Prem vs Cloud · CapEx vs OpEx
- Pay-as-you-go
- Public · Private · Hybrid
- IaaS · PaaS · SaaS
- Cloud Service Providers
- Shared Responsibility Model

1.1 Server vs Client

★★☆

IN ONE LINE

A client requests a service; a server is an always-on computer that provides it.

Almost everything on the internet works as a **conversation**: a **client** sends a request, a **server** sends back a response. A "server" is just a computer (or a program on it) that is switched on all the time and offers a service to many clients.

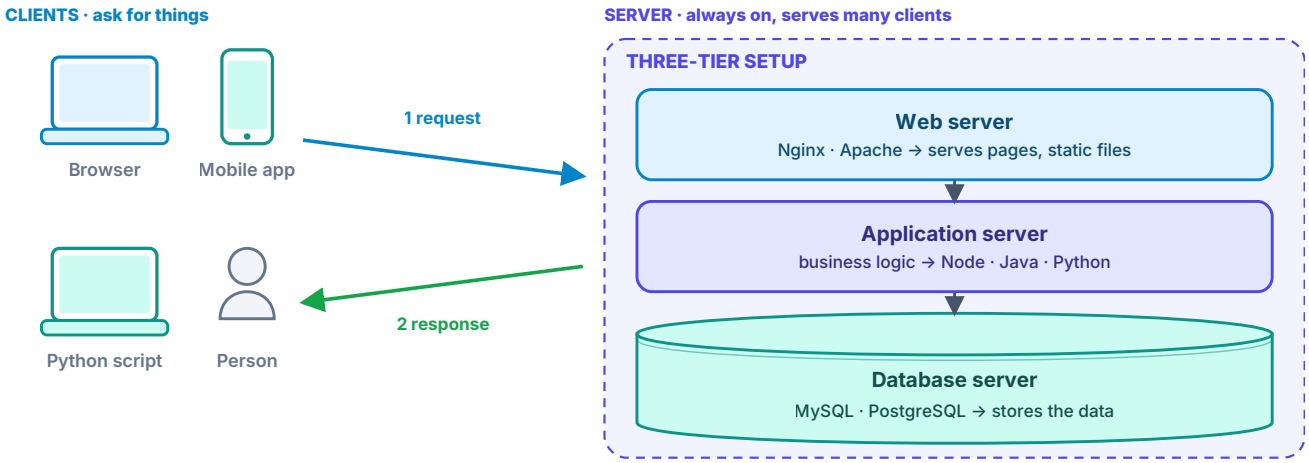


Figure 1.1 — Clients send requests; servers respond

	Client	Server
Role	Asks for a service	Provides the service
Examples	Browser, mobile app, `curl`, a script	Web server, database server, mail server, file server
Lifetime	Starts when the user needs it	Runs 24 × 7, waits for requests
Hardware	Laptop, phone	Powerful machine in a data centre — or a cloud VM

REAL-WORLD ANALOGY

A restaurant: **you are the client** (you order), the **kitchen is the server** (it prepares and serves, for many tables at once). The waiter is the network.

INTERVIEW ANGLE

"Server" means two things: the **physical machine** and the **software** running on it (e.g. an Nginx web server). In the cloud you rent the machine as a virtual server, and you install or use the software on top.

1.2 Cloud Computing

★★★

IN ONE LINE

Renting computing resources over the internet, on demand, and paying only for what you use.

Cloud computing is the on-demand delivery of IT resources — servers, storage, databases, networking, software, analytics — over the internet with pay-as-you-go pricing. Instead of buying and running your own hardware, you **rent** it from a provider who runs huge data centres.

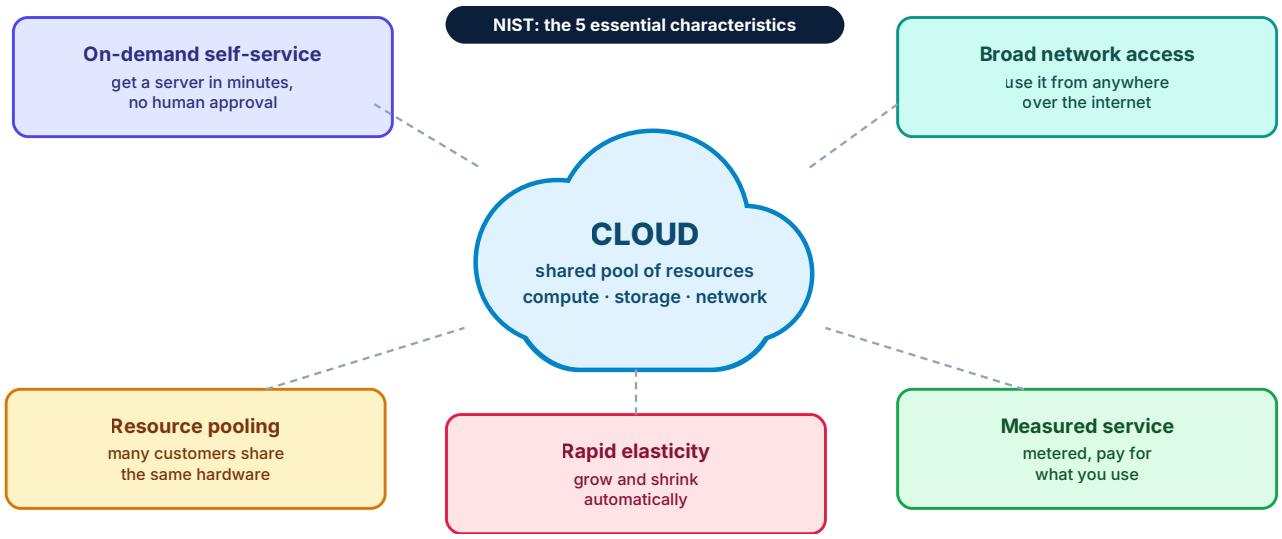
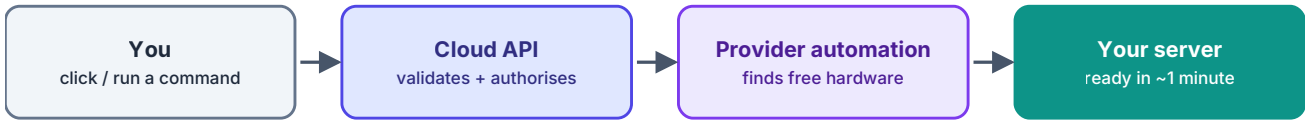


Figure 1.2a — The five essential characteristics of cloud (NIST definition)



No purchase order, no shipping, no rack-and-stack: the whole flow is automated.

Figure 1.2b — What "on-demand" means in practice

Why companies move to cloud

- **Speed** — experiment in minutes, not months.
- **Cost model** — no big upfront spend.
- **Global reach** — deploy near users worldwide.
- **Reliability** — built-in redundancy.
- **Managed services** — the provider runs databases, ML, analytics for you.

Trade-offs to be honest about

- Ongoing bills can grow if unmanaged.
- Less control over underlying hardware.
- Security is **shared** (see 1.9).
- Dependence on the provider (vendor lock-in, 8.2).
- Needs new skills (IAM, networking, cost control).

REAL-WORLD ANALOGY

Electricity: you do not build a power plant, you plug in and pay for the units you use. Cloud is the same for computing.

1.3 On-Premises vs Cloud

★★★

IN ONE LINE

Owning and running your own hardware versus renting it from a provider.

On-premises (on-prem) means the company owns and operates its servers in its own building or leased space. **Cloud** means renting equivalent capacity from a provider. Most real companies today use both (see hybrid, 1.6).

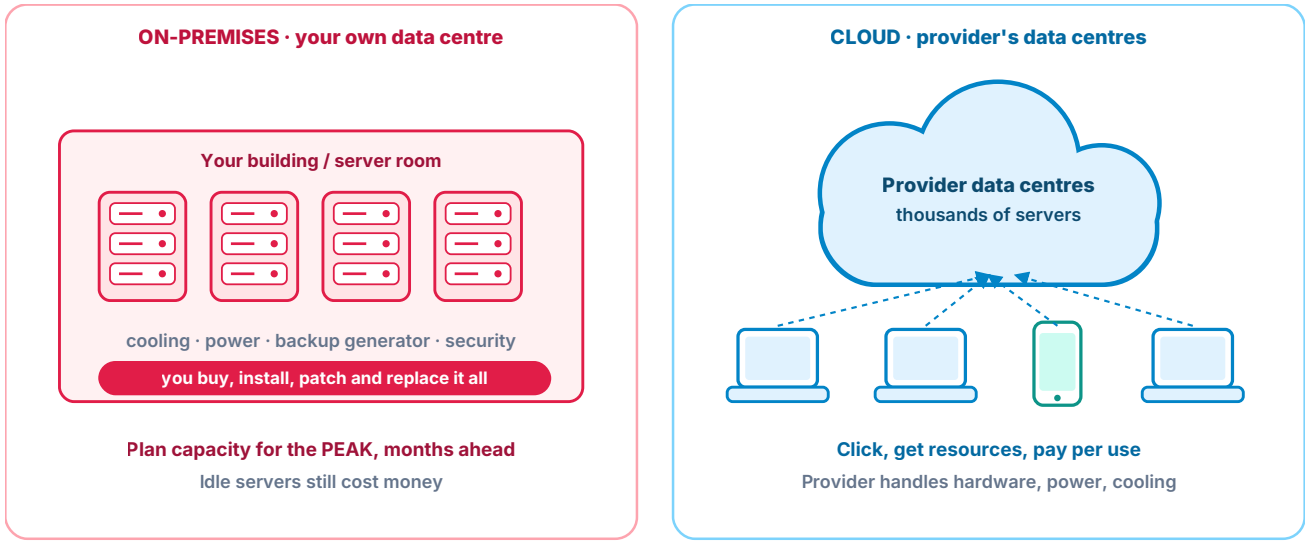


Figure 1.3 — Who owns and runs the hardware?

	On-premises	Cloud
Upfront cost	High (buy servers, build the room)	Near zero
Setup time	Weeks to months (procurement, install)	Minutes
Scaling	Buy more hardware; slow, lumpy	Instant, automatic, granular
Maintenance	Your team: patches, repairs, upgrades	Provider handles hardware and facility
Control	**Full** control of hardware and data location	Shared; you control configuration, not the hardware
Security / compliance	You are fully responsible	Shared responsibility (1.9)
Cost shape	Fixed, whether used or not	Variable, follows usage
Good fit	Strict data-residency rules, stable predictable load, legacy systems	Variable load, fast growth, startups, global apps, experiments

INTERVIEW ANGLE

Do not answer “cloud is always better”. A balanced line: **cloud wins on agility and variable workloads; on-prem can win on cost for steady, predictable, long-running workloads or when regulations demand physical control.**

1.4 CapEx vs OpEx

★★★

IN ONE LINE

Capital expenditure buys assets up front; operating expenditure pays for usage as it happens.

Two ways of paying for IT. **CapEx** (capital expenditure) is a large up-front purchase of an asset that is used for years. **OpEx** (operating expenditure) is a recurring cost for running the business. Cloud turns IT from CapEx into OpEx.

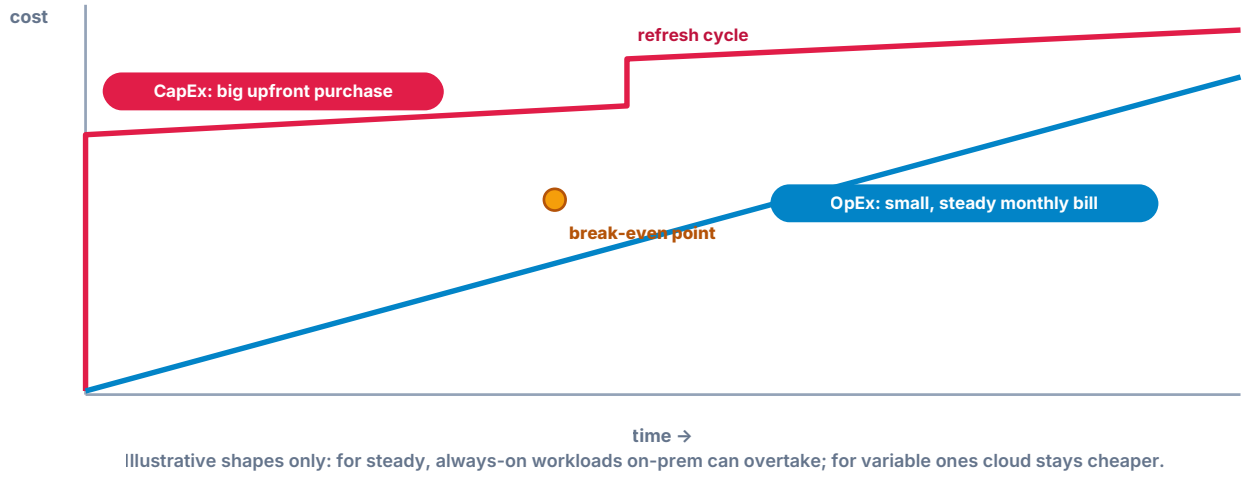


Figure 1.4 — Cost shape over time: up-front purchase vs pay-as-you-go

	CapEx	OpEx
Meaning	Buy assets: servers, network gear, building	Pay for services as you consume them
Cash flow	Large payment up front	Smaller regular payments
Accounting	Capitalised and **depreciated** over years	Expensed in the period incurred
Risk	You carry it: over-buy and waste, under-buy and run out	Provider carries capacity risk
Flexibility	Locked in for the asset's life	Stop paying when you stop using
Typical example	Buying 50 servers for a data centre	Monthly cloud bill, SaaS subscription

REAL-WORLD ANALOGY

Buying a car (CapEx: big payment, you maintain it, it loses value) vs using a ride-hailing app (OpEx: pay per trip, no maintenance).

COMMON TRAP

OpEx is not automatically cheaper. A busy, always-on workload can cost more in the cloud than owned hardware. The real benefit is **flexibility and lower risk**, which is why finance teams like OpEx for uncertain projects.

1.5 Pay-as-you-go Pricing

★★★

IN ONE LINE

Billing based on actual consumption, metered by the second, GB or request.

In **pay-as-you-go** (utility) pricing you are billed for what you actually consume — seconds of compute, GB-months of storage, GB of data transferred, number of requests — instead of a flat fee for capacity you may not use.

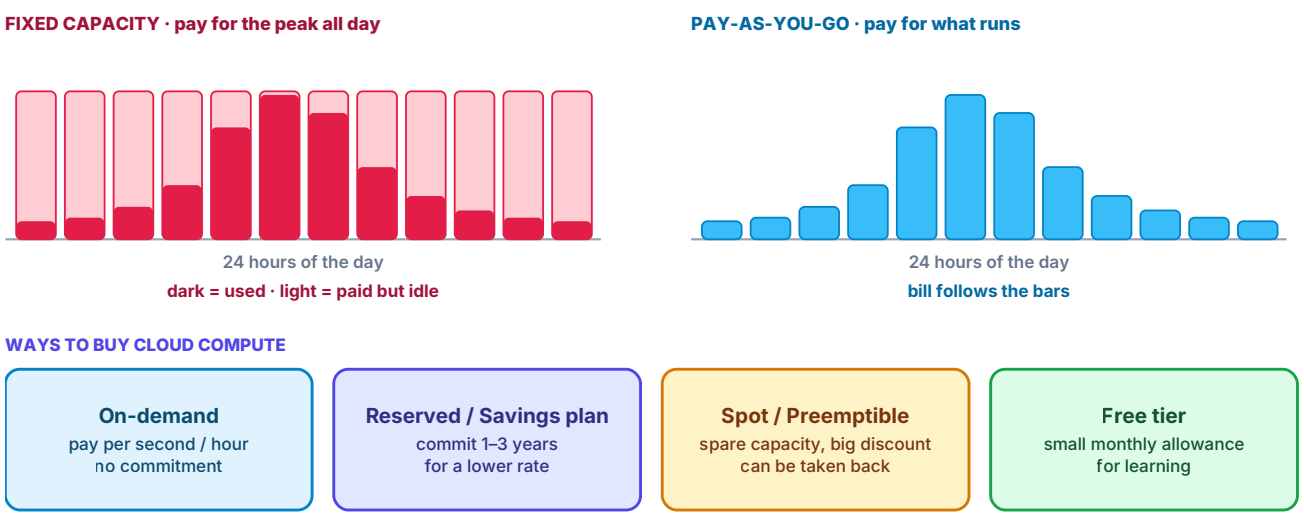


Figure 1.5 — Fixed capacity vs metered usage, and the common purchase options

What is metered	Typical unit	Example
Compute	per second / hour a VM runs	A VM running 10 hours costs for 10 hours
Storage	per GB per month	500 GB in object storage for a month
Data transfer	per GB **leaving** the cloud (egress)	Inbound is usually free; outbound is charged
Requests / executions	per million requests	Serverless function invocations, API calls
Managed service	per hour or per unit of throughput	A database instance, query bytes scanned

COMMON TRAP

"Pay for what you use" also means **pay for what you forget to turn off**. A forgotten VM, an unattached disk or a public-facing data egress can quietly build a large bill. Set budgets and alerts from day one (see cost optimisation, 8.1).

INTERVIEW ANGLE

Know the three levers of the bill: **compute time**, **storage volume**, and **data egress**. Reserved or savings commitments suit steady workloads; spot capacity suits fault-tolerant batch jobs such as Spark ETL.

1.6 Public vs Private vs Hybrid Cloud

★★★

IN ONE LINE

Deployment models: who owns the infrastructure, and who shares it.

These describe **who the infrastructure belongs to and who shares it** — the deployment model.

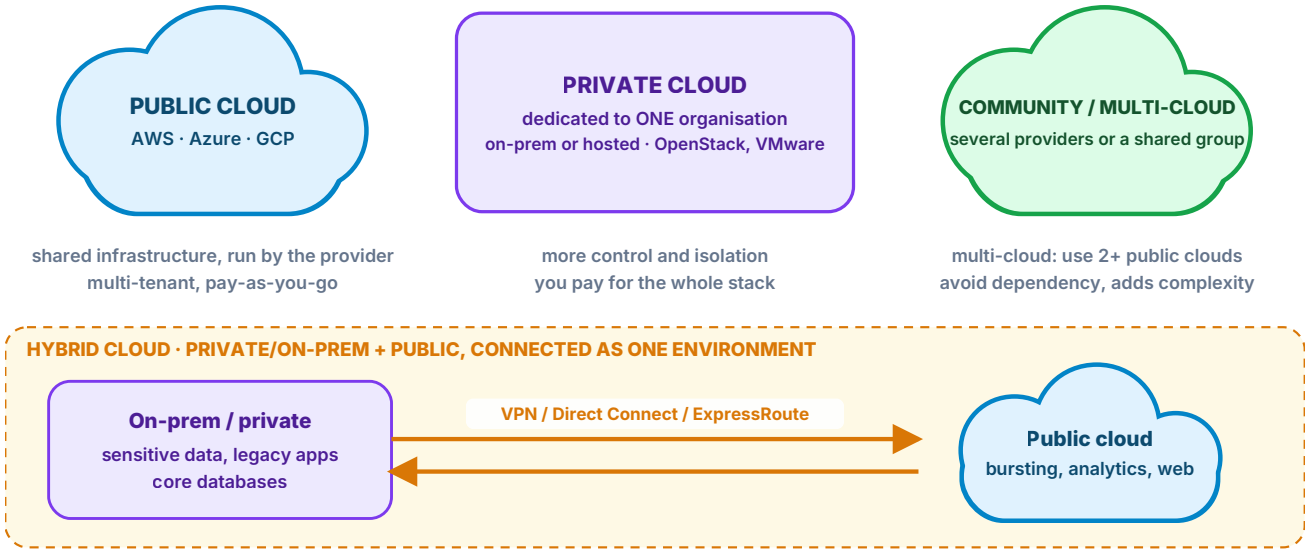


Figure 1.6 — Public, private, hybrid (and multi-cloud)

	Public cloud	Private cloud	Hybrid cloud
Owner / operator	Cloud provider	One organisation (or hosted exclusively for it)	Mix of both, connected
Who shares it	Many customers (multi-tenant)	Only you	Depends on the part
Cost	Pay-as-you-go, no capex	High: hardware + staff	Balanced
Scalability	Practically unlimited	Limited to your hardware	Burst into the public side
Control / compliance	Lower (shared model)	**Highest**	Keep regulated data private
Example use	Startups, web apps, analytics	Banks, defence, healthcare records	Core DB on-prem, analytics and web in cloud

INTERVIEW ANGLE

Multi-cloud (using AWS *and* Azure) is different from **hybrid** (on-prem *and* cloud). Interviewers like checking that you know the difference. Real-life reasons for hybrid: data-residency laws, existing investments, low-latency to factory floors, gradual migration.

1.7 IaaS vs PaaS vs SaaS

★★★

IN ONE LINE

Service models that differ in how much of the technology stack the provider manages.

The three **service models** describe **how much of the stack the provider manages for you**. Moving from IaaS to SaaS you give up control and get less operational work.

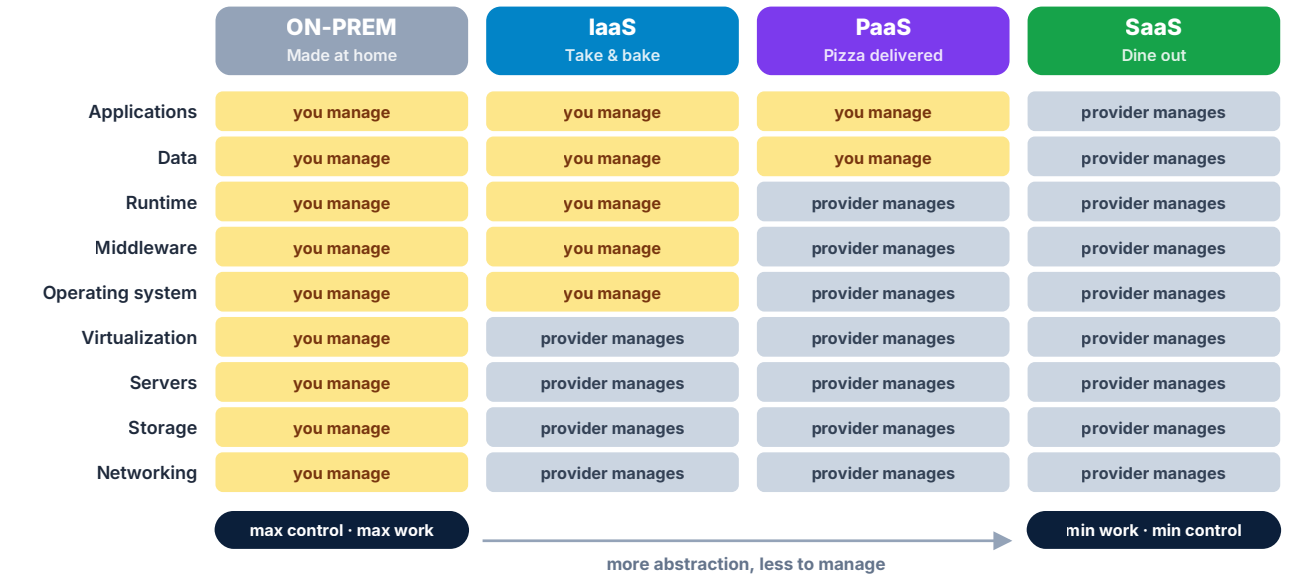


Figure 1.7a — "Pizza as a Service": who manages which layer

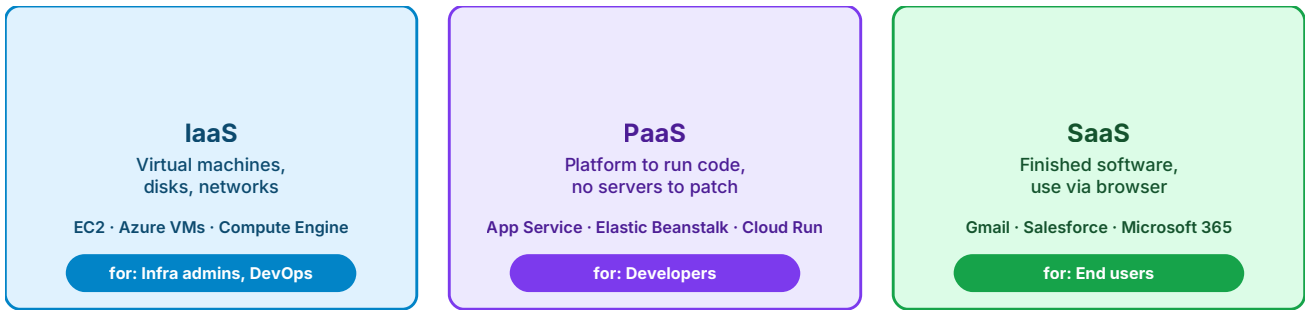


Figure 1.7b — The three models with examples

	IaaS	PaaS	SaaS
You get	Raw building blocks: VMs, storage, networks	A ready platform to deploy code or data	A complete application
You manage	OS, runtime, apps, data, security config	Your application and data	Only your usage / settings
Provider manages	Hardware, virtualisation, facilities	+ OS, runtime, scaling, patching	Everything
Flexibility	Highest	Medium	Lowest
Examples	EC2, Azure VM, Compute Engine, S3	Heroku, Azure App Service, Cloud Run, managed databases	Gmail, Slack, Salesforce, Google Workspace
Persona	Sysadmin / DevOps	Developer / data engineer	Business user

INTERVIEW ANGLE

Other "aaS" terms to recognise: **FaaS** (functions as a service — serverless, 2.6), **DBaaS** (managed databases), **CaaS** (containers as a service). Be ready to **classify examples**: S3 = IaaS-style storage service, Snowflake = SaaS/PaaS-style data platform, Databricks = PaaS/SaaS depending on view — say your reasoning, not just the label.

1.8 Cloud Service Provider (CSP)

★★☆

IN ONE LINE

A company that runs global data centres and rents computing, storage and services to customers.

A **Cloud Service Provider (CSP)** is a company that owns data centres around the world and rents out computing services over the internet. The "big three" are **AWS, Microsoft Azure and Google Cloud**.

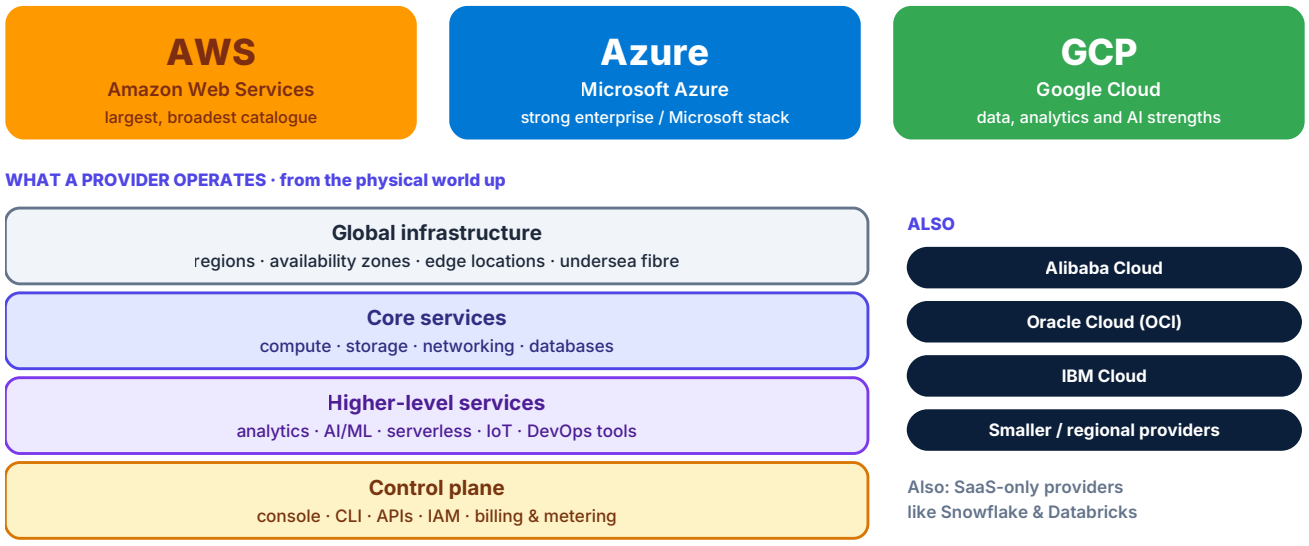


Figure 1.8 — The major providers and what they run

Category	AWS	Azure	Google Cloud
Virtual machines	EC2	Virtual Machines	Compute Engine
Object storage	S3	Blob Storage	Cloud Storage
Serverless functions	Lambda	Functions	Cloud Functions / Cloud Run
Managed Kubernetes	EKS	AKS	GKE
Identity & access	IAM	Entra ID + Azure RBAC	Cloud IAM
Data warehouse	Redshift	Synapse / Fabric	BigQuery

INTERVIEW ANGLE

Services are **equivalent across providers with different names**. Learn one provider deeply and map the concepts to the others — the full cheat sheet is in 8.6. At the fresher level, interviewers care that you understand the *concept* (object storage, VM, serverless), not memorised brand names.

How you talk to a CSP

- **Web console** (point and click)
- **CLI** (`aws`, `az`, `gcloud`)
- **SDKs** (Python `boto3`, etc.)
- **Infrastructure as Code** (Terraform, 7.5)

Choosing a provider

- Existing skills and contracts
- Services you need (e.g. AI, analytics)
- Regions and compliance coverage
- Pricing and support
- Ecosystem and community

1.9 Shared Responsibility Model

★★★

IN ONE LINE

The provider secures the infrastructure; you secure your data, access and configuration.

Security in the cloud is a **partnership**. The provider secures the cloud itself; **you** secure what you put in it and how you configure it. Most real-world cloud breaches are caused by **customer misconfiguration** (e.g. a public bucket, an over-permissive role), not by the provider being hacked.

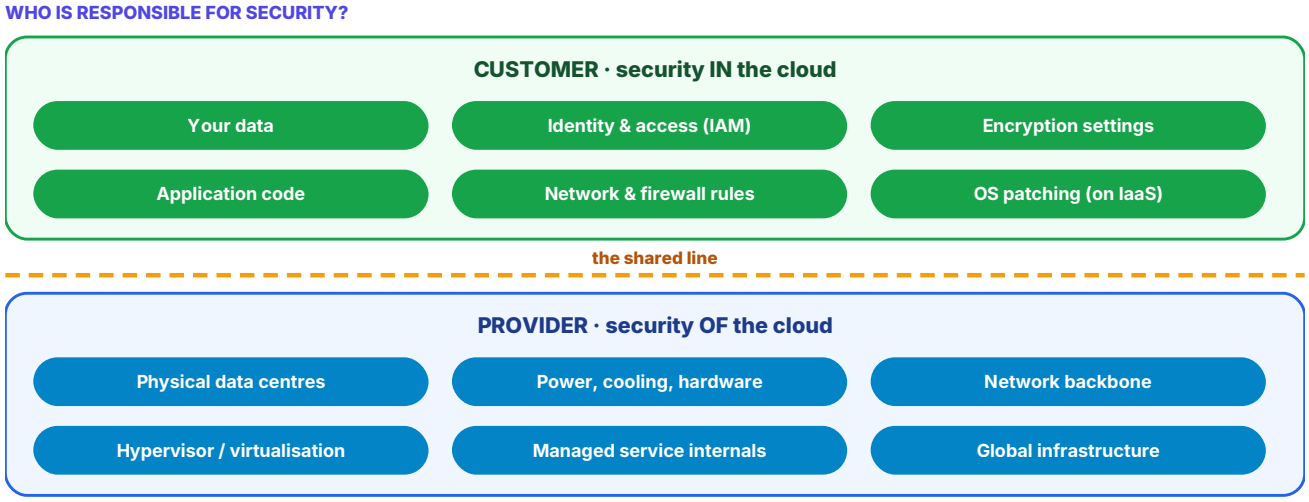


Figure 1.9a — Security OF the cloud vs security IN the cloud

	Data & access	Application	OS & runtime	Virtualisation	Physical / network
On-prem	customer	customer	customer	customer	customer
IaaS	customer	customer	customer	customer	provider
PaaS	customer	customer	provider	provider	provider
SaaS	customer	provider	provider	provider	provider

Figure 1.9b — The split changes with the service model

Example	Who is responsible?
A fire in the data centre; failed disk hardware	**Provider**
Patching the hypervisor	**Provider**
Patching the OS on your EC2 / Azure VM	**You** (IaaS)
Patching the OS under a managed database or serverless function	**Provider** (PaaS / serverless)
Who is allowed to read your S3 bucket	**You** (always)
Enabling MFA and strong passwords for your users	**You** (always)
Encrypting your data and managing the keys	**You** choose and configure; provider supplies the tools

REMEMBER

A simple rule: **the provider is responsible for the cloud; the customer is responsible for everything they put in it** — always your data, identities, access settings and configuration.

02

Part 2 · Compute & Scaling

How workloads run: virtual machines, containers, serverless, and how capacity grows and shrinks.

- Virtualization · Hypervisor · VM
- Containers vs VMs · Kubernetes
- Serverless
- Scalability vs Elasticity
- Vertical vs Horizontal Scaling
- Auto Scaling

2.1 Virtualization

★★★

IN ONE LINE

Software technology that splits one physical machine into many isolated virtual ones.

Virtualization uses software to turn one physical machine into **several isolated virtual machines**, each behaving like a real computer with its own OS. It is the technology that makes the cloud economical: providers split huge servers among many customers.

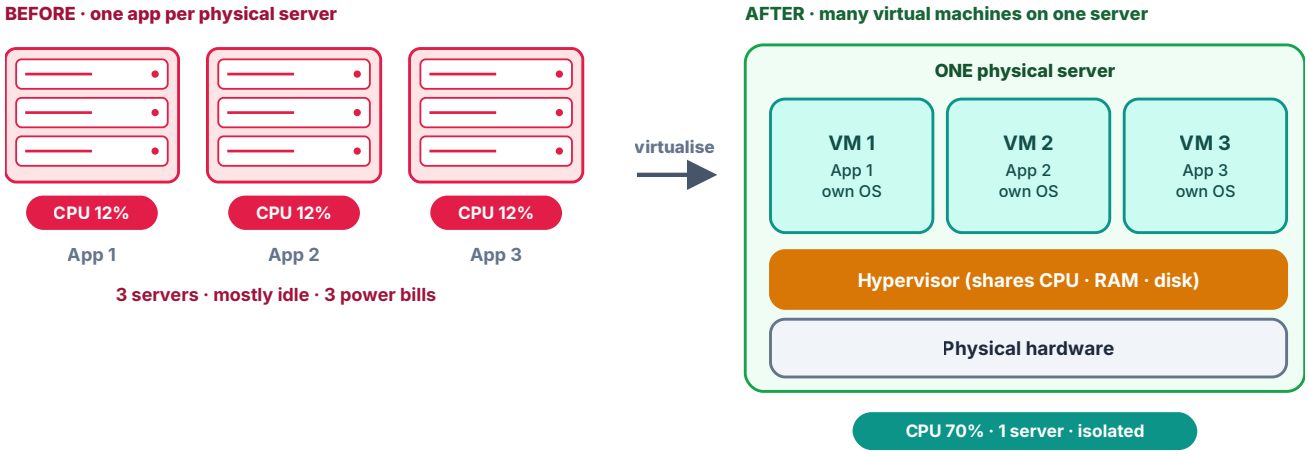


Figure 2.1 — From one-app-per-server to many VMs per server

Benefits

- **Higher utilisation** — less idle hardware.
- **Isolation** — a crash in one VM does not affect another.
- **Speed** — create a VM in seconds from an image.
- **Portability** — move or snapshot a VM.
- **Cost** — fewer physical servers.

Kinds of virtualization

- **Server** virtualisation (VMs) — this topic.
- **Storage** — pool many disks into one logical volume.
- **Network** — virtual networks and switches (VPC, 4.6).
- **Desktop** — remote virtual desktops (VDI).
- **OS-level** — containers (2.4).

REAL-WORLD ANALOGY

An apartment building: one physical building (server) divided into private flats (VMs). Residents share the foundations and plumbing but cannot walk into each other's homes.

2.2 Hypervisor

★★★

IN ONE LINE

The software layer that creates and manages virtual machines on shared hardware.

A **hypervisor** (virtual machine monitor) is the software layer that **creates, runs and isolates virtual machines**, handing each one a share of the physical CPU, memory, storage and network. It is the heart of virtualisation.

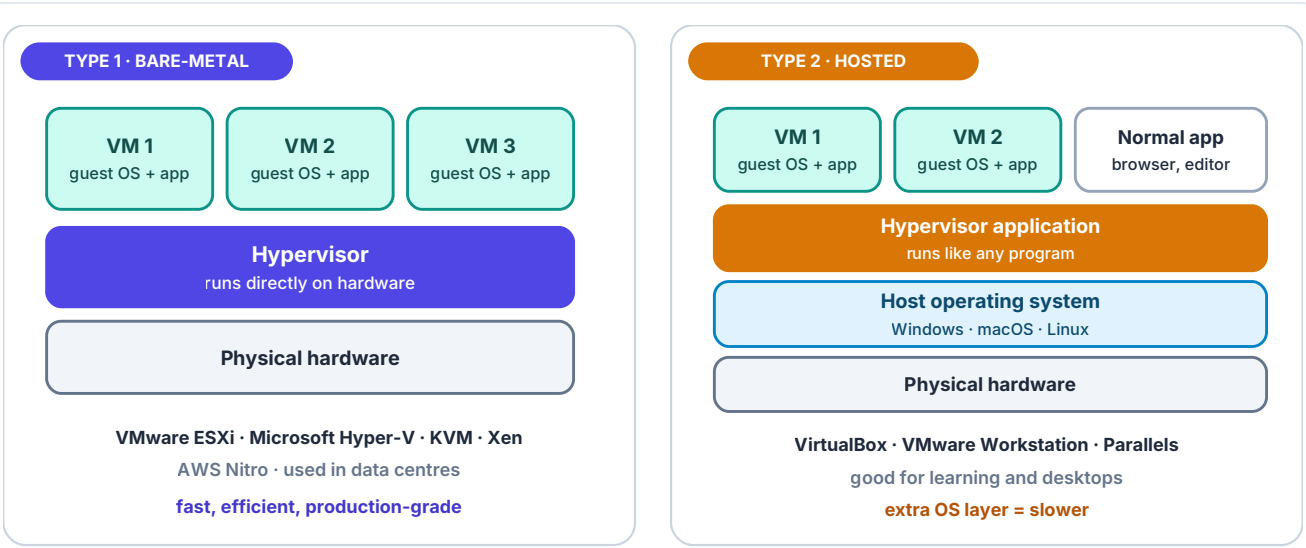


Figure 2.2 — Type 1 (bare-metal) vs Type 2 (hosted) hypervisors

	Type 1 (bare-metal)	Type 2 (hosted)
Runs on	Directly on hardware	On top of a host OS
Performance	High, low overhead	Lower (extra OS layer)
Used in	Cloud and enterprise data centres	Developer laptops, labs
Examples	ESXi, Hyper-V, KVM, Xen	VirtualBox, VMware Workstation

INTERVIEW ANGLE

Public clouds use type 1 hypervisors (AWS uses a KVM-based “Nitro” system, Azure uses Hyper-V, GCP uses KVM). The hypervisor is the part of the **shared responsibility model** that is always the provider’s job. If asked “what stops one customer’s VM from reading another’s memory?” — **hypervisor isolation**.

2.3 Virtual Machine (VM)

★★★

IN ONE LINE

A software-defined computer with its own OS, running on shared physical hardware.

A **virtual machine (VM)** is a software-defined computer: it has virtual CPUs, memory, disk and network, runs its own full operating system, and behaves like a physical server. In the cloud it is called an **instance** (EC2 instance, Azure VM, Compute Engine instance).

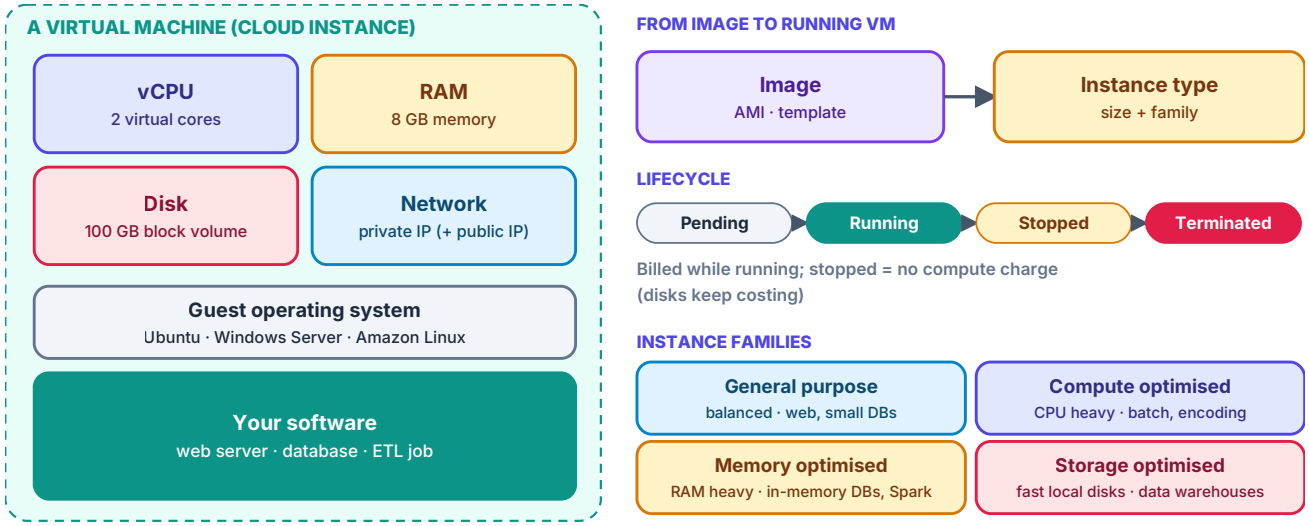


Figure 2.3 — Anatomy, image, lifecycle and families of a cloud VM

Term	Meaning
Image (AMI, VM image)	A template: OS + pre-installed software. Launch many identical VMs from one image.
Instance type	The size/shape: how many vCPUs and how much RAM (e.g. `m5.large`).
Snapshot	A point-in-time copy of a disk, used for backup and cloning.
Key pair / SSH	How you log in to a Linux VM securely, without a password (port 22).
User data / bootstrap	A script that runs on first boot to install and configure software automatically.

DATA ENGINEERING TIE-IN

Data engineers use VMs to host self-managed tools (Airflow, Kafka, Spark clusters) and pick **memory-optimised** instances for Spark and **storage-optimised** ones for database nodes. Managed services remove this VM work — that is the PaaS trade-off.

2.4 Container vs Virtual Machine

★★★

IN ONE LINE

Containers share the host OS and are lightweight; VMs carry a full OS each and isolate more strongly.

Both isolate applications, but at different levels. A VM virtualises the **hardware** (each VM carries a full OS). A container virtualises the **operating system** (all containers share the host OS kernel and package only the app and its libraries).

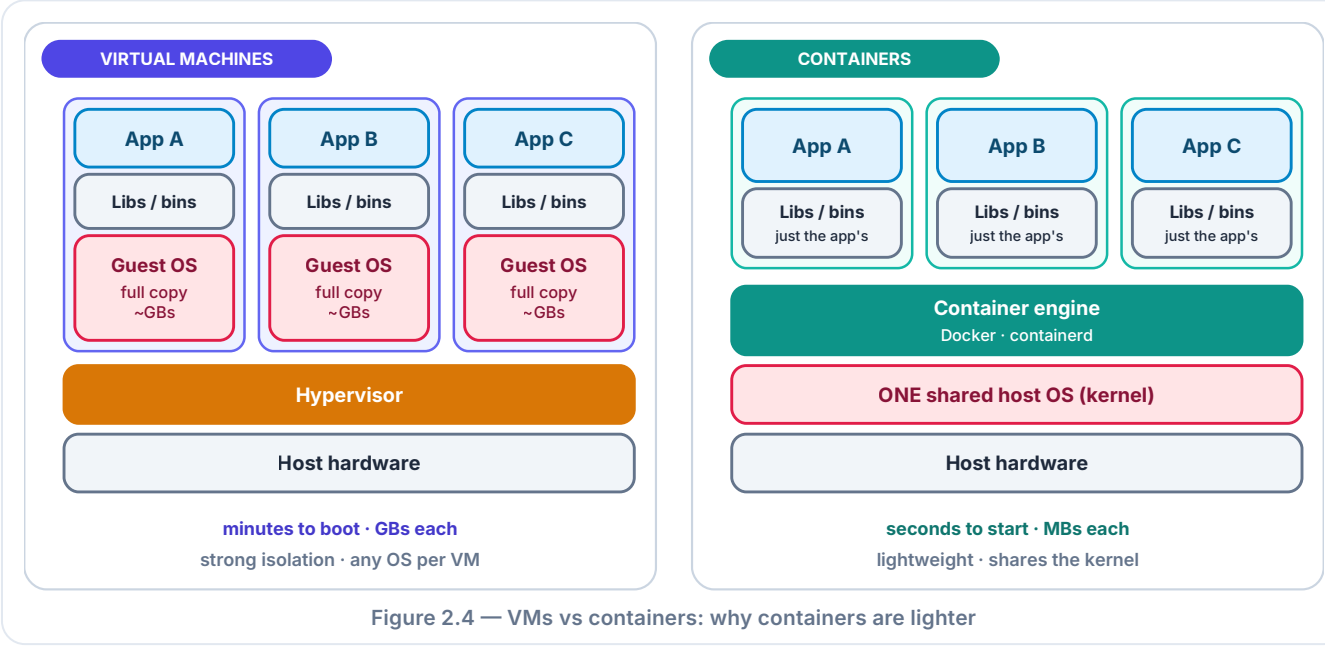


Figure 2.4 — VMs vs containers: why containers are lighter

	Virtual machine	Container
Isolation level	Hardware (strong)	Process / OS (lighter)
Size	GBs (includes OS)	MBs
Start-up time	Minutes	Seconds or less
Density	Tens per server	Hundreds per server
OS flexibility	Any OS per VM	Must match host kernel (Linux on Linux)
Best for	Legacy apps, strong isolation, different OSes	Microservices, CI/CD, portable apps, bursts of workers
Tools	VMware, Hyper-V, EC2	Docker, Podman, Kubernetes

Docker in 30 seconds

Key words

- **Dockerfile** — recipe for building an image.
- **Image** — read-only template (app + dependencies).
- **Container** — a running instance of an image.
- **Registry** — where images live (Docker Hub, ECR, ACR).

DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "etl_job.py"]
```

INTERVIEW ANGLE

"It works on my machine" is the classic problem containers solve: the **same image** runs identically on a laptop, in CI and in production. They are not a replacement for VMs — in the cloud, containers usually run *on* VMs.

2.5 Container Orchestration & Kubernetes

★★☆

IN ONE LINE

Automating deployment, scaling and healing of many containers across many machines.

Running one container is easy. Running **hundreds across many machines** — restarting failed ones, scaling, rolling out updates — needs an **orchestrator**. **Kubernetes (K8s)** is the industry standard. Managed versions: **EKS** (AWS), **AKS** (Azure), **GKE** (Google).

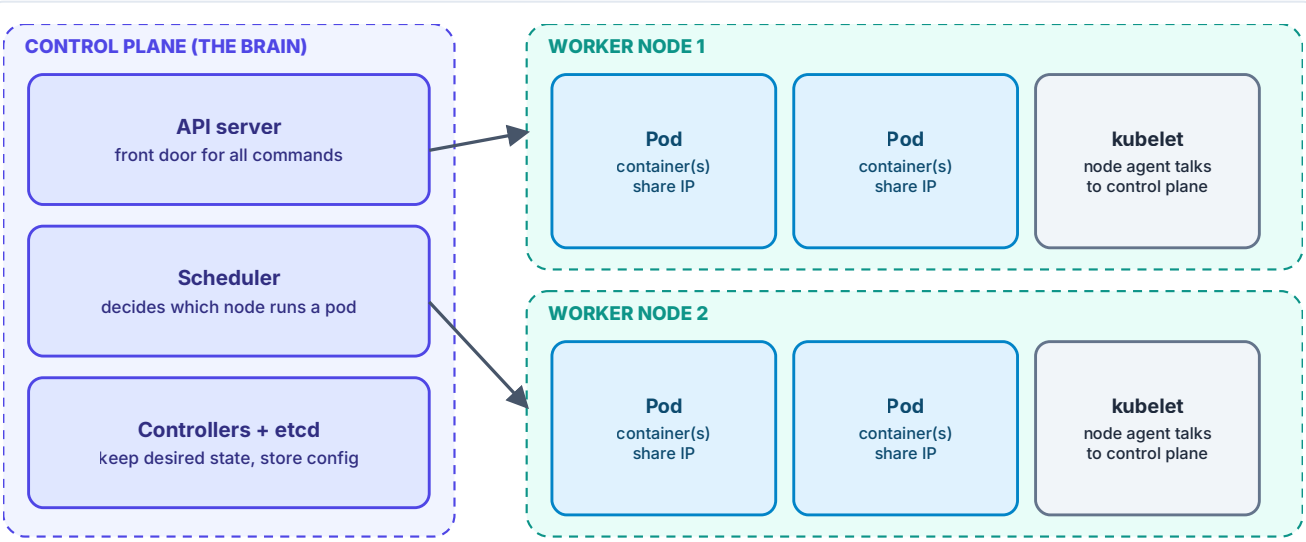


Figure 2.5 — Kubernetes: control plane plus worker nodes running pods

Concept	Meaning
Pod	Smallest deployable unit: one or more containers sharing network and storage.
Node	A worker machine (VM) that runs pods.
Deployment	Declares "keep N replicas of this pod running" and handles rolling updates.
Service	A stable address / load balancer in front of changing pods.
Desired state	You declare what you want; Kubernetes continuously works to make reality match (self-healing).

DATA ENGINEERING TIE-IN

Spark on Kubernetes, Airflow on Kubernetes (KubernetesExecutor) and containerised ETL jobs are common. At fresher level, knowing **what problem Kubernetes solves** matters more than writing manifests.

2.6 Serverless Computing & Serverless Architecture

★★★

IN ONE LINE

Run code in response to events without managing servers; pay only while it executes.

Serverless means you write code and the provider runs it **on demand**: it provisions, scales, patches and bills per execution. You never see or manage the servers (they exist, but they are the provider's problem). The compute flavour is **FaaS** (Functions as a Service).

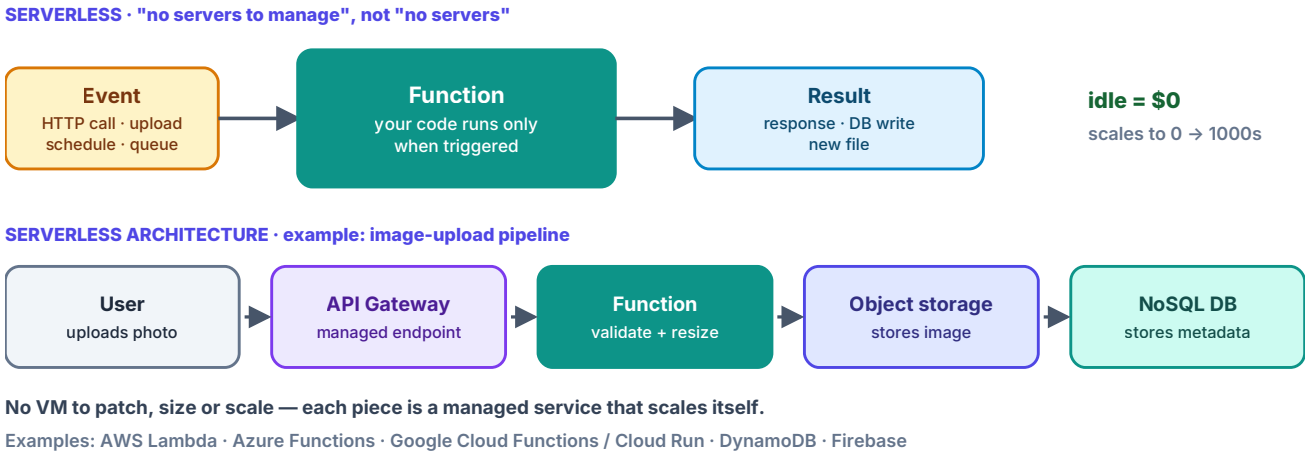


Figure 2.6 — A function triggered by events, and a full serverless architecture

Advantages

- **No server management**.
- **Auto-scaling** from zero to thousands.
- **Pay per use**, nothing when idle.
- Fast to build and ship.
- Naturally event-driven.

Limitations

- **Cold start** — first call after idle is slower.
- **Time limits** (e.g. 15 min on Lambda) — not for long jobs.
- Stateless: keep state in a DB or storage.
- Harder local testing and debugging.
- Costly at very high, constant load.

AWS LAMBDA · PYTHON

```python
import json

def lambda_handler(event, context):
    # triggered when a file lands in the bucket
    for rec in event['Records']:
        key = rec['s3']['object']['key']
        print(f'new file: {key}')
    return {'statusCode': 200, 'body': json.dumps('ok')}
```

DATA ENGINEERING TIE-IN

A classic pipeline: a file lands in object storage → an event triggers a function → it validates the file and starts an ETL job or loads a table. Serverless also covers **serverless databases and warehouses** (BigQuery, Aurora Serverless) where you do not size clusters.

COMMON TRAP

"Serverless" is not "no servers" and not always cheaper. For constant heavy traffic, always-on containers or VMs can cost less.

2.7 Scalability vs Elasticity

★★★

IN ONE LINE

Scalability is the ability to handle growth; elasticity is doing so automatically, in both directions.

Scalability is the ability of a system to handle **growth** by adding resources. **Elasticity** is the ability to **automatically add and remove** resources as demand changes, so you never pay for idle capacity. Elasticity = scalability + automation + shrinking back down.

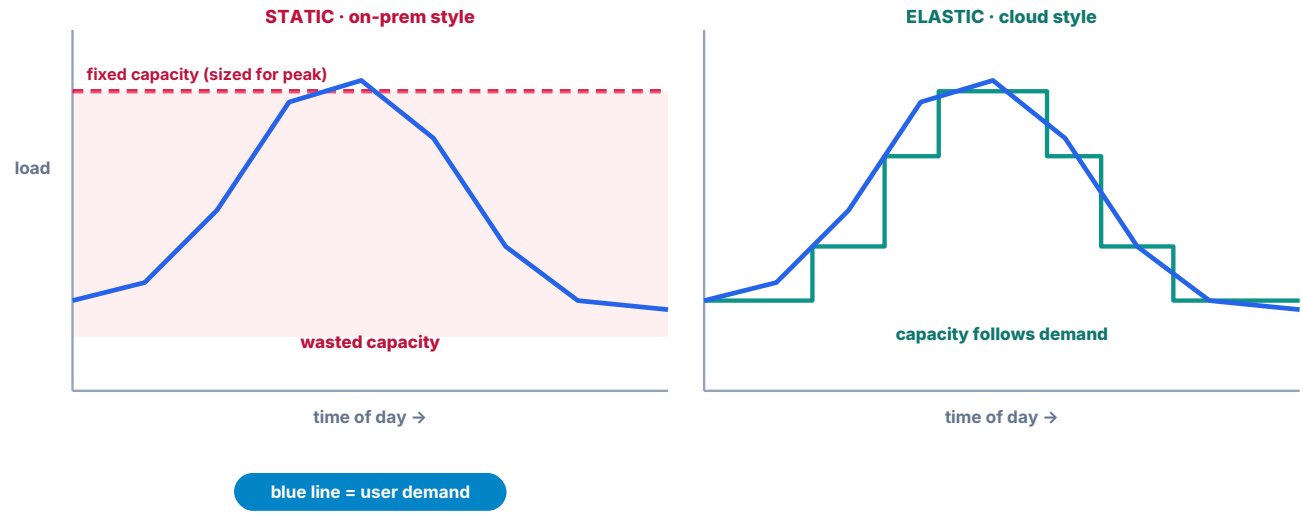


Figure 2.7 — Capacity vs demand over a day

	Scalability	Elasticity
Question it answers	"Can the system handle more load?"	"Can it grow *and shrink* automatically?"
Direction	Mostly growth over the long term	Up and down in near real time
Mechanism	Design: stateless services, partitioned data	Automation: auto scaling, serverless
Time scale	Planned growth, months / years	Minutes / seconds, spikes and quiet hours
Example	Architecture can support 10x users after redesign	Online shop adds servers on sale day, removes them after

REAL-WORLD ANALOGY

A restaurant that can add more tables when it expands (scalable) vs one that brings in extra staff during the dinner rush and sends them home after (elastic).

2.8 Vertical vs Horizontal Scaling

★★★

IN ONE LINE

Scale up by making a machine bigger; scale out by adding more machines.

The two ways to add capacity. **Vertical scaling (scale up)** makes one machine bigger. **Horizontal scaling (scale out)** adds more machines and spreads the work across them.

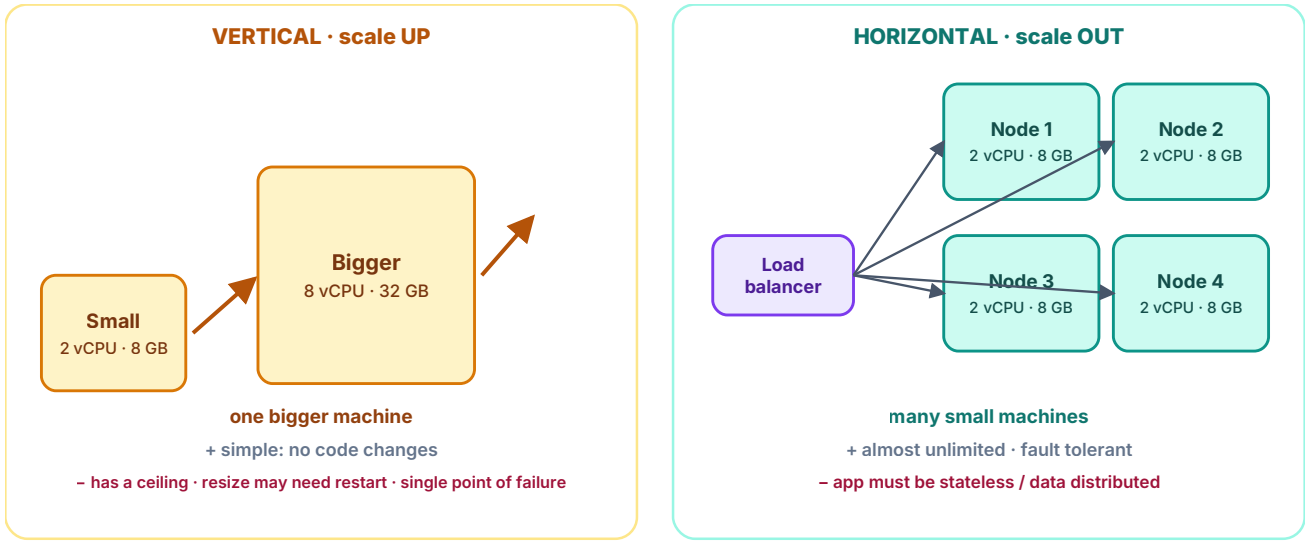


Figure 2.8 — Scale up (bigger) vs scale out (more)

	Vertical (up / down)	Horizontal (out / in)
How	Resize to a bigger instance type	Add or remove instances
Limit	Biggest machine available	Practically none
Downtime	Often needs a restart	None: add nodes while running
Fault tolerance	Single point of failure	Survives node failures
Complexity	Simple, no app changes	Needs load balancer + stateless design
Cost curve	Gets expensive at the top end	Roughly linear, uses commodity machines
Typical use	Relational databases, quick fixes	Web servers, Spark workers, microservices, Kafka brokers

REMEMBER

Scaling **up/down** = change the size of one machine. Scaling **out/in** = change the number of machines. Cloud designs prefer **out** because it is elastic and resilient.

DATA ENGINEERING TIE-IN

Big-data engines such as Spark, Kafka and Hadoop are built to scale **horizontally** — that is the whole idea of distributed processing. Traditional relational databases often scale vertically first, then use read replicas or sharding.

2.9 Auto Scaling

★★★

IN ONE LINE

Automatically adding and removing capacity based on demand, within limits you define.

Auto Scaling is the cloud feature that implements elasticity: it watches a metric (CPU, request count, queue length) and **automatically adds or removes instances** within limits you set. Examples: AWS Auto Scaling groups, Azure VM Scale Sets, GCP Managed Instance Groups.

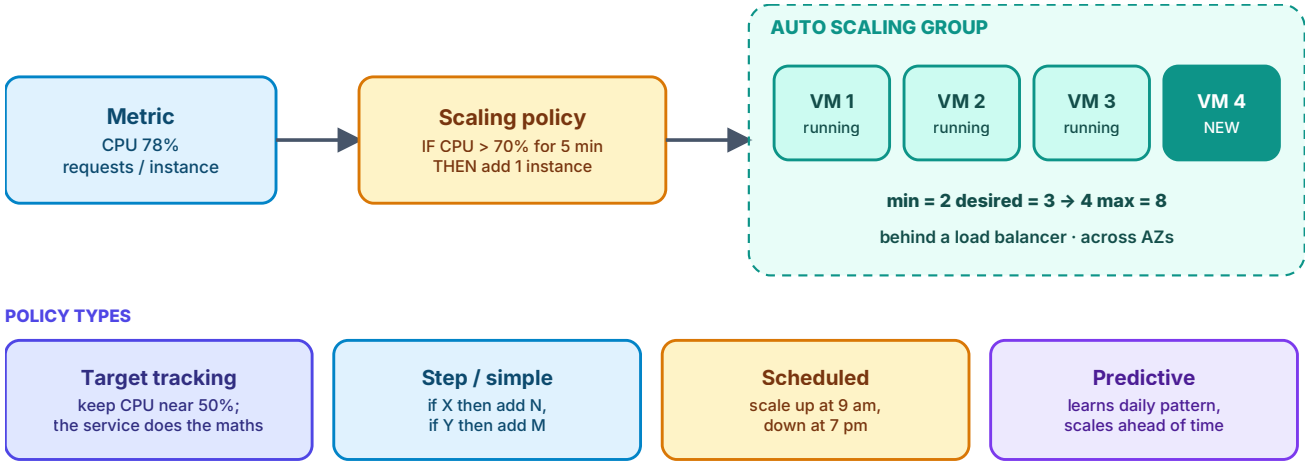


Figure 2.9 — Metric → policy → scale out, with min / desired / max

Setting	Meaning
Min	Never go below this many instances (guarantees baseline availability).
Desired	How many should be running right now; the group moves it up and down.
Max	Hard cap, protects you from runaway cost.
Health check	Unhealthy instances are terminated and replaced automatically (self-healing).
Cooldown	Wait time after a scaling action before the next one, to avoid thrashing.
Launch template	Describes the VM to launch: image, size, security group, startup script.

INTERVIEW ANGLE

Auto scaling needs **stateless instances** (no data stored only on the VM) and a **load balancer** to spread traffic over the changing set of VMs. Putting the group across **multiple Availability Zones** gives scaling *and* high availability at once.

COMMON TRAP

Always set a sensible **max** and a budget alert. A bug or traffic attack that triggers endless scale-out becomes a very large bill.

03

Part 3 · Storage & Databases

Where data lives in the cloud and how it is kept safe, durable and recoverable.

- Object · Block · File storage
- Buckets
- Durability vs Availability
- Backup vs Replication
- Managed databases

3.1 Object Storage

★★★

IN ONE LINE

Flat, scalable storage of data as objects (data + metadata + key) accessed through an HTTP API.

Object storage stores data as **objects** — the data itself, its metadata and a unique key — in a flat namespace, accessed over HTTP APIs. It is almost infinitely scalable, very cheap and extremely durable. Examples: **Amazon S3, Azure Blob Storage, Google Cloud Storage**.

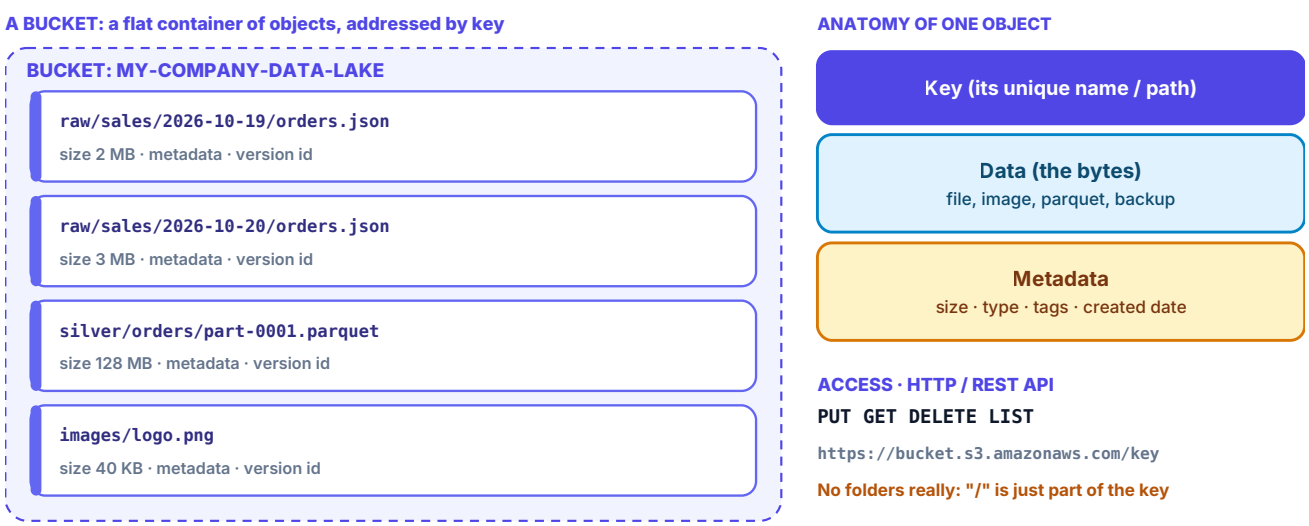


Figure 3.1 — Objects live in buckets and are addressed by key

Great for

- Backups and archives
- Images, video, documents
- **Data lakes** (Parquet, JSON, CSV)
- Static website hosting
- Logs and ML training data

Not great for

- Operating system disks
- Databases needing small random writes
- Frequently **editing part of a file** (objects are replaced whole)
- Low-latency transactional workloads

COMMON TRAP

Objects are **immutable**: to change a byte you re-upload the whole object. That is exactly why lake file formats like Parquet plus table formats (Delta, Iceberg) exist.

DATA ENGINEERING TIE-IN

Object storage is the **foundation of the data lake and lakehouse**. Bronze/Silver/Gold layers are just prefixes in a bucket. Know: S3 = IaaS storage; cheap; decoupled from compute; use partitioned Parquet, avoid millions of tiny files.

3.2 Block Storage

★★★

IN ONE LINE

Raw, fixed-size blocks presented to a server as a fast disk; ideal for OS disks and databases.

Block storage splits data into equal-sized **blocks** and presents them to a server as a raw disk. The OS puts a file system on it, so it behaves like the C: drive or `/dev/sda`. It gives **low latency and high IOPS**, which is why databases and boot disks use it. Examples: **Amazon EBS, Azure Managed Disks, Google Persistent Disk**.

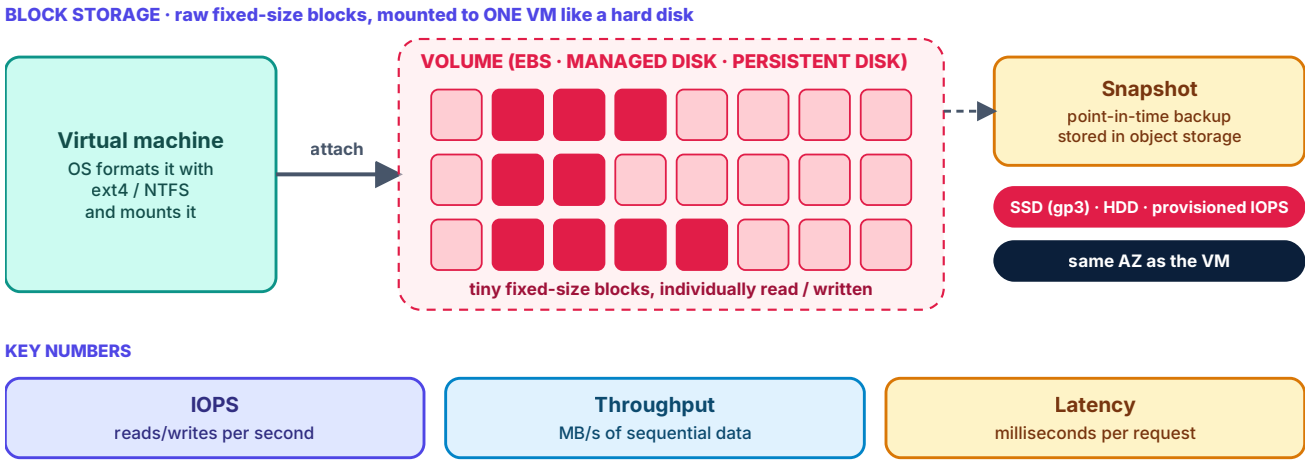


Figure 3.2 — A block volume attached to a VM, with snapshots

Characteristics

- Attached to **one** VM at a time (usually) and in the **same Availability Zone**.
- Persists independently of the VM (data survives a stop).
- Resizable; choose SSD for speed or HDD for cheap throughput.
- Back up with **snapshots**.

Typical uses

- VM boot / OS disks
- Relational database storage (MySQL, PostgreSQL)
- Transaction-heavy and low-latency apps
- Self-managed Kafka / Elasticsearch nodes

COMMON TRAP

Instance store (disks physically inside the host) is *ephemeral*: data is lost when the VM stops or fails. Block volumes (EBS) persist. Never keep the only copy of important data on ephemeral storage.

3.3 File Storage

★★☆

IN ONE LINE

A shared, hierarchical network file system (NFS / SMB) mounted by many servers at once.

File storage organises data in the familiar **hierarchy of folders and files** and lets **many machines share it at once** over a network protocol (**NFS** on Linux, **SMB/CIFS** on Windows). It is a managed "network drive". Examples: **Amazon EFS / FSx, Azure Files, Google Filestore**.

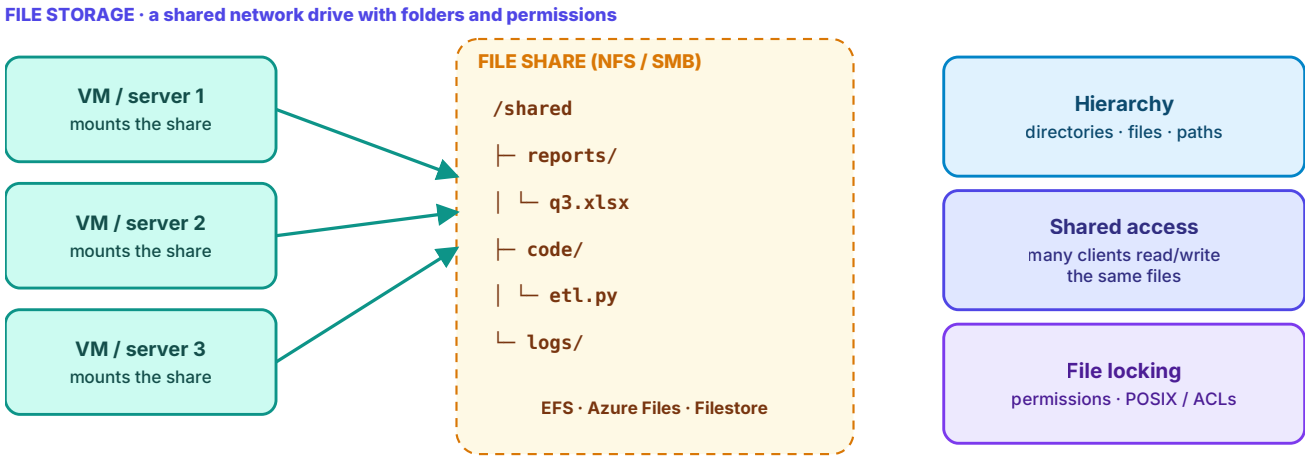


Figure 3.3 — Several servers mounting one shared file system

Typical uses

- Shared content for several web servers
- Home directories, team shares
- Lift-and-shift of apps that expect a file system
- Dev tools, CI build caches

Limits

- More expensive per GB than object storage
- Slower than block storage for databases
- Scaling limits vs object storage
- Needs network mounting (NFS / SMB)

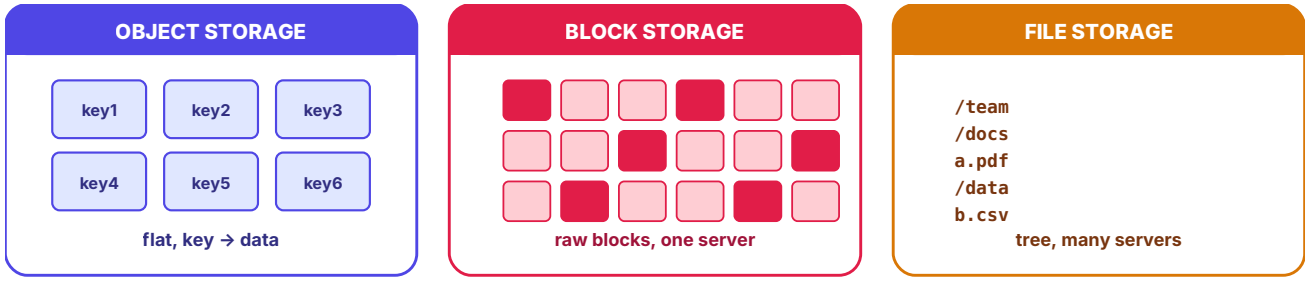
3.4 Object vs Block vs File Storage

★★★

IN ONE LINE

Choosing the right storage type by structure, access pattern, speed and cost.

The three storage types suit different jobs. A good fresher answer picks by **access pattern**: raw speed for one server (block), shared folders (file), or massive cheap scalable storage (object).



	Object	Block	File
Access	HTTP API (REST)	Mounted as a disk	NFS / SMB mount
Latency	Higher	Lowest	Medium
Scale	Virtually unlimited	TBs per volume	Large, shared
Cost per GB	Lowest	Highest	Medium

Figure 3.4a — Side-by-side comparison

WHICH ONE SHOULD I USE?

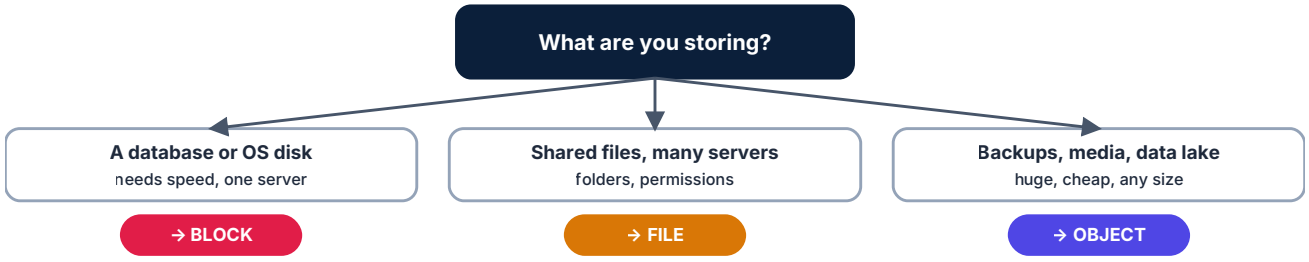


Figure 3.4b — A quick decision guide

	Object	Block	File
Structure	Flat objects with metadata	Fixed-size blocks	Hierarchical files and folders
Shared by many servers?	Yes (HTTP)	Usually one at a time	Yes (NFS / SMB)
Modify part of data?	No, replace whole object	Yes, per block	Yes, per file
Best for	Data lakes, backups, media, static sites	Databases, boot volumes	Shared content, home directories
AWS / Azure / GCP	S3 / Blob / Cloud Storage	EBS / Managed Disks / Persistent Disk	EFS / Azure Files / Filestore

REAL-WORLD ANALOGY

Object = a huge warehouse where every box has a label and you ask for it by label. Block = the hard drive inside your own computer. File = a shared office filing cabinet that everyone opens at once.

3.5 Storage Bucket

★★★

IN ONE LINE

The globally named, region-bound container that holds objects and carries access and lifecycle settings.

A **storage bucket** is the top-level container in object storage. You create a bucket, then upload objects into it. The bucket name is **globally unique**, belongs to a **region**, and is where you set access rules, encryption, versioning and lifecycle policies.

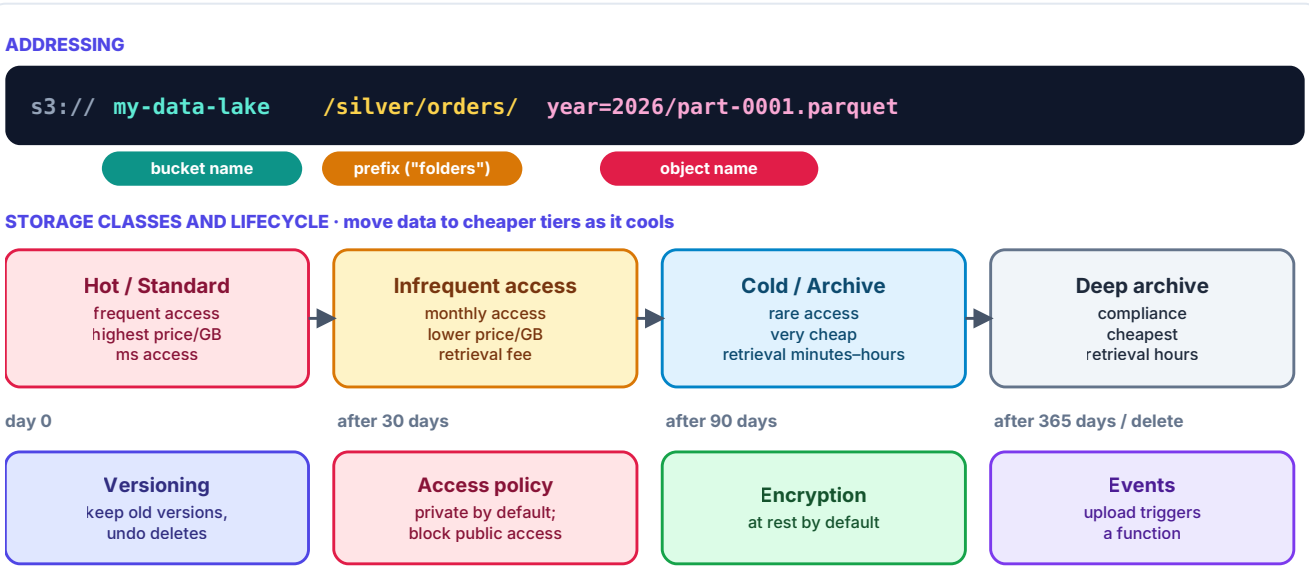


Figure 3.5 — Bucket addressing, storage tiers and key features

Feature	What it does
Region	Data physically stays in the chosen region (important for latency and data-residency laws).
Access control	Bucket policies + IAM + ACLs. **Default private.** Turn on “block public access” unless you are deliberately hosting a public site.
Versioning	Keeps every version of an object so you can recover from overwrites and deletes.
Lifecycle rules	Automatically transition objects to cheaper tiers or delete them after N days.
Replication	Copy objects to another bucket or region for DR or lower read latency.
Event notifications	Trigger functions or queues on upload, e.g. start an ETL job.

COMMON TRAP

Public buckets are the #1 cloud data leak. A misconfigured bucket policy exposes data to the whole internet. Keep buckets private, give access through IAM roles and use short-lived signed URLs for sharing.

DATA ENGINEERING TIE-IN

Lakehouse layout: `s3://lake/bronze/` , `/silver/` , `/gold/` . Use lifecycle rules to age raw Bronze data into cheaper tiers, and partition prefixes (e.g. `year=2026/month=10/`) so engines can prune.

3.6 Data Durability vs Data Availability

★★★

IN ONE LINE

Durability: data is not lost. Availability: data is reachable right now.

Two different promises that are often confused. **Durability** is the probability that stored data is **not lost or corrupted** over time. **Availability** is the percentage of time a service is **up and responding**.

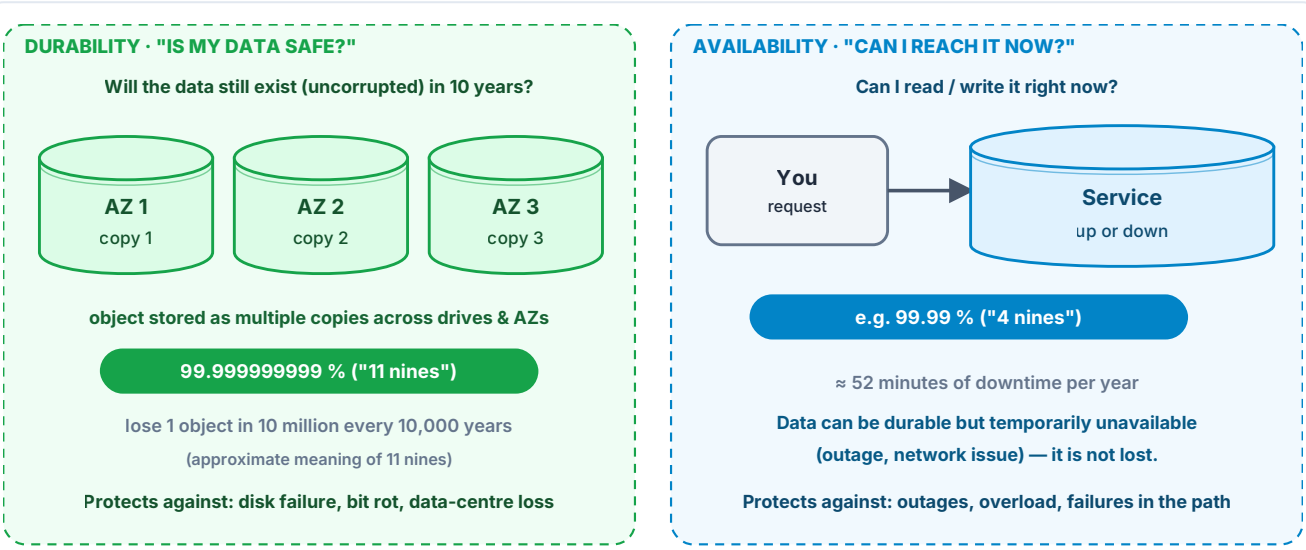


Figure 3.6 — Safe data vs reachable data

Availability	Downtime per year	Per month	Common name
99 %	3.65 days	7.3 hours	two nines
99.9 %	8.76 hours	43.8 minutes	three nines
99.95 %	4.38 hours	21.9 minutes	
99.99 %	52.6 minutes	4.4 minutes	four nines
99.999 %	5.26 minutes	26 seconds	five nines

How durability is achieved

- Store **multiple copies** across disks and Availability Zones.
- Checksums detect and repair corruption.
- Erasure coding and background healing.
- Versioning protects against human error.

How availability is achieved

- Redundant servers behind a load balancer.
- Deploy across multiple Availability Zones.
- Health checks and automatic failover.
- Graceful handling of overload.

INTERVIEW ANGLE

Memorise the line: **"Durability is about not losing the data; availability is about being able to access it."** Object storage typically advertises ~11 nines of durability but a lower figure (e.g. 99.9–99.99 %) for availability. Durability does *not* protect against you deleting data, which is why you also need backups and versioning (next topic).

3.7 Backup vs Replication

★★★

IN ONE LINE

Replication keeps a live copy for availability; backups keep point-in-time copies for recovery.

Replication continuously copies data to another location so a failure does not stop you — it is about **availability**. **Backup** takes **point-in-time copies** you can go back to — it is about **recovery from mistakes and corruption**. You need both.

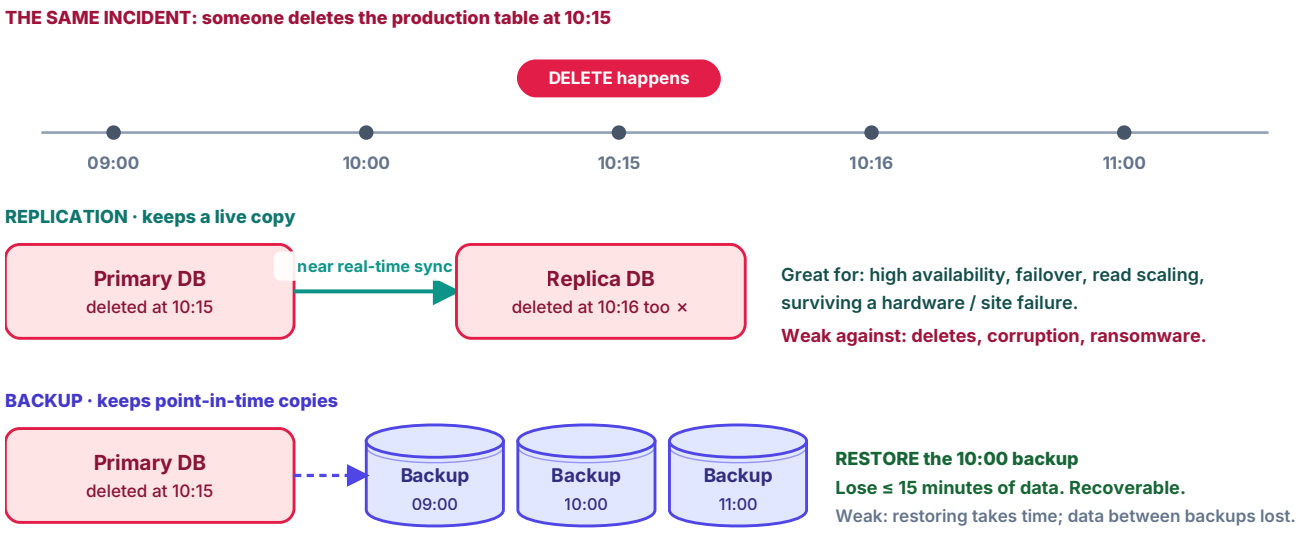


Figure 3.7 — A delete replicates instantly; a backup lets you go back in time

	Replication	Backup
Goal	High availability, fast failover, DR	Recover from deletion, corruption, ransomware, bad deploys
Freshness	Near real-time (sync / async)	As of the last backup (hourly, daily)
History	Only the latest state	Many point-in-time versions
Mistakes	**Replicates mistakes** instantly	Lets you roll back before the mistake
Recovery speed	Seconds to minutes (failover)	Minutes to hours (restore)
Cost	Higher (a second live system)	Lower (storage of snapshots)

The 3-2-1 backup rule

3 copies of your data, on **2** different types of storage, with **1** copy off-site (another region / account). Many add "immutable" backups so ransomware cannot delete them.

Replication flavours

- **Synchronous** — write confirmed on both; no data loss, higher latency.
- **Asynchronous** — replica lags slightly; faster, small risk of loss.
- Same-AZ, cross-AZ, **cross-region**.

INTERVIEW ANGLE

The classic interview question: *"Is replication a backup?"* **No.** If you drop a table, the replica drops it too. Always pair replication (for uptime) with backups or versioning (for recovery).

3.8 Managed Databases in the Cloud

★★☆

IN ONE LINE

Provider-run SQL and NoSQL databases: the provider operates them, you design and query.

Cloud providers sell **managed databases** (DBaaS): the provider handles installation, patching, backups, replication and failover; you handle schema and queries. It is a classic **PaaS** example and a favourite fresher topic because it links storage, availability and cost.

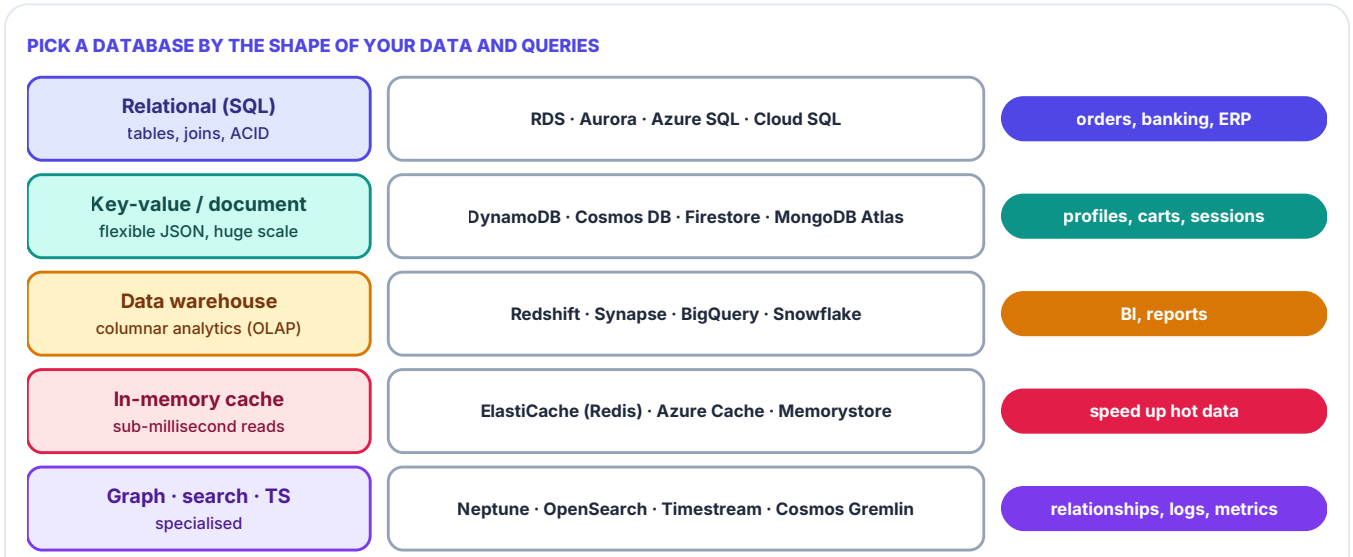


Figure 3.8 — Database families and cloud examples

	SQL (relational)	NoSQL
Schema	Fixed, defined up front	Flexible / schema-less
Scaling	Mostly vertical, then read replicas / sharding	Built for horizontal scale-out
Consistency	ACID transactions	Often eventual (tunable)
Query	SQL, joins	Key lookups, document queries
Use when	Relationships and correctness matter	Huge scale, simple access patterns, changing data shape

What "managed" gives you

- Automated backups and point-in-time restore
- **Multi-AZ** standby with automatic failover
- Read replicas for scaling reads
- Patching and monitoring
- Encryption at rest by default

Still your job

- Schema design and indexing
- Query performance
- Access control and network exposure
- Cost: instance size, storage, I/O

DATA ENGINEERING TIE-IN

OLTP databases (RDS/Aurora) serve the application; data is copied via **CDC or ETL** into an OLAP **warehouse or lakehouse** for analytics. Refer back to the Data Engineering handbook (OLTP vs OLAP, CDC, Lakebase).

04

Part 4 · Networking

How cloud resources are addressed, secured and connected: IPs, DNS, subnets and load balancing.

- Public vs Private IP
- DNS
- HTTP vs HTTPS · Ports
- Firewalls
- VPC & Subnets
- Connecting networks
- Load Balancer

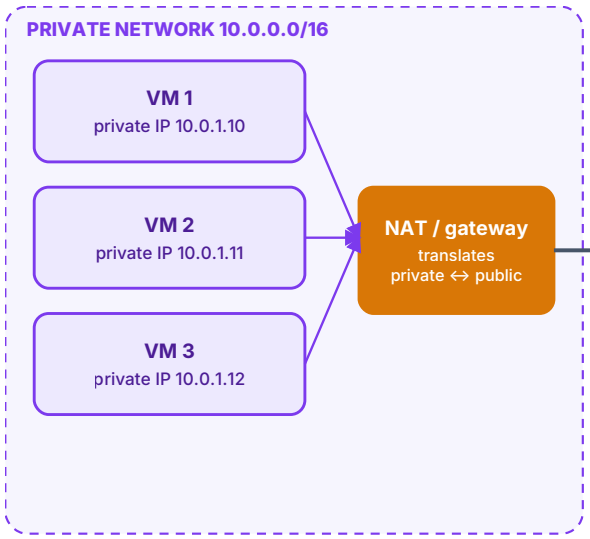
4.1 Public IP vs Private IP

★★★

IN ONE LINE

Public addresses are reachable on the internet; private addresses live only inside a network.

Every device on a network needs an **IP address**. **Public IPs** are unique across the whole internet and reachable from outside. **Private IPs** are used only inside a private network and are **not routable on the internet**; many networks reuse the same private ranges.



Private ranges	Block	Addresses
Class A	10.0.0.0/8	16.7 million
Class B	172.16.0.0/12	1 million
Class C	192.168.0.0/16	65,536

Figure 4.1 — Private addresses inside, a public address outside, NAT in between

	Public IP	Private IP
Reachable from internet?	Yes (if firewall allows)	No, only inside the network or via VPN / NAT
Uniqueness	Globally unique	Unique only within its network
Assigned by	ISP / cloud provider	You (network admin / DHCP)
Cost / risk	May cost money; exposed to attacks	Free; safer by default
Typical use	Web servers, load balancers, NAT gateways	App servers, databases, internal services
Cloud notes	**Elastic / static IP** stays the same when VM restarts	Fixed in the subnet CIDR

IPv4 vs IPv6

IPv4: 32-bit, e.g. `192.168.1.10`, about 4.3 billion addresses (exhausted). **IPv6**: 128-bit, e.g. `2001:db8::1`, effectively unlimited.

NAT in one line

Network Address Translation lets many private machines **share one public IP** to reach the internet, while outsiders cannot start connections to them.

INTERVIEW ANGLE

Best practice: only **load balancers and a few entry points** get public IPs. Application servers and databases stay on **private IPs** in private subnets (see 4.6).

4.2 DNS (Domain Name System)

★★★

IN ONE LINE

The internet's phone book: translates domain names into IP addresses.

DNS (Domain Name System) is the internet's phone book: it translates human-friendly **domain names** (`www.example.com`) into **IP addresses** (`93.184.216.34`) that computers use to connect.

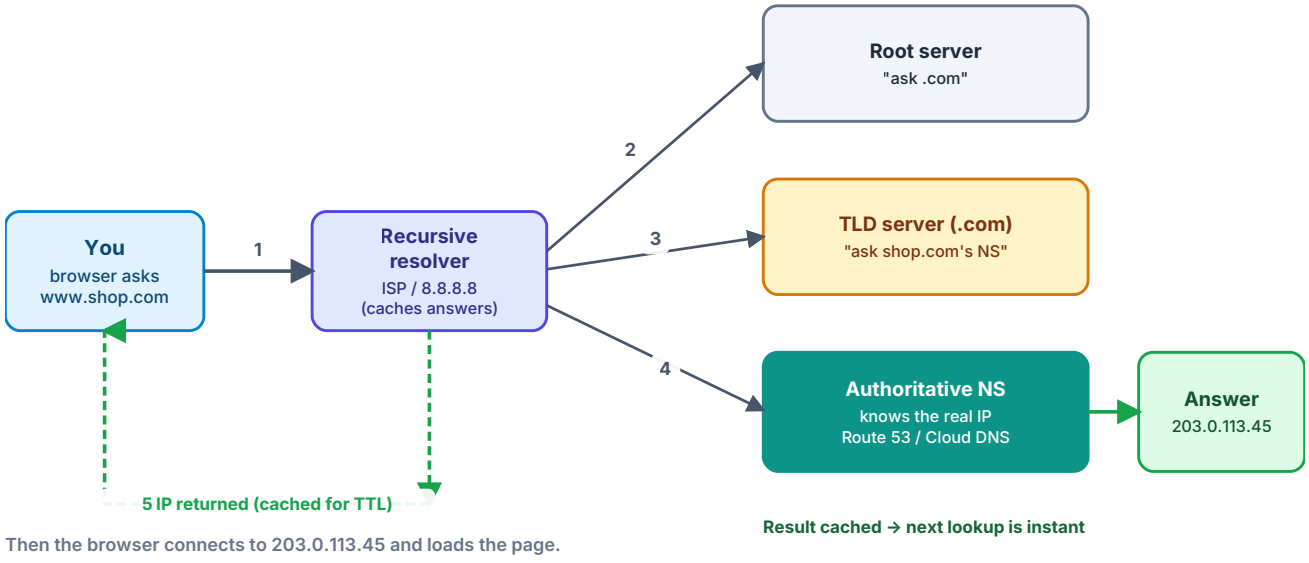


Figure 4.2 — Resolving a domain name, step by step

Record	Purpose	Example
A	Name → IPv4 address	`shop.com → 203.0.113.45`
AAAA	Name → IPv6 address	`shop.com → 2001:db8::1`
CNAME	Alias of another name	`www.shop.com → shop.com`
MX	Mail servers for the domain	`shop.com → mail.shop.com`
TXT	Free text: domain verification, SPF	`"v=spf1 ..."`
NS	Which servers are authoritative	`ns1.route53...`

TTL (time to live)

How long a resolver may cache an answer. Short TTL = changes spread fast; long TTL = fewer lookups. Lower TTL *before* a planned migration.

Cloud DNS services

Amazon Route 53, **Azure DNS**, **Cloud DNS**. They also offer **health-check-based routing**, latency-based and geo routing — used for failover and global load distribution.

INTERVIEW ANGLE

If a site "works by IP but not by name", the problem is DNS. Useful commands: `nslookup example.com` , `dig example.com` .

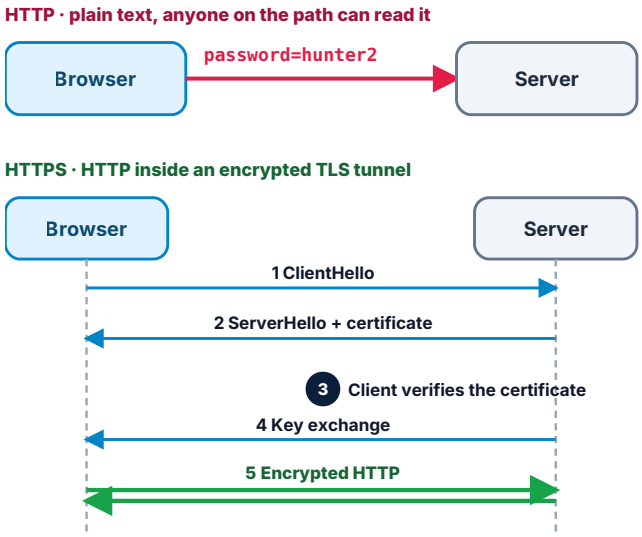
4.3 HTTP vs HTTPS

★★★

IN ONE LINE

HTTP carries web traffic in plain text; HTTPS wraps it in TLS encryption with verified identity.

HTTP (HyperText Transfer Protocol) is the language browsers and servers use to exchange web content. **HTTPS** is HTTP wrapped in **TLS encryption**, giving confidentiality, integrity and proof of the server's identity.



	HTTP	HTTPS
Port	80	443
Encrypted	No	Yes (TLS)
Tamper-proof	No	Yes
Server identity	Unverified	Certificate
Cost	free	free certs (ACM, Let's Enc

Figure 4.3a — HTTP is readable on the wire; HTTPS first sets up an encrypted tunnel

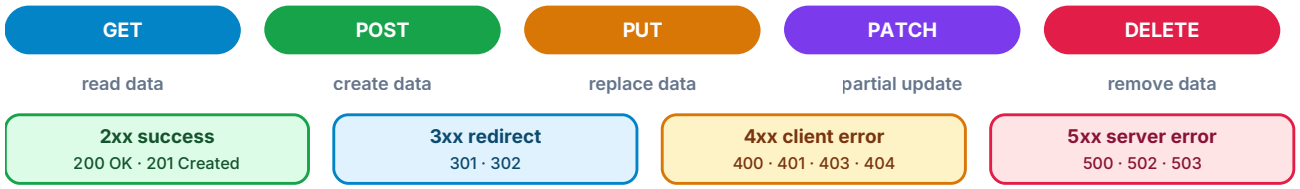


Figure 4.3b — Common HTTP methods and status code families

What a certificate proves

A trusted **Certificate Authority (CA)** signed a statement that this public key belongs to this domain. Browsers check the signature, domain and expiry before trusting the site.

In the cloud

Terminate HTTPS at the **load balancer / CDN** using managed certificates (**AWS ACM**, Azure Key Vault certs). Redirect HTTP to HTTPS. Use HTTPS for every API, even internal ones where possible.

COMMON TRAP

401 means "not authenticated" (who are you?), **403** means "authenticated but not allowed". **502/503/504** usually point at the server or a gateway in front of it. Interviewers use status codes to test debugging skills.

4.4 Ports and Protocols

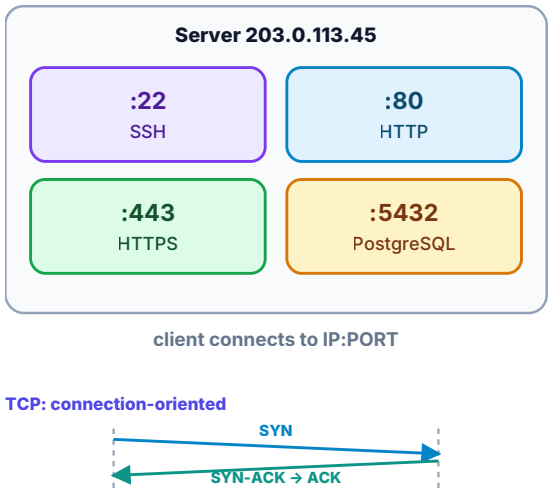
★★★

IN ONE LINE

Ports identify programs on a machine; protocols define how two systems communicate.

An IP address finds the **machine**; a **port** (0–65535) finds the **program** on it. A **protocol** is an agreed set of rules for how two sides talk (HTTP, DNS, SSH…). Firewalls (next topic) work by allowing or blocking **protocol + port + source**.

IP ADDRESS = the building · PORT = the apartment · PROTOCOL = the language spoken



Port	Service	Used for
22	SSH	Secure remote login to Linux
80	HTTP	Unencrypted web
443	HTTPS	Encrypted web, APIs
3389	RDP	Windows remote desktop
53	DNS	Name resolution (UDP/TCP)
3306 / 543	MySQL / Postgres	Relational databases
1433	SQL Server	Microsoft databases
6379	Redis	In-memory cache
9092	Kafka	Message broker
25 · 587	SMTP	Sending email

Figure 4.4 — IP + port + protocol, and the commonly asked port numbers

	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless, fire and forget
Reliability	Guaranteed, ordered delivery; retransmits	No guarantee, no ordering
Speed	Slower (overhead)	Faster, lightweight
Used by	HTTP/HTTPS, SSH, databases, email	DNS queries, video streaming, VoIP, gaming

Port ranges

- **0–1023** well-known (HTTP, SSH…)
- **1024–49151** registered applications
- **49152–65535** ephemeral (client side)

Other protocols to recognise

- **FTP/SFTP** file transfer (21 / 22)
- **SMTP/IMAP** email
- **ICMP** ping
- **DHCP** hands out IP addresses
- **gRPC, WebSocket, MQTT** modern app / IoT

INTERVIEW ANGLE

Memorise **22, 80, 443, 3389, 53** and the database ports (**3306, 5432, 1433**). A very common scenario: “I cannot SSH into my VM” → check the firewall / security group allows **TCP 22** from your IP.

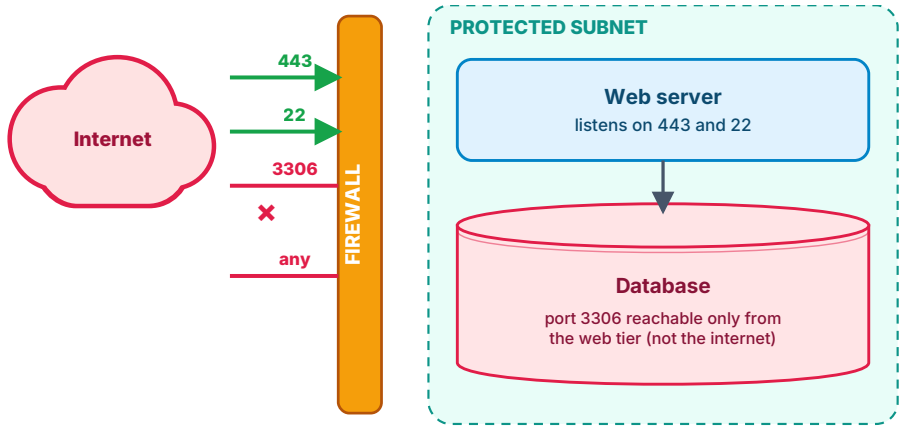
4.5 Firewall

★★★

IN ONE LINE

Rules that allow or block network traffic by source, destination, protocol and port.

A **firewall** filters network traffic based on rules — allowing or blocking by **source, destination, protocol and port**. It is the network's security guard. In the cloud, firewalls are virtual and attached to resources or subnets.



Rule	Value
Direction	Inbound
Protocol	TCP
Port	443
Source	0.0.0.0/0
Action	ALLOW

Default deny: anything not explicitly allowed is blocked.

Rules match on direction · protocol · port · source / destination IP range.

Figure 4.5 — A firewall allows only what the rules permit

	Security group (SG) / NSG	Network ACL (NACL)
Applies to	A VM / network interface (instance level)	A whole subnet
State	**Stateful**: reply traffic is automatically allowed	**Stateless**: must allow both directions explicitly
Rules	Allow rules only (everything else denied)	Allow *and* deny rules, evaluated in order
Typical use	Primary day-to-day control	Extra subnet-level guardrail, blocking an IP range

Kinds of firewall

- **Network firewall** — layers 3–4 (IP, port).
- **WAF** (web application firewall) — layer 7, blocks SQL injection, XSS, bots.
- **Next-gen firewall** — deep packet inspection, intrusion prevention.

Good habits

- Default deny; open only what is needed.
- Never open **22 / 3389 / DB ports to 0.0.0.0/0**.
- Reference other security groups instead of IP lists.
- Separate tiers: web → app → database.

COMMON TRAP

`0.0.0.0/0` means "the entire internet". Seeing it on an SSH, RDP or database rule is a red flag in any security review.

4.6 Virtual Network and Subnet

★★★

IN ONE LINE

Your private, isolated cloud network, divided into subnets with their own routing and exposure.

A **virtual network** (AWS **VPC**, Azure **VNet**, GCP **VPC**) is your own logically isolated network in the cloud, where you choose the IP range, subnets, routes and firewalls. A **subnet** is a smaller slice of that range, tied to one Availability Zone, used to group resources with similar exposure.

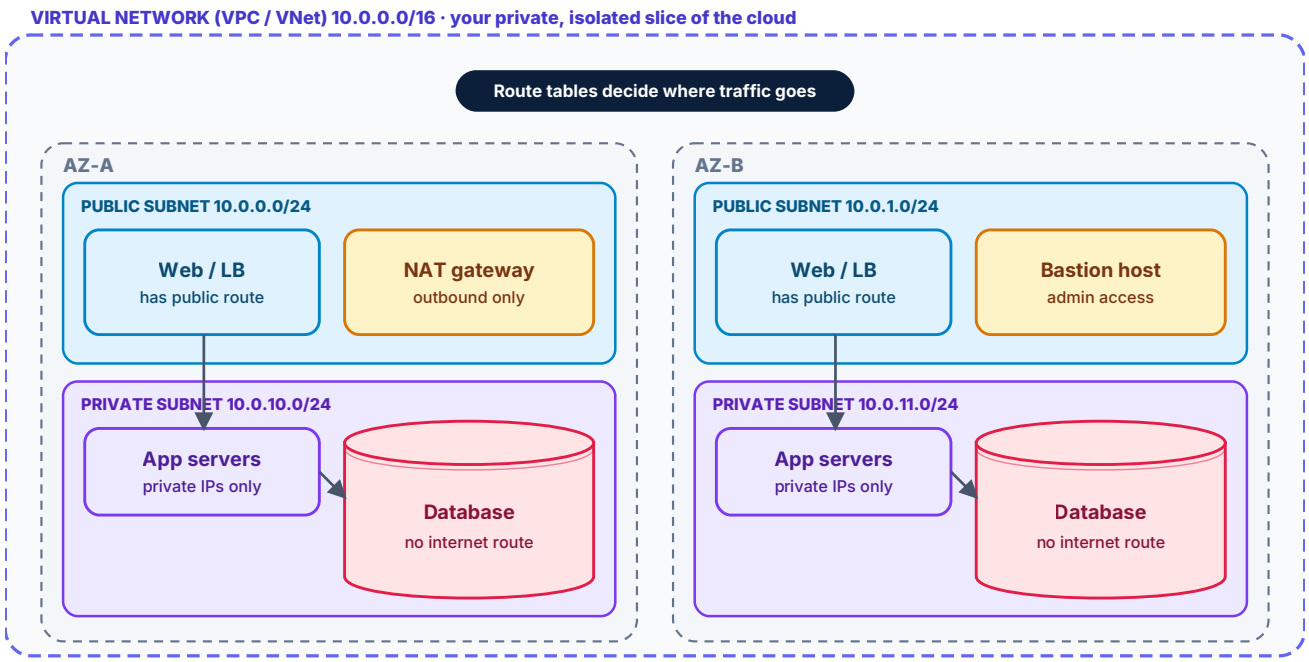


Figure 4.6a — A typical VPC: public and private subnets across two AZs

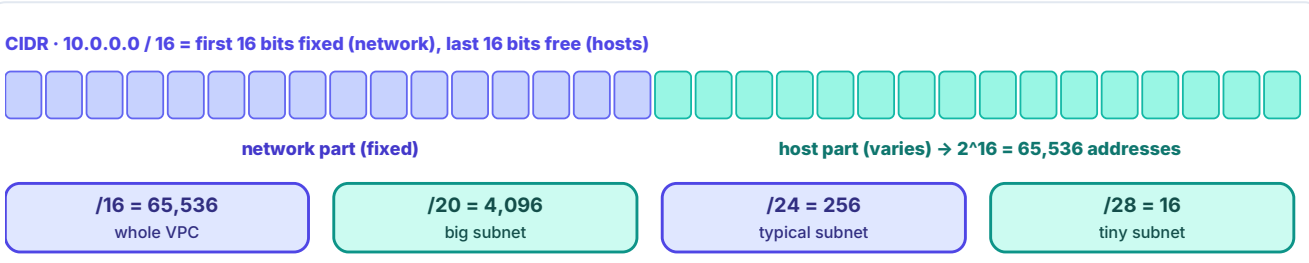


Figure 4.6b — CIDR notation sizes the network

Component	Role
Public subnet	Has a route to the Internet Gateway; holds load balancers, bastion hosts, NAT gateways.
Private subnet	No direct internet route; holds app servers and **databases**.
Internet Gateway	The door between the VPC and the public internet.
NAT Gateway	Lets private instances start *outbound* connections (updates, APIs) without being reachable inbound.
Route table	Rules deciding where each destination range is sent.
Security group / NACL	Firewalls for instances and subnets (4.5).

INTERVIEW ANGLE

The textbook secure design: **public subnet = load balancer only; private subnets = app and database**. Plan CIDR blocks that **do not overlap** with other VPCs or your on-prem network, otherwise you cannot connect them later.

4.7 Connecting Networks: VPN, Peering & Private Links

★★☆

IN ONE LINE

Linking data centres, VPCs and cloud services securely.

Real systems rarely live in one network. You connect a data centre to the cloud, one VPC to another, and private resources to cloud services — without exposing them to the open internet.

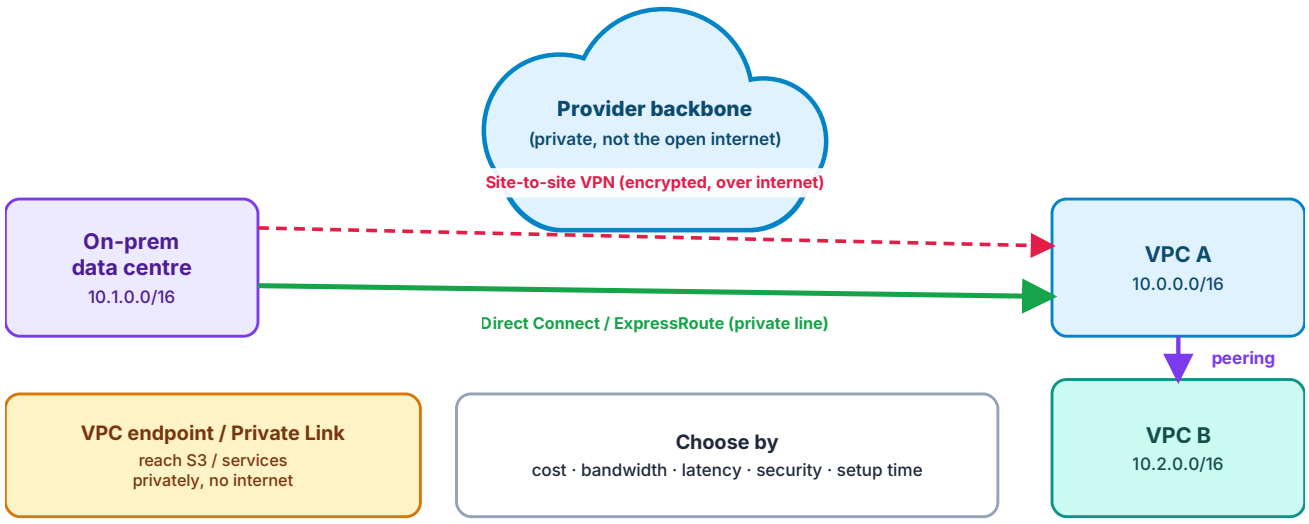


Figure 4.7 — Ways to connect networks

Option	What it is	Pros	Cons
Site-to-site VPN	Encrypted tunnel over the public internet	Quick, cheap	Variable latency, limited bandwidth
Direct Connect / ExpressRoute / Interconnect	Dedicated private line to the provider	Stable, fast, predictable	Costly, weeks to install
VPC peering	Private link between two VPCs	Simple, low latency	Not transitive; no overlapping CIDR
Transit gateway / hub	Central router for many networks	Scales to many VPCs	Extra cost
Private endpoint (PrivateLink)	Reach a service via a private IP	Traffic never leaves the backbone	One endpoint per service

Bastion host / jump box

A hardened VM in the public subnet used as the only entry point for admin access to private machines. Modern alternatives: **SSM Session Manager**, Azure Bastion.

VPN vs proxy (quick)

A **VPN** encrypts and tunnels all traffic between networks or a user and a network. A **proxy** relays specific application requests on your behalf.

DATA ENGINEERING TIE-IN

Connecting an on-prem database to a cloud data platform for ingestion normally uses a VPN or Direct Connect plus a private endpoint to the object store, so data never crosses the public internet.

4.8 Load Balancer

★★★

IN ONE LINE

Spreads traffic across healthy servers behind a single entry point.

A **load balancer** distributes incoming requests across a group of servers so no single one is overloaded, and sends traffic only to servers that pass **health checks**. It gives clients **one stable address** while servers come and go behind it.

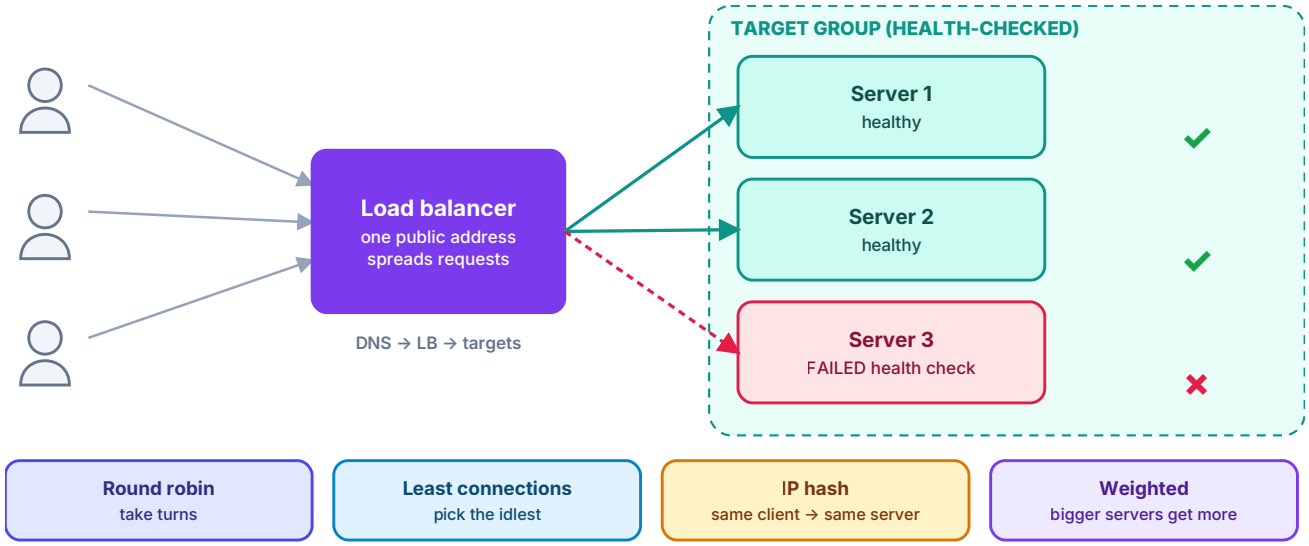


Figure 4.8 — One front door, several health-checked servers

	Layer 4 (network / TCP)	Layer 7 (application / HTTP)
Sees	IPs and ports only	URLs, headers, cookies, methods
Speed	Very fast, ultra-low latency	Slightly slower, smarter
Features	Pass-through TCP/UDP, static IPs	Path routing (`/api` vs `/img`), host routing, TLS termination, WAF
Examples	AWS NLB, Azure Load Balancer	AWS ALB, Azure Application Gateway, GCP HTTP(S) LB

What it gives you

- **Scalability** — add servers behind it.
- **High availability** — dead servers removed automatically.
- **Security** — servers stay private; TLS ends here.
- **Zero-downtime deploys** — drain, update, return.

Related ideas

- **Sticky sessions** keep a user on one server (avoid if you can).
- **Global load balancing** across regions (DNS / anycast).
- The LB itself is redundant — managed, multi-AZ.

INTERVIEW ANGLE

Load balancer + auto scaling group across two AZs is the **standard answer** to “how do you make a web app scalable and highly available?”. Make the servers **stateless**: keep sessions in a cache or database, not in server memory.

05

Part 5 · Reliability & Disaster Recovery

Designing systems that stay up, and recover quickly when something fails.

- Region vs Availability Zone
- High Availability
- Fault Tolerance
- SLA / SLO / SLI
- Disaster Recovery
- RTO vs RPO

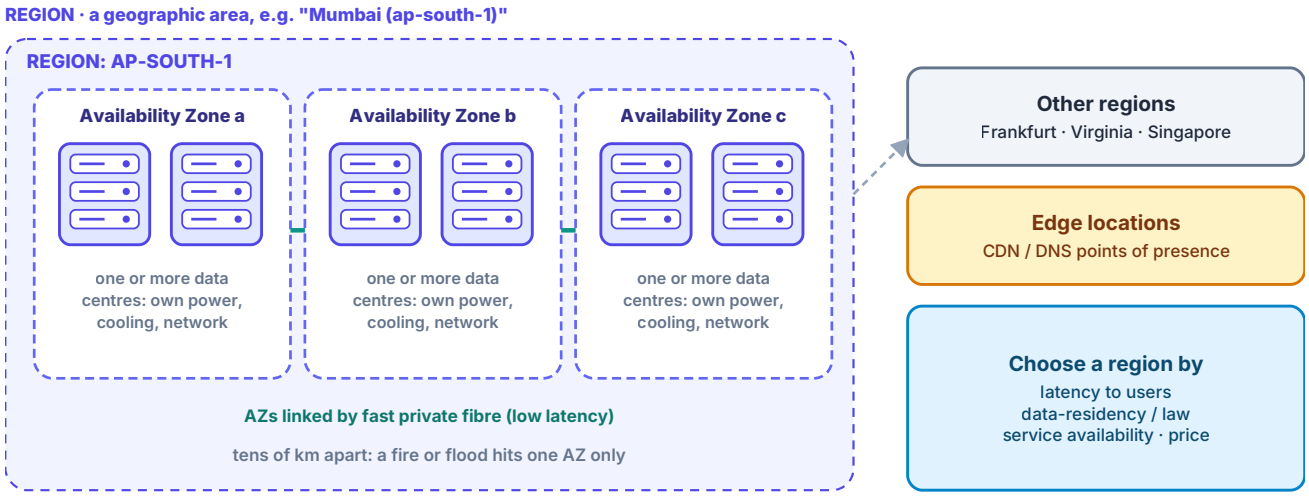
5.1 Region vs Availability Zone

★★★

IN ONE LINE

Regions are geographic areas; Availability Zones are isolated data-centre groups inside a region.

A **Region** is a geographic location containing multiple, isolated data-centre groups called **Availability Zones (AZs)**. AZs are separate buildings with independent power, cooling and networking, close enough for fast links but far enough that one disaster does not take down all.



Global infrastructure = many Regions, each with ≥ 3 AZs (typically), plus a worldwide edge network.

Azure & GCP use the same ideas (Azure: regions + availability zones; GCP: regions + zones).

Figure 5.1 — Regions contain Availability Zones; edge locations sit around them

	Region	Availability Zone
What	A geographic area (e.g. Mumbai, Frankfurt)	An isolated location inside a region
Failure it survives	Natural disaster, regional outage (if multi-region)	Data-centre failure (power, fire, network)
Latency between	Higher (tens to hundreds of ms)	Very low (single-digit ms)
Used for	Disaster recovery, data residency, latency to users	**High availability** inside a region
Data transfer cost	Charged between regions	Small charge between AZs, free inside one

INTERVIEW ANGLE

Multi-AZ = high availability. Multi-region = disaster recovery (and global latency). Most apps need multi-AZ; only the most critical need multi-region because it is costly and complex. Also mention that many services are **regional** (S3, Lambda) while others are **zonal** (EBS volumes, single VMs).

5.2 High Availability

★★★

IN ONE LINE

Keeping a system accessible with minimal downtime by removing single points of failure.

High availability (HA) means designing a system so it stays **accessible with minimal downtime** despite failures. You achieve it by removing **single points of failure** (SPOFs): run redundant copies, in separate failure domains, with automatic detection and failover.

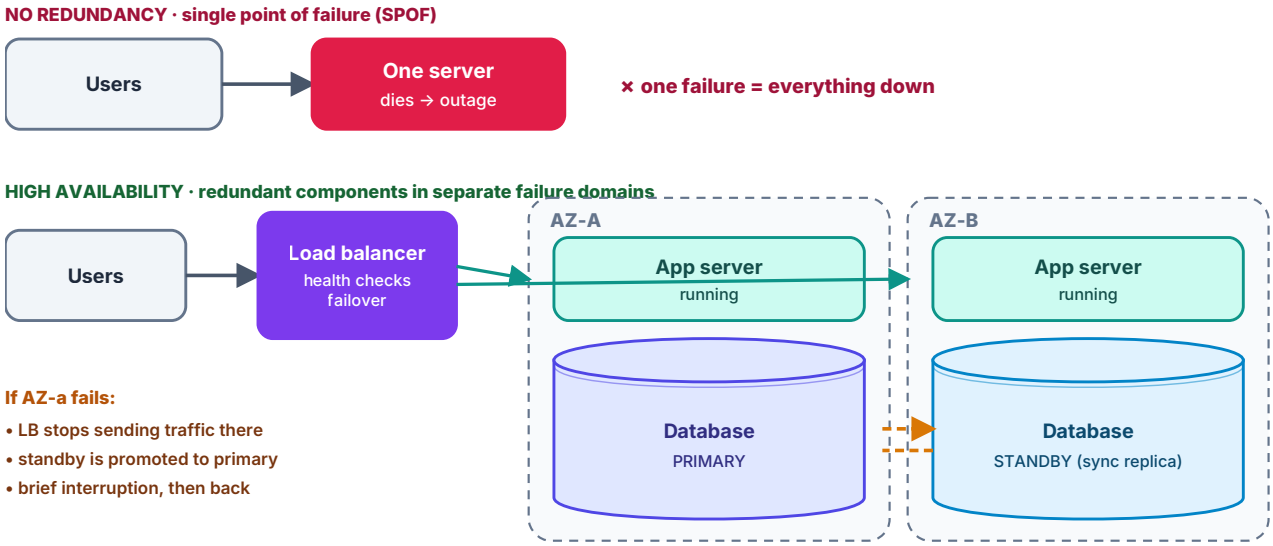


Figure 5.2 — From one server to a multi-AZ, redundant design

HA building blocks

- **Redundancy** at every tier (servers, DBs, load balancers).
- **Multiple AZs**.
- **Health checks** and automatic **failover**.
- **Auto scaling** replaces failed instances.
- **Stateless** app tier; replicated data tier.

Measuring it

Availability % over time: **99.9 %** ≈ 8.8 h downtime/year, **99.99 %** ≈ 53 min. See the table in 3.6 and the SLA topic (5.5).

Active-passive: standby waits. **Active-active**: all nodes serve traffic.

REMEMBER

HA aims for **minimal** downtime (seconds to minutes during failover). It does not promise *zero* interruption — that is fault tolerance (next topic).

INTERVIEW ANGLE

Hunt the SPOF: “What if this one component dies?” applied to the load balancer, the database, the DNS, the single AZ, the one engineer who knows the system. This question structure impresses interviewers.

5.3 Fault Tolerance

★★★

IN ONE LINE

Operating continuously, with no interruption, even when components fail.

Fault tolerance is the ability of a system to **keep operating without any interruption** when a component fails. Instead of "recover quickly" (HA), a fault-tolerant system is built so the failure is **invisible** to users — usually via duplicated components running simultaneously.

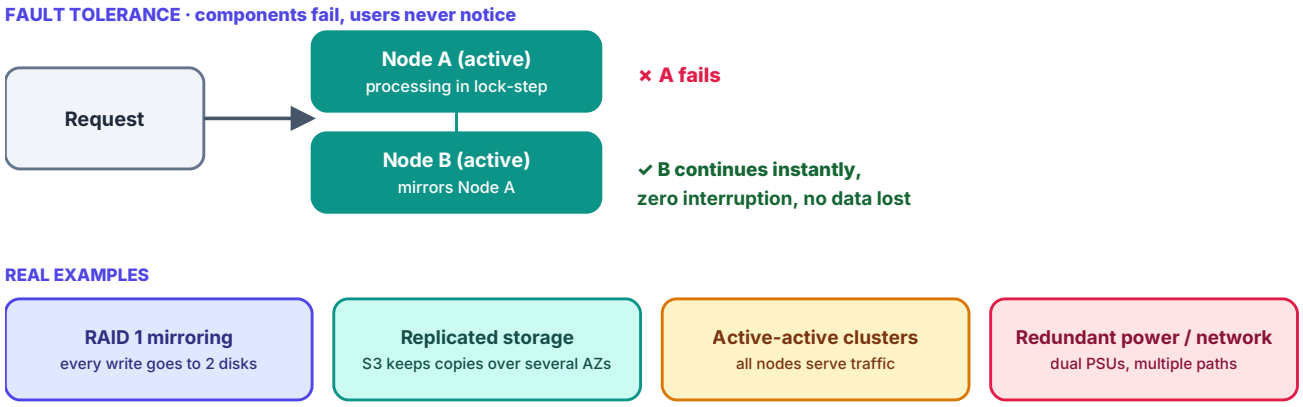


Figure 5.3 — Duplicate, run in parallel, mask the failure

Techniques

- **Redundancy** of hardware and data.
- **Replication** with synchronous writes.
- **Active-active** designs and quorum (majority) systems.
- **Graceful degradation** and retries with back-off.
- **Chaos engineering** (e.g. Netflix Chaos Monkey) to test it.

Trade-offs

- Higher cost (idle-equivalent capacity).
- Added complexity and synchronisation overhead.
- Needed mainly for life-critical or financial systems.

REAL-WORLD ANALOGY

An aircraft with two engines can keep flying if one fails (fault tolerant). A car with a spare tyre can continue after you stop and swap it (highly available: brief interruption, then back).

5.4 High Availability vs Fault Tolerance

★★★

IN ONE LINE

HA minimises downtime; fault tolerance eliminates it.

Both aim to survive failures, but with different promises. **HA = minimise downtime. FT = no downtime at all.**

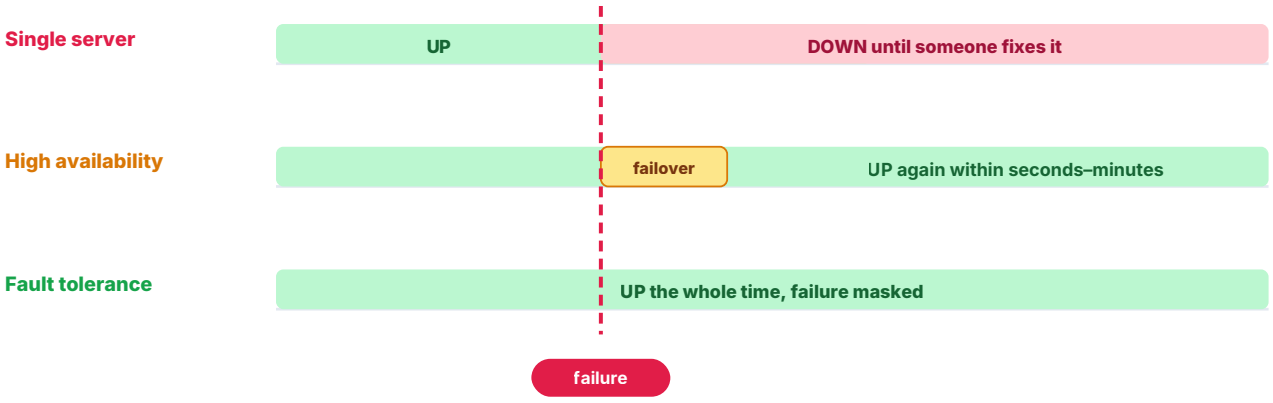


Figure 5.4 — Same failure, three experiences

	High availability	Fault tolerance
Goal	Minimal downtime	Zero downtime
On failure	Detect, fail over; brief interruption possible	Continues seamlessly, nothing noticed
Redundancy	Standby / extra capacity ready	Components run simultaneously, in lock-step
Data loss on failure	Possible with async replication	None
Cost / complexity	Moderate	High
Typical target	99.9–99.99 % uptime	Effectively continuous operation
Examples	Multi-AZ database with automatic failover; ASG behind a load balancer	Mirrored storage, active-active clusters, redundant power

INTERVIEW ANGLE

A great one-liner: **“HA recovers fast; FT never goes down.”** And the practical point: most business applications choose **HA** because it is far cheaper; FT is reserved for systems where even seconds of downtime are unacceptable.

5.5 SLA, SLO & SLI

★★☆

IN ONE LINE

Measuring and promising reliability: indicators, objectives and contractual agreements.

How reliability is promised and measured. **SLI** is the metric, **SLO** is the goal for that metric, **SLA** is the legal contract with consequences if missed. Cloud providers publish SLAs per service (e.g. 99.99 % for a multi-AZ database).



A chain is weaker than its weakest link; two redundant copies in parallel IMPROVE availability:

```
1 − (1 − 0.99)² = 99.99 %
```

Figure 5.5 — SLI → SLO → SLA, and how availabilities combine

Term	Meaning	Example
SLI	A measured indicator of service health	request success rate, latency p95, uptime
SLO	Target value for an SLI (internal)	99.9 % of requests under 300 ms
SLA	External contract, with penalties / credits	“99.95 % monthly uptime or 10 % credit”
Error budget	Allowed unreliability = 100 % − SLO	0.1 % ≈ 43 min per month to spend on changes

COMMON TRAP

A provider’s SLA **only covers their part**, and the remedy is usually service credits — not compensation for your lost business. Your **own** architecture (multi-AZ, retries, backups) decides your real availability.

5.6 Disaster Recovery

★★★

IN ONE LINE

Restoring systems and data after a major outage or loss, using backups and cross-region copies.

Disaster Recovery (DR) is the set of policies and technology for **restoring systems and data after a major failure** — a region outage, ransomware, data corruption, human error. It is part of **business continuity**. Backups (3.7) and multi-region replication are its main tools.



Figure 5.6 — The four classic DR strategies and the cost/speed trade-off

Strategy	How it works	Recovery time	Cost
Backup & restore	Periodic backups copied off-site; rebuild on demand	Hours	$
Pilot light	Minimal always-on core (data replicated); build the rest at failover	Tens of minutes	$$
Warm standby	Scaled-down but functional copy always running	Minutes	$$$
Active-active (multi-site)	Full capacity in 2+ regions at once	Near zero	$$$$

HA vs DR

HA handles *component and AZ* failures automatically inside a region. **DR** handles *region-level or catastrophic* events, often with a manual or semi-automatic decision.

Cloud advantages

Pay for DR capacity only when used, replicate easily across regions, automate with IaC (7.5), and run failover drills cheaply.

INTERVIEW ANGLE

Pick the strategy from the business: **how long can we be down (RTO) and how much data can we lose (RPO)?** Then price the options. Always mention **testing** the plan.

5.7 RTO vs RPO

★★★

IN ONE LINE

RPO: how much data you can lose. RTO: how long you can be down.

The two numbers that define a DR plan. **RPO (Recovery Point Objective)** — the maximum amount of **data** you can afford to lose, measured as time back from the disaster. **RTO (Recovery Time Objective)** — the maximum **downtime** you can tolerate before service must be back.

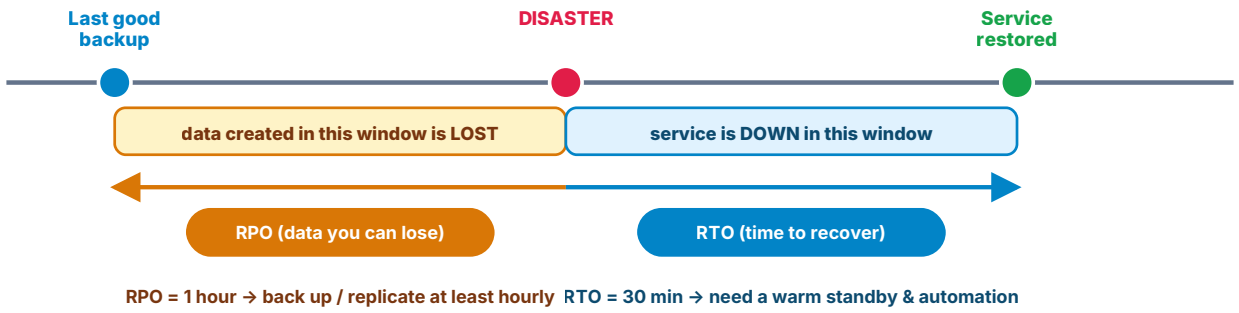


Figure 5.7 — RPO looks back in time; RTO looks forward

	RPO	RTO
Measures	Acceptable **data loss**	Acceptable **downtime**
Direction	Backwards from the disaster	Forwards from the disaster
Question	"How old can the restored data be?"	"How fast must we be running again?"
Drives	Backup / replication frequency	DR strategy and automation level
Smaller means	Replication more frequent (synchronous ~ 0)	Hot / warm standby, automated failover
Example	RPO 15 min → back up every 15 min	RTO 1 hour → restore must finish in 1 hour

REMEMBER

Lower RPO / RTO = higher cost. RPO ≈ 0 needs synchronous replication; RTO ≈ 0 needs active-active. Different systems get different targets: payments need tiny values; an internal wiki does not.

INTERVIEW ANGLE

Trick-question check: "Our RPO is 24 hours and RTO is 5 minutes — what does that mean?" → losing up to a day of data is acceptable, but service must return within minutes (e.g. automated failover to a standby restored from nightly backups).

06

Part 6 · Security

Identity, permissions and encryption: protecting data and access in the cloud.

- IAM
- Authentication vs Authorization
- Least Privilege
- Encryption at rest vs in transit
- Secrets, keys & compliance

6.1 IAM (Identity and Access Management)

★★★

IN ONE LINE

The system that defines who (or what) can perform which actions on which cloud resources.

IAM (Identity and Access Management) controls **who can do what on which resources** in the cloud. Every API call is checked against IAM, so it is the foundation of cloud security.

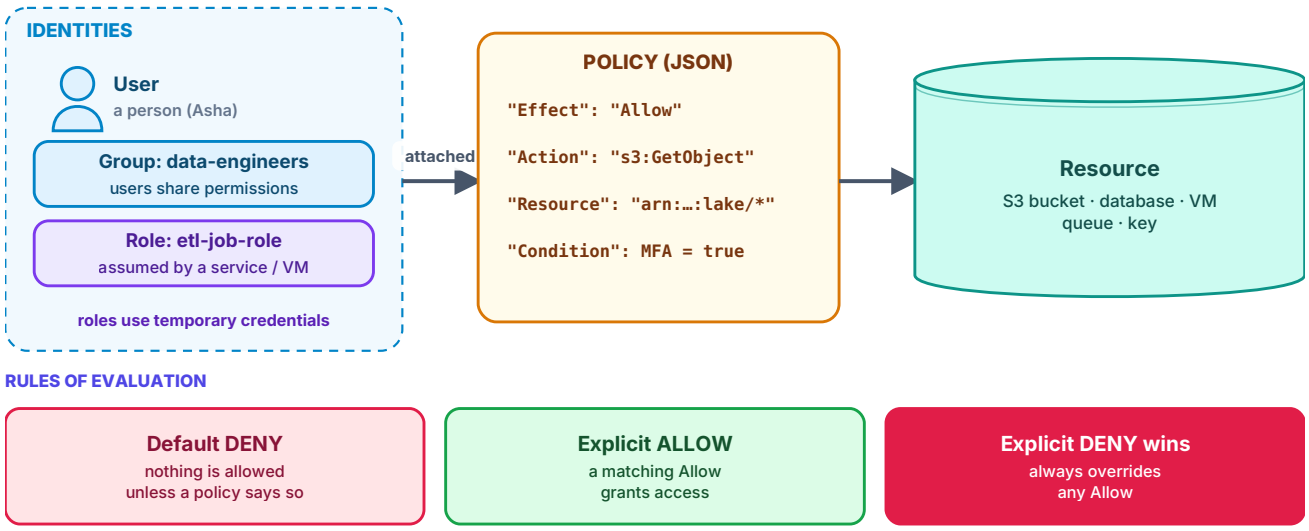


Figure 6.1 — Identities, policies and resources

Concept	Meaning
User	A human (or legacy app) with long-term credentials: password + MFA, access keys.
Group	A set of users that share the same policies (e.g. *data-engineers*).
Role	An identity with **temporary** credentials that a user, VM, function or another account *assumes*. Preferred for services.
Policy	A document that lists allowed or denied actions on resources, optionally with conditions.
Root / owner account	Full power. Protect with MFA and almost never use it.

AWS IAM POLICY · READ-ONLY ACCESS TO ONE PREFIX

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::my-lake", "arn:aws:s3:::my-lake/silver/*"]
  }]
}
```

INTERVIEW ANGLE

Services should never use access keys hard-coded in code. Give the VM / function / pipeline an **IAM role** and let the platform supply short-lived credentials. Mapping: AWS IAM → Azure Entra ID + RBAC → Google Cloud IAM.

6.2 Authentication vs Authorization

★★★

IN ONE LINE

Authentication proves identity; authorization decides what that identity may do.

Authentication (AuthN) proves **who you are**. **Authorization (AuthZ)** decides **what you are allowed to do**. You must authenticate first; then each request is authorized.

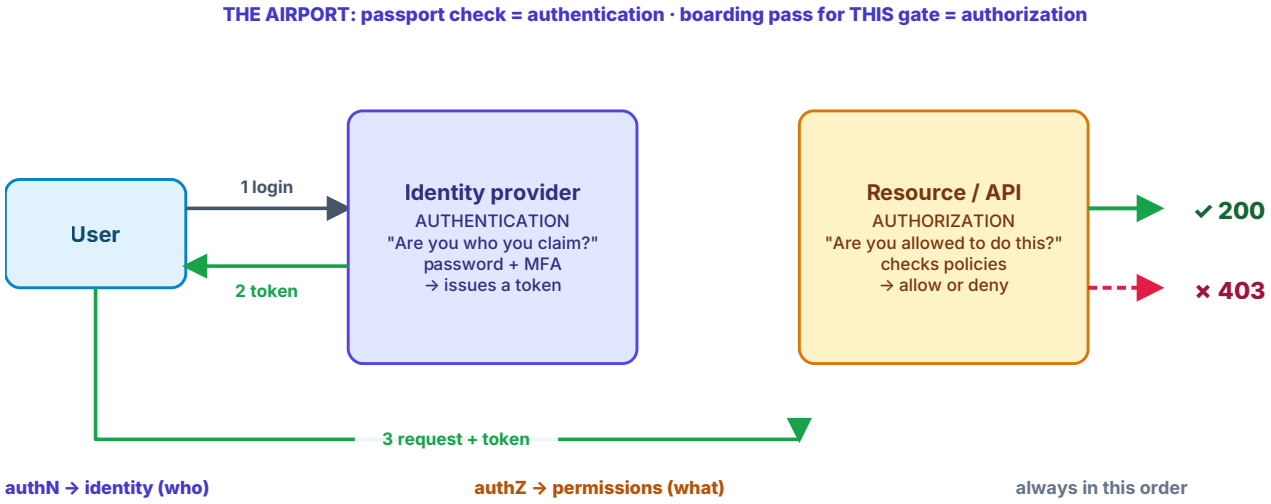


Figure 6.2 — Login, token, permission check

	Authentication	Authorization
Question	Who are you?	What may you do?
Methods	Password, MFA, biometrics, certificates, tokens, SSO	Policies, roles, ACLs, RBAC, scopes
Failure code	**401 Unauthorized** (not authenticated)	**403 Forbidden** (authenticated, not allowed)
Cloud example	Sign in to the console with MFA	IAM policy lets the role read one bucket

Multi-factor authentication

Combine **something you know** (password), **something you have** (phone / hardware key) and **something you are** (fingerprint). Blocks most stolen-password attacks.

Words you will hear

- **SSO** — one login for many apps.
- **SAML / OIDC** — standards for federated login (OIDC adds identity on top of **OAuth 2.0**).
- **OAuth 2.0** — delegated *authorization* ("let this app read my calendar").
- **JWT** — a signed token carrying claims.

6.3 Principle of Least Privilege

★★★

IN ONE LINE

Grant only the minimum permissions needed, for the minimum time.

The **Principle of Least Privilege (PoLP)**: give every user, role and service **only the minimum permissions needed, for only as long as needed**. It limits the damage (blast radius) from mistakes, bugs and compromised credentials.

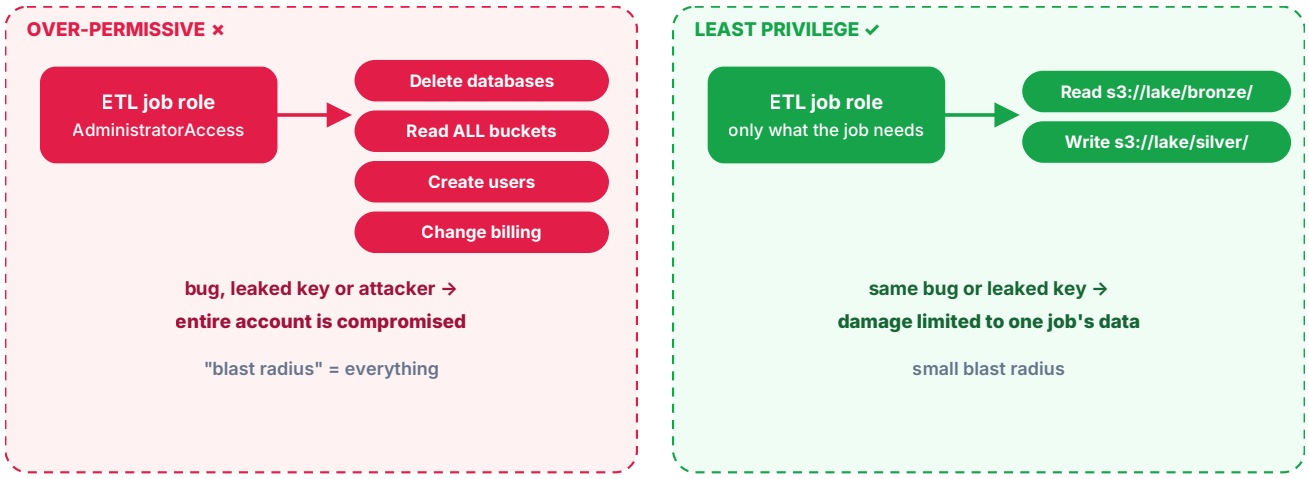


Figure 6.3 — Same failure, very different impact

How to apply it

- Start with **no access** and add what is required.
- Prefer **roles + groups** over individual grants.
- Scope by **resource** (one bucket / prefix), not `*`.
- Use **temporary credentials** and just-in-time elevation.
- Add **conditions** (MFA, source IP, time).
- Separate **dev / test / prod** accounts.

Keeping it that way

- Regularly review and remove unused permissions (access analyzers).
- Rotate or delete old keys.
- Log and alert on privileged actions.
- Avoid wildcards like `Action: "*"` and `Resource: "*"`.

INTERVIEW ANGLE

Related ideas: **separation of duties** (no one person can do everything), **defence in depth** (many independent layers: network + IAM + encryption + monitoring) and **zero trust** (“never trust, always verify”, even inside the network).

6.4 Encryption at Rest vs Encryption in Transit

★★★

IN ONE LINE

Protect stored data with keys on disk; protect moving data with TLS.

Encryption turns readable data into unreadable ciphertext that only holders of the right **key** can decode. Protect data in both of its states: **at rest** (stored) and **in transit** (moving). A third state, **in use**, is covered by confidential computing.

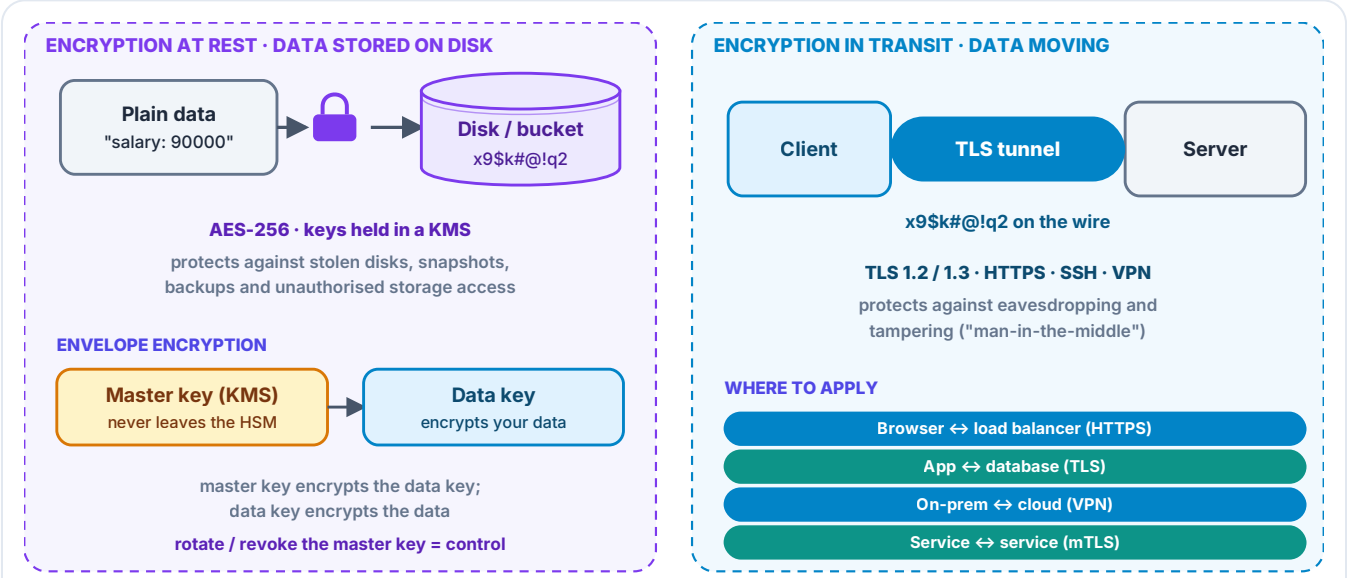


Figure 6.4 — Two states of data, two protections

	At rest	In transit
Protects	Stored data: disks, buckets, databases, backups	Data moving across a network
Threat	Stolen disk / snapshot, insider, storage breach	Eavesdropping, tampering, man-in-the-middle
Typical tech	AES-256, server-side encryption, KMS-managed keys, TDE for databases	TLS 1.2/1.3 (HTTPS), SSH, IPsec VPN
Cloud default	Often on by default for object storage and disks	You must enforce HTTPS / TLS
Key question	Who controls the key?	Is the certificate valid and trusted?

Key management choices

- **Provider-managed** keys — easiest.
- **Customer-managed** keys (CMK) in KMS — you control rotation and access.
- **Bring/hold your own key** (BYOK/HYOK) — strict compliance.

Symmetric vs asymmetric

Symmetric (AES): one shared key, fast, for bulk data.
Asymmetric (RSA, ECC): public/private key pair, used for key exchange and signatures, e.g. in TLS handshakes.

REMEMBER

Encrypt **at rest** so a stolen disk is useless; encrypt **in transit** so network spies see nothing. Encryption does not replace access control: a user with permission to read the decrypted data still can.

6.5 Secrets, Key Management & Compliance

★★☆

IN ONE LINE

Protecting credentials and keys, defending in layers, and meeting legal and audit requirements.

Beyond IAM and encryption, a few more security concepts regularly come up in fresher cloud interviews.

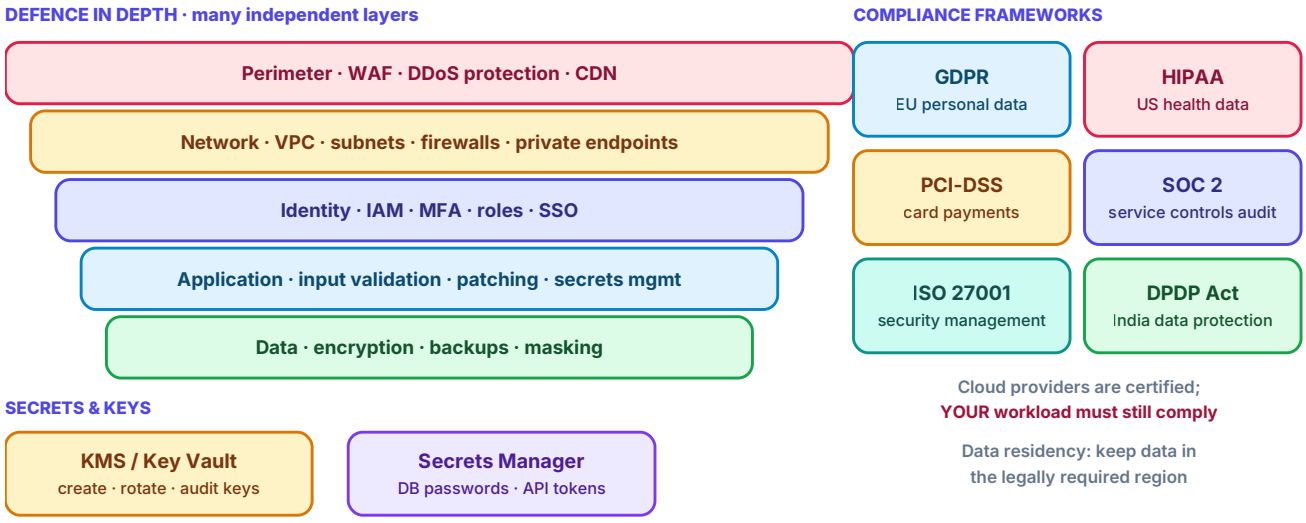


Figure 6.5 — Layers of defence, secrets management, and compliance

Concept	Why it matters
Secrets management	Passwords, API keys and tokens belong in a vault (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) — **never in code, Git or images**. Rotate them automatically.
KMS (key management service)	Creates and protects encryption keys in hardware security modules (HSMs); logs every use.
DDoS protection	Absorbs floods of malicious traffic (AWS Shield, Azure DDoS Protection, Cloud Armor).
Security monitoring	Audit logs (CloudTrail, Azure Activity Log) + threat detection (GuardDuty, Defender, Security Command Center).
Compliance & residency	Laws and standards dictate where data may live and how it must be protected; providers supply audit reports, you configure correctly.
Common attacks	Leaked keys, public buckets, weak passwords / no MFA, unpatched VMs, SQL injection, phishing.

COMMON TRAP

The most common real incident: **an access key accidentally committed to a public Git repository**. Bots find it in minutes. Use roles, secret managers and pre-commit secret scanners; if a key leaks, **revoke it immediately** and rotate.

DATA ENGINEERING TIE-IN

Mask or tokenise PII on its way from Bronze to Silver, restrict access with table/column permissions (e.g. Unity Catalog), and keep data in the allowed region. Credentials for pipelines come from a secret store, not config files.

07

Part 7 · Architecture & DevOps

Application styles and the automation that builds, ships and observes them.

- Monolith vs Microservices
- APIs
- Queues & events
- Monitoring vs Logging
- Infrastructure as Code
- CI/CD
- Edge Computing
- CDN

7.1 Microservices vs Monolithic Architecture

★★★

IN ONE LINE

One deployable application versus many small, independently deployable services.

A **monolith** packages the whole application — UI, business logic, data access — as **one deployable unit**. **Microservices** split it into **small, independent services**, each owning one business capability and its own data, communicating over APIs or events.

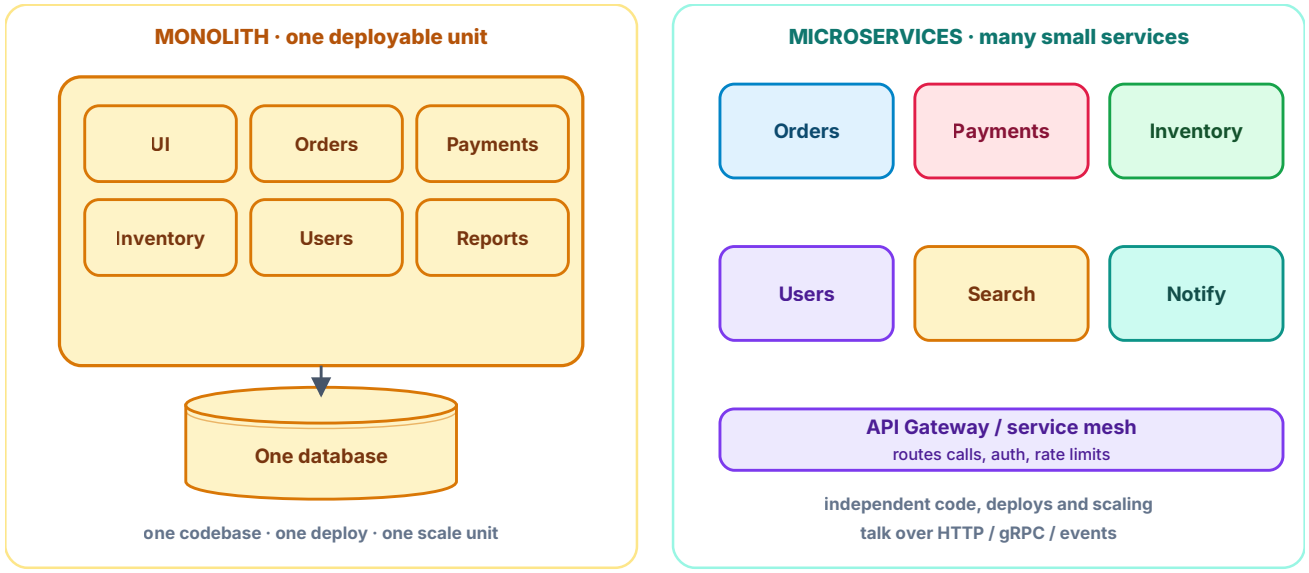


Figure 7.1 — One big unit vs many small services

	Monolith	Microservices
Deployment	All-or-nothing release	Each service deployed independently
Scaling	Scale the entire app	Scale only the busy service
Technology	One stack	Different language / DB per service
Fault isolation	A bug can bring down everything	A failure is contained (with good design)
Team structure	One large team, tight coupling	Small autonomous teams
Complexity	**Simple** to build and debug at first	Distributed-systems complexity: networking, tracing, data consistency
Best for	Small teams, early products, simple domains	Large systems, many teams, uneven scaling needs

INTERVIEW ANGLE

Do not claim microservices are always better. A grown-up answer: **"start with a well-structured monolith, split into services when team size and scaling needs justify the extra operational cost."** Cloud makes microservices easier via containers, Kubernetes, API gateways, serverless and managed queues.

REAL-WORLD ANALOGY

A single food-court stall that cooks everything (monolith) vs a food court with specialised counters: pizza, noodles, dessert (microservices). Busy pizza counter? Add staff only there.

7.2 API (Application Programming Interface)

★★★

IN ONE LINE

A defined interface through which software systems request services from each other, usually REST + JSON over HTTP.

An **API (Application Programming Interface)** is a contract that lets one piece of software ask another to do something without knowing how it works inside. In the cloud, **everything is an API**: creating a VM, uploading to a bucket, calling a model. Most web APIs follow **REST** over HTTP and exchange **JSON**.

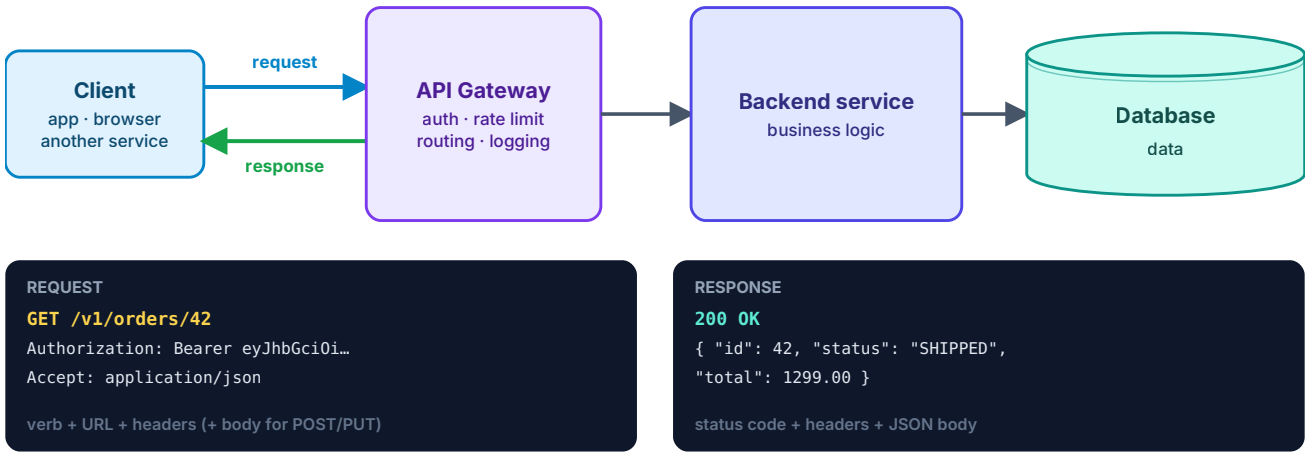


Figure 7.2 — Client, API gateway, backend, and a REST request / response

REST verb	Action	Idempotent?	Example
GET	Read	Yes	`GET /orders/42`
POST	Create	No	`POST /orders`
PUT	Replace	Yes	`PUT /orders/42`
PATCH	Partial update	Usually	`PATCH /orders/42`
DELETE	Remove	Yes	`DELETE /orders/42`

API gateway features

- Authentication and authorisation
- **Rate limiting** / throttling
- Request routing, caching, versioning
- Logging and metrics

REST vs GraphQL vs gRPC

- **REST**: resources + HTTP verbs, universal.
- **GraphQL**: client chooses exact fields, one endpoint.
- **gRPC**: binary, fast, service-to-service.

PYTHON · CALLING A REST API

```python
import requests

r = requests.get('https://api.example.com/v1/orders/42',
                 headers={'Authorization': 'Bearer <token>'}, timeout=10)
r.raise_for_status()          # 4xx / 5xx -> exception
order = r.json()
print(order['status'])
```

DATA ENGINEERING TIE-IN

Data engineers constantly **ingest from APIs**: handle pagination, rate limits (back off and retry), authentication, incremental pulls and idempotent writes.

7.3 Event-Driven Architecture & Message Queues

★★☆

IN ONE LINE

Asynchronous messaging that decouples producers from consumers and absorbs spikes.

Instead of services calling each other directly and waiting (**synchronous**), they can exchange **messages or events** through a broker (**asynchronous**). This **decouples** the sender from the receiver, smooths traffic spikes and makes systems more resilient. It is the glue of microservices and serverless architectures.

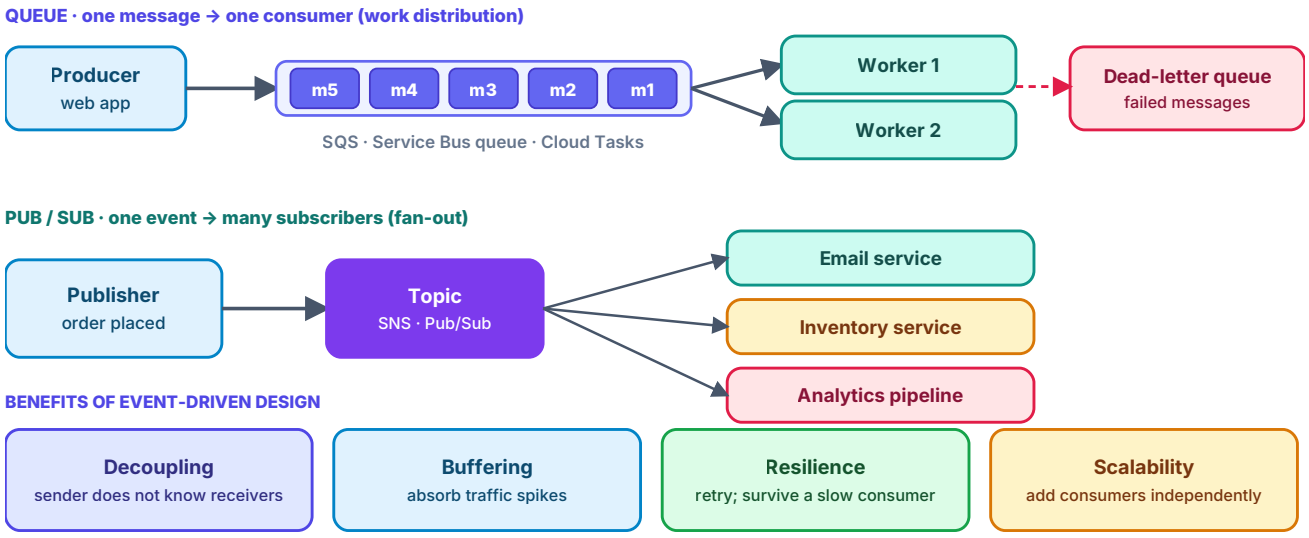


Figure 7.3 — Queue (work distribution) vs publish/subscribe (fan-out)

	Message queue	Pub / Sub topic	Event stream / log
Delivery	Each message to **one** consumer	Each message to **all** subscribers	Consumers read at their own offset; replayable
Retention	Until processed	Short, until delivered	Hours to forever
Use for	Task / job queues	Notifications, fan-out events	Analytics, CDC, event sourcing
Examples	SQS, Azure Service Bus, RabbitMQ	SNS, Event Grid, Google Pub/Sub	Kafka, Kinesis, Event Hubs

INTERVIEW ANGLE

Know these delivery words: **at-least-once** (duplicates possible, so consumers must be **idempotent**), **dead-letter queue** for poison messages, and **back-pressure**. This connects directly to exactly-once and idempotency in the Data Engineering handbook.

7.4 Monitoring vs Logging

★★★

IN ONE LINE

Monitoring watches metrics to detect problems; logging records events to explain them.

Monitoring continuously collects and watches **metrics** to tell you **whether the system is healthy and when it is not** (alerts and dashboards). **Logging** records **detailed events** so you can find out **what happened and why**. Together with **tracing** they form **observability**.

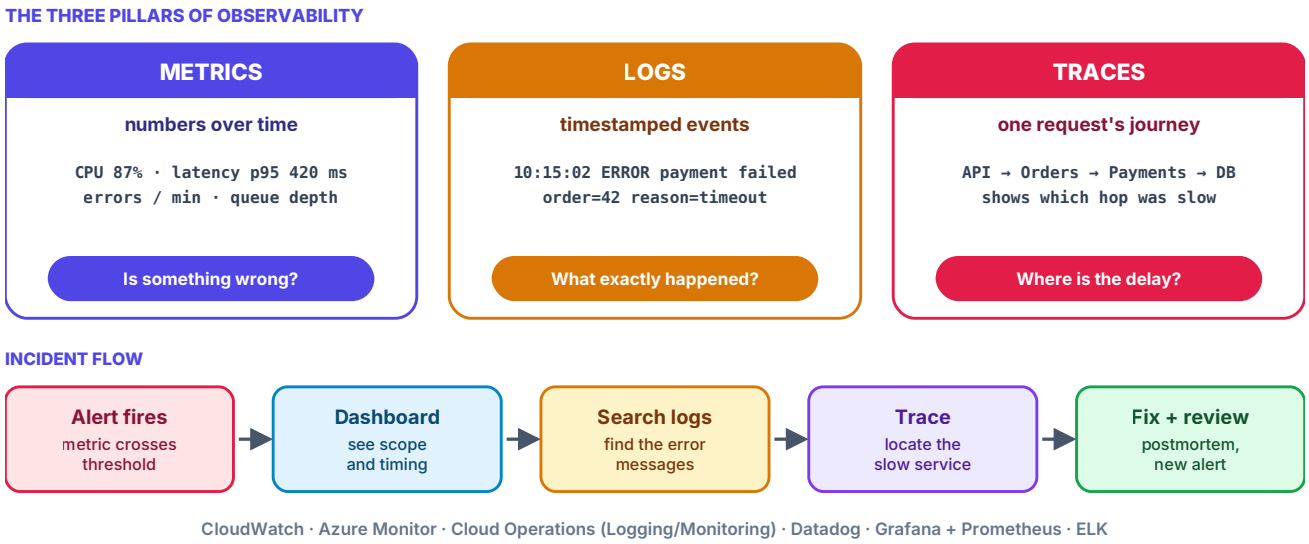


Figure 7.4 — Metrics, logs, traces and how an incident is investigated

	Monitoring	Logging
Data	Numeric metrics sampled over time	Text / structured event records
Answers	"Is it healthy right now? Is it getting worse?"	"What happened at 10:15 for order 42?"
Volume	Small, aggregated	Large, detailed (cost!)
Used for	Dashboards, alerts, auto scaling, SLOs	Debugging, auditing, security forensics
Cloud tools	CloudWatch metrics, Azure Monitor, Cloud Monitoring	CloudWatch Logs, Log Analytics, Cloud Logging, ELK

Good alerting

- Alert on **symptoms users feel** (errors, latency), not every CPU blip.
- Every alert must be **actionable** with an owner.
- Define severity and escalation paths.
- Avoid alert fatigue.

Golden signals

- **Latency** — how long requests take
- **Traffic** — demand on the system
- **Errors** — rate of failures
- **Saturation** — how full the resources are

DATA ENGINEERING TIE-IN

For pipelines monitor **freshness, row counts, failures, runtime and cost**; log each run with run-id and input window so you can debug and backfill. See data observability in the Data Engineering handbook.

7.5 Infrastructure as Code (IaC)

★★★

IN ONE LINE

Managing infrastructure through version-controlled definition files instead of manual clicks.

Infrastructure as Code (IaC) means defining servers, networks, databases and permissions in **text files** that a tool turns into real cloud resources. Infrastructure becomes **versioned, reviewable, repeatable** and testable like application code.

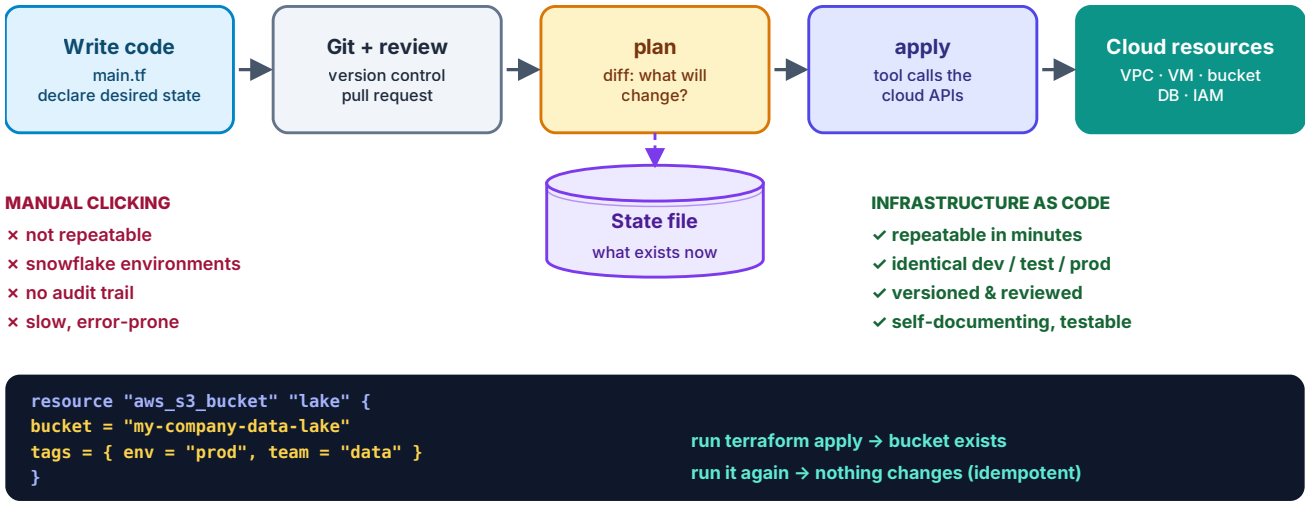


Figure 7.5 — From code to cloud: write, review, plan, apply

Concept	Meaning
Declarative	You describe the *end state* ("3 servers"); the tool works out the steps. Terraform, CloudFormation, Bicep.
Imperative	You script the *steps* ("create, then attach"). Shell scripts, SDK code, some Pulumi usage.
State	The tool's record of what it created; stored remotely and locked for teams.
Drift	Reality differs from code (someone clicked in the console). `plan` shows it.
Modules	Reusable building blocks (e.g. a standard VPC).
Idempotency	Applying the same code twice changes nothing the second time.

Tools

- **Terraform / OpenTofu** — multi-cloud, widely used
- **AWS CloudFormation / CDK**
- **Azure Bicep / ARM**
- **Pulumi** — real programming languages
- **Ansible** — configuration management of what runs *on* servers

Why companies insist on it

- Rebuild an environment (or DR region) in minutes.
- Consistency between dev, test, prod.
- Code review and audit trail for infra changes.
- Enables CI/CD for infrastructure.

INTERVIEW ANGLE

Common question: **Terraform vs CloudFormation**. Terraform is provider-agnostic (helps with multi-cloud and lock-in, 8.2); CloudFormation is AWS-native with deep integration and no separate state to manage.

7.6 CI/CD

★★★

IN ONE LINE

Automating building, testing and releasing software: continuous integration, delivery and deployment.

CI/CD automates the path from code change to running software. **Continuous Integration (CI)**: developers merge small changes frequently and every change is automatically built and tested. **Continuous Delivery (CD)**: every passing change is automatically prepared for release, with a **manual approval** to go live. **Continuous Deployment (CD)**: it goes live **automatically**.

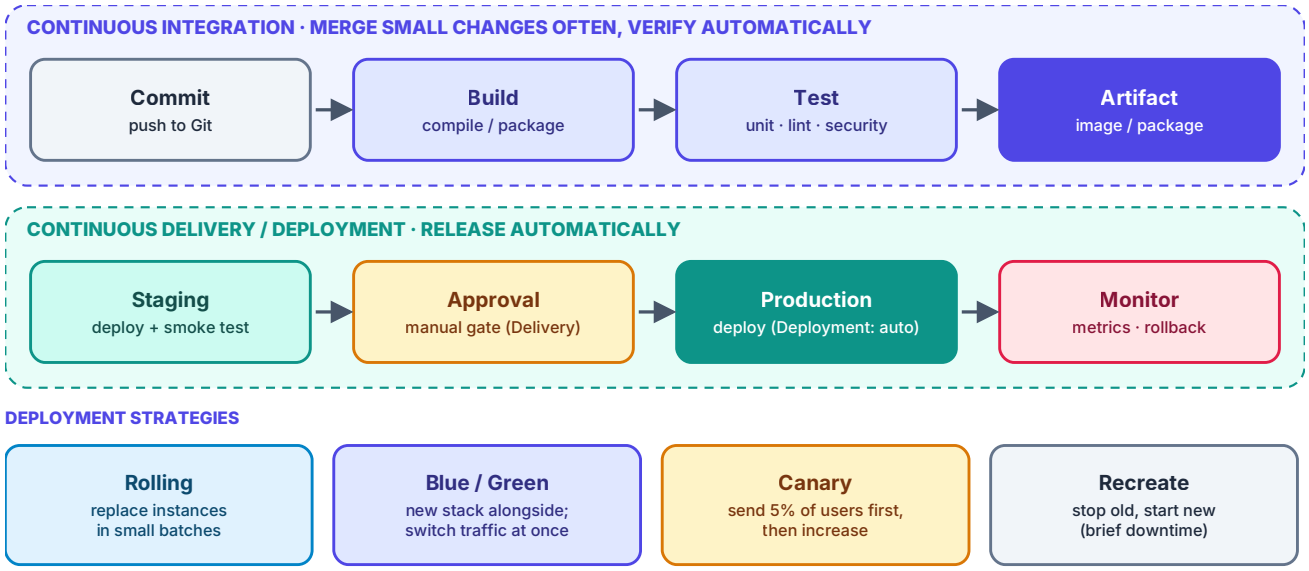


Figure 7.6 — A CI/CD pipeline and deployment strategies

	CI	Continuous Delivery	Continuous Deployment
Automates	Build + test on every commit	Release-ready artifact, deploy on button press	Deploy to production with no human step
Human gate	Code review	Yes, before production	No (relies on strong tests + monitoring)
Goal	Find bugs early	Always releasable	Fast, frequent releases

Tools

- **GitHub Actions**, GitLab CI, Jenkins
- **AWS CodePipeline / CodeBuild**, Azure DevOps / Pipelines, Google Cloud Build
- Argo CD / Flux for GitOps on Kubernetes

Good practices

- Small, frequent changes.
- Automated tests at every stage.
- Same artifact promoted through environments.
- Automatic **rollback** on failure.
- Secrets from a vault, not the pipeline file.

DATA ENGINEERING TIE-IN

Data pipelines need CI/CD too: lint SQL, unit-test transformations, run dbt tests in an isolated schema, deploy jobs via IaC / asset bundles, and promote dev → staging → prod. Details in the Data Engineering handbook (3.13).

7.7 Edge Computing

★★☆

IN ONE LINE

Processing data close to its source to cut latency and bandwidth.

Edge computing moves computation and data storage **closer to where data is generated or used** — devices, stores, factories, cell towers — instead of sending everything to a distant central cloud. It cuts **latency**, saves **bandwidth**, allows **offline operation** and can keep sensitive data local.

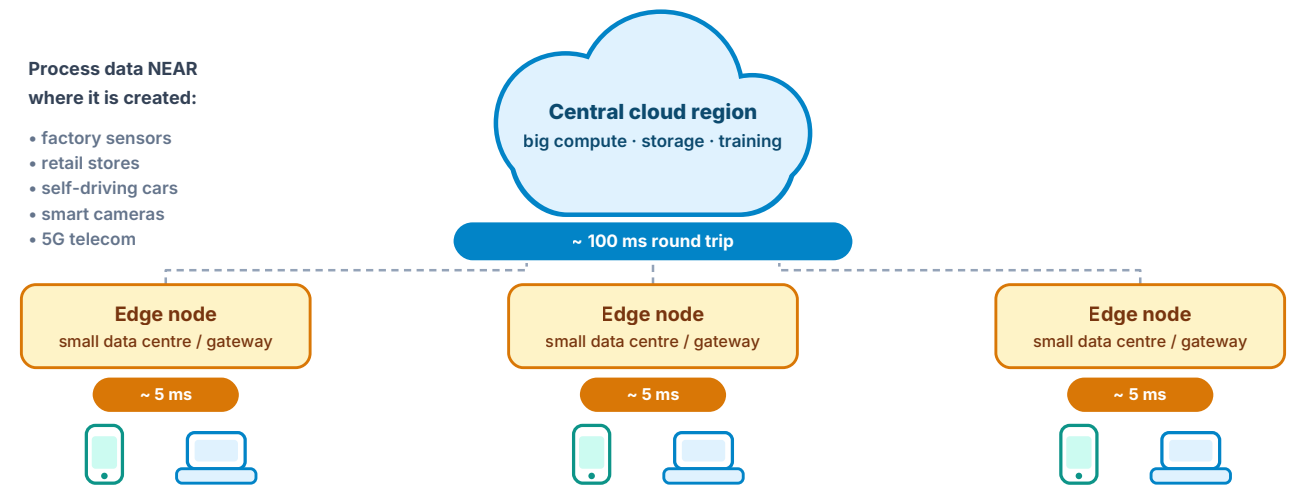


Figure 7.7 — Processing near the source vs in a central region

	Centralised cloud	Edge computing
Location	Large regional data centres	Near devices / users
Latency	Tens to hundreds of ms	Few ms
Bandwidth	Ships all raw data	Filters / aggregates locally, sends summaries
Connectivity	Needs a reliable link	Works with intermittent links
Best for	Heavy analytics, training, storage	Real-time decisions, IoT, video analytics

Examples of edge services

- AWS Outposts, Wavelength, IoT Greengrass
- Azure Stack Edge, IoT Edge
- Google Distributed Cloud
- CDN edge functions (CloudFront Functions, Cloudflare Workers)

Edge + cloud together

Typical pattern: devices → edge gateway (filter, react in ms) → cloud (store, train models, dashboards) → updated model pushed back to the edge.

REMEMBER

Edge does not replace the cloud; it **extends** it. Related: **fog computing** (an intermediate layer), and **CDN** (the most common form of edge delivery, next topic).

7.8 CDN (Content Delivery Network) ★★★

IN ONE LINE

A global network of edge caches that delivers content from the location nearest the user.

A **CDN (Content Delivery Network)** is a globally distributed network of **edge servers that cache copies of your content** close to users. Requests are served from the nearest edge instead of travelling to the origin server, giving faster loads, lower origin load and protection from traffic spikes and attacks.

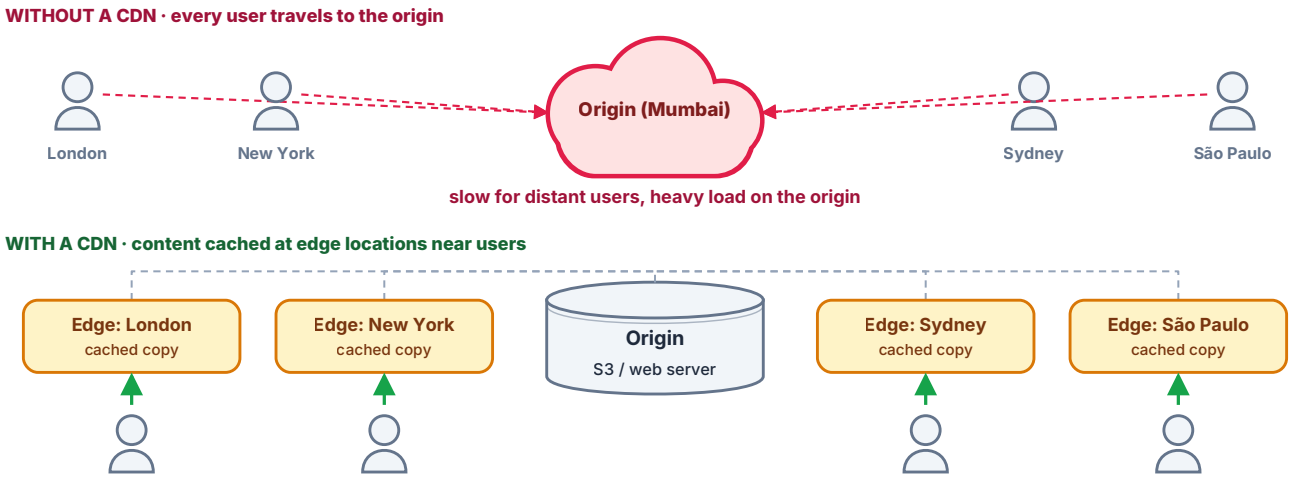


Figure 7.8a — Origin-only delivery vs delivery through edge caches

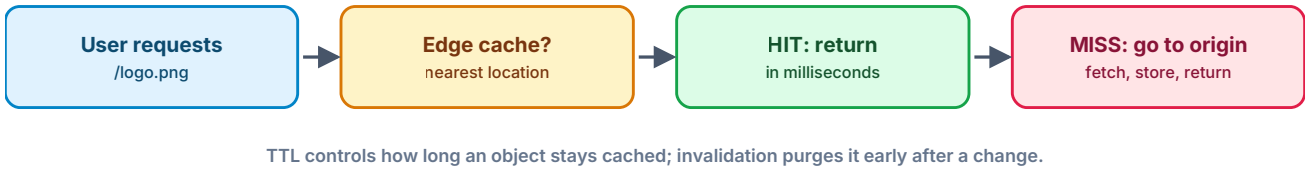


Figure 7.8b — Cache hit and cache miss

Benefit	How
Lower latency	Content served from a nearby edge location.
Reduced origin load & cost	Most requests never reach the origin; less egress from the origin.
Scalability	Edge network absorbs flash crowds (sales, live events).
Security	DDoS absorption, WAF at the edge, TLS termination.
Availability	Can serve cached content even if the origin is briefly down.

What to cache

Static content: images, CSS/JS, video segments, downloads. Dynamic or personalised pages can be cached briefly or accelerated by optimised routes.

Typical setup

Static site in an **object-storage bucket** (origin) → **CDN** in front → **HTTPS certificate** + custom domain via **DNS**. Ties together 3.1, 4.2, 4.3.

INTERVIEW ANGLE

CDN vs edge computing: a CDN primarily **caches and delivers** content near users; edge computing **runs code** near users or devices. Many CDNs now offer both (edge functions).

08

Part 8 · Cost, Strategy & Interview Toolkit

Spending wisely, avoiding lock-in, migrating, plus a cheat sheet, design template, Q&A and glossary.

- Cost Optimization
- Vendor Lock-in
- Well-Architected Framework
- Migration (6 Rs)
- Cloud for Data Engineers
- AWS / Azure / GCP cheat sheet
- Architecture-answer template
- Rapid-fire Q&A · Glossary

8.1 Cloud Cost Optimization

★★★

IN ONE LINE

Maximising business value per unit of cloud spend through right-sizing, smart purchasing and governance.

Cloud cost optimisation means getting the **most business value for every rupee/dollar spent** — not just spending less. It combines engineering choices, purchase options and governance, practised as **FinOps**.



Figure 8.1a — Typical waste and the lever for each



Figure 8.1b — The FinOps loop

Pricing option	Discount idea	Best for	Catch
On-demand	None (pay per second/hour)	Unpredictable, short-lived	Highest unit price
Reserved / Savings Plan	Substantial discount for 1–3 year commitment	Steady 24×7 workloads	Commitment risk if usage drops
Spot / Preemptible	Very large discount on spare capacity	Fault-tolerant batch, Spark workers, CI	Can be reclaimed with short notice
Serverless	Pay only for execution	Spiky, event-driven	Costly at constant high load

Quick wins

- Delete unused resources and old snapshots.
- Stop non-prod outside working hours.
- Move cold data to cheaper tiers.
- Set a **budget alert** today.

Cost-aware design

- Autoscale instead of over-provisioning.
- Cache and use a CDN to cut egress.
- Use managed services when ops time costs more.
- Compress / partition data to scan less.

DATA ENGINEERING TIE-IN

For warehouses and Spark: auto-suspend idle clusters, use spot workers for batch, partition and compress data, avoid `SELECT *` on pay-per-scan engines, and tag every pipeline's resources so cost can be attributed to teams.

8.2 Vendor Lock-in

★★★

IN ONE LINE

Dependence on one provider's proprietary services that makes switching costly.

Vendor lock-in is when moving away from a provider becomes so costly, slow or risky that you are effectively stuck with it. It comes from proprietary services and APIs, the effort of moving big datasets (**data gravity** and **egress fees**), staff skills and contracts.

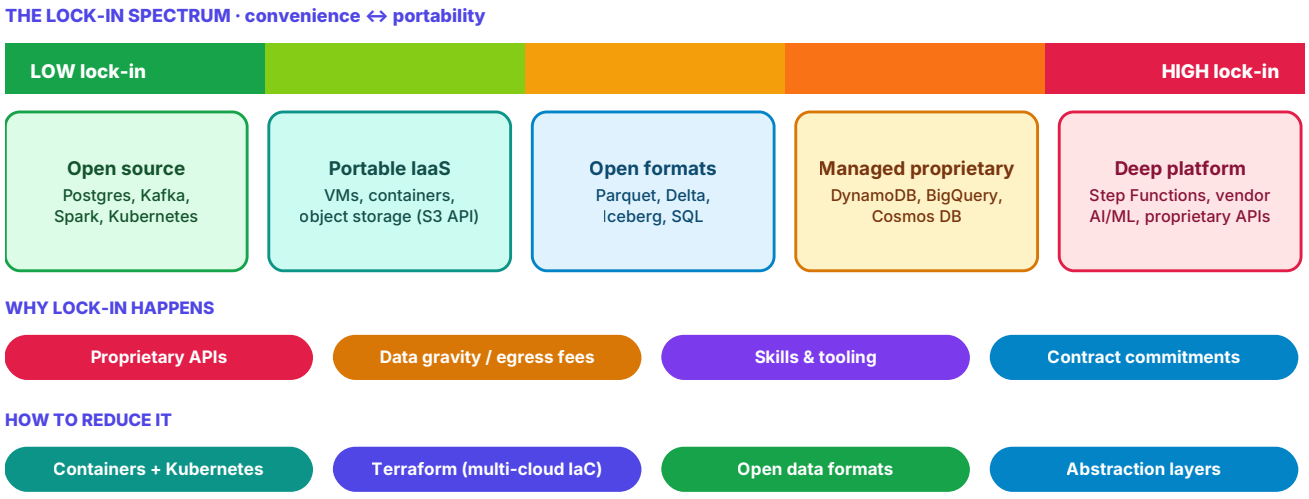


Figure 8.2 — The lock-in spectrum, its causes and mitigations

Strategy	What you do	Trade-off
Use open standards	Postgres, Kafka, Spark, Kubernetes, SQL, Parquet / Delta / Iceberg	May forgo some provider-specific conveniences
Containers & IaC	Docker + Kubernetes + Terraform	Extra abstraction to learn
Multi-cloud	Run on two or more providers	Complexity, cost, lowest-common-denominator features
Exit plan	Document how to migrate; keep data exportable	Planning effort now
Accept it deliberately	Use best-fit managed services; revisit regularly	Needs a clear cost / benefit view

INTERVIEW ANGLE

Balanced answer: **some lock-in is the price of productivity.** Managed services save engineering time, so avoiding all lock-in can cost more than it saves. The art is to **lock in knowingly** where the value is high and keep the **data and core logic portable** (open formats, standard APIs).

COMMON TRAP

"Multi-cloud" is not a free fix. It doubles skills, tooling and security surface. Many companies use **multi-cloud for specific reasons** (best-of-breed, acquisitions, regulation), not as blanket insurance.

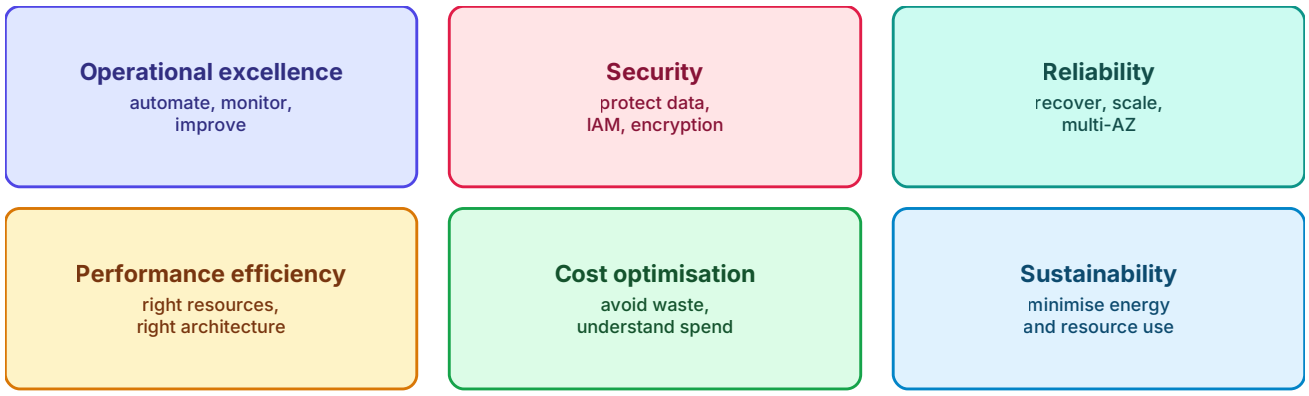
8.3 Well-Architected Framework

★★☆

IN ONE LINE

Six pillars of good cloud design: a checklist for evaluating architectures.

The **Well-Architected Framework** (AWS, with equivalents from Azure and Google) is a set of **best-practice pillars** for judging a cloud design. It gives you a ready-made structure for answering architecture questions and for justifying trade-offs.



AWS names six pillars; Azure and Google Cloud have very similar frameworks.

Use them as a checklist for any architecture discussion.

Figure 8.3 — The six pillars

Pillar	Key question	Where covered here
Operational excellence	Can we run, monitor and improve it?	7.4 Monitoring · 7.5 IaC · 7.6 CI/CD
Security	Is data and access protected?	Part 6 · 1.9 · 4.5
Reliability	Does it recover and scale?	Part 5 · 2.9 · 3.7
Performance efficiency	Are resources matched to need?	2.7 – 2.8 · 7.7 – 7.8
Cost optimisation	Are we spending wisely?	1.4 – 1.5 · 8.1
Sustainability	Are we minimising impact?	Right-sizing, serverless, regions with cleaner energy

INTERVIEW ANGLE

When asked to critique a design, walk the pillars: "security: private subnets and least privilege; reliability: multi-AZ with auto scaling; cost: right-sized and spot for batch…". It shows structure and breadth in under a minute.

8.4 Cloud Migration Strategies (the 6 Rs)

★★☆

IN ONE LINE

Standard options for moving existing applications to the cloud.

Companies rarely build everything new in the cloud: they **migrate existing systems**. The "**6 Rs**" (AWS framing) are the standard choices for each application, ordered roughly by effort and benefit.

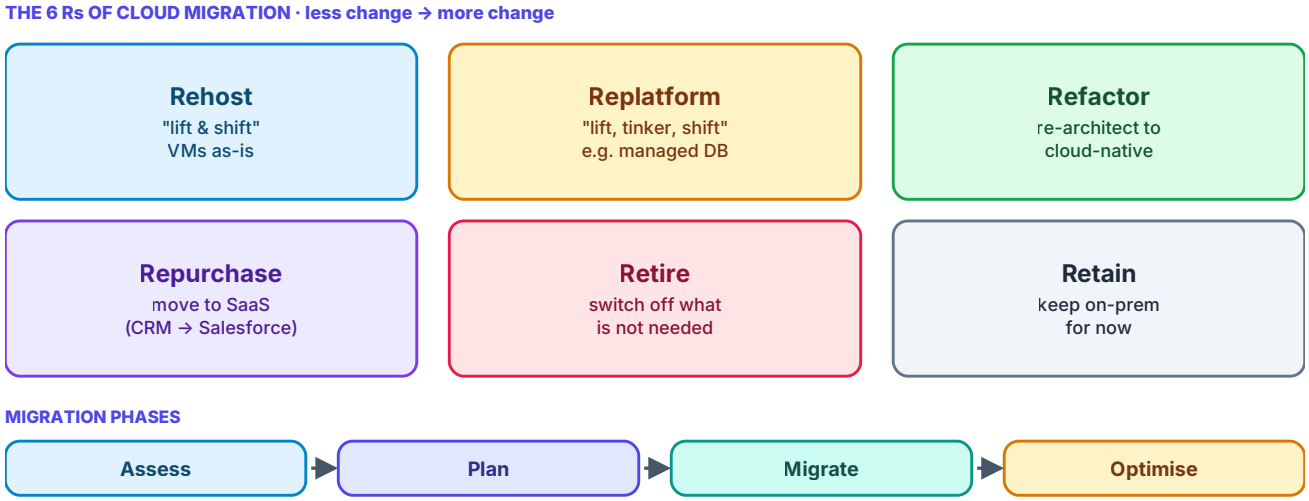


Figure 8.4 — Six migration strategies and the four phases

Strategy	What it means	Example	Effort / benefit
Rehost	Move as-is to cloud VMs ("lift and shift")	App server → EC2	Low / quick win, limited cloud benefit
Replatform	Small optimisations, no core redesign	Self-hosted MySQL → managed RDS	Medium / good balance
Refactor	Re-architect to cloud-native (microservices, serverless)	Monolith → services + functions	High / max benefit
Repurchase	Replace with SaaS	On-prem CRM → SaaS CRM	Medium
Retire	Decommission unneeded systems	Unused reports app	Saves cost
Retain	Leave on-prem for now (compliance, recent investment)	Mainframe	None / revisit later

DATA ENGINEERING TIE-IN

Typical data migration: lift Hadoop/on-prem warehouse data into object storage + a lakehouse (**replatform/refactor**), use CDC to keep source and target in sync during cut-over, validate with row counts and checksums, then retire the old system.

8.5 Cloud for Data Engineering: Putting It Together

★★★

IN ONE LINE

How cloud storage, compute, networking, security and DevOps combine into a data platform.

This is where the two handbooks meet. Every cloud concept so far shows up in a **cloud data platform**: object storage is the lake, VMs / serverless / Kubernetes run the engines, IAM and KMS secure it, networks connect it, and IaC + CI/CD ship it.

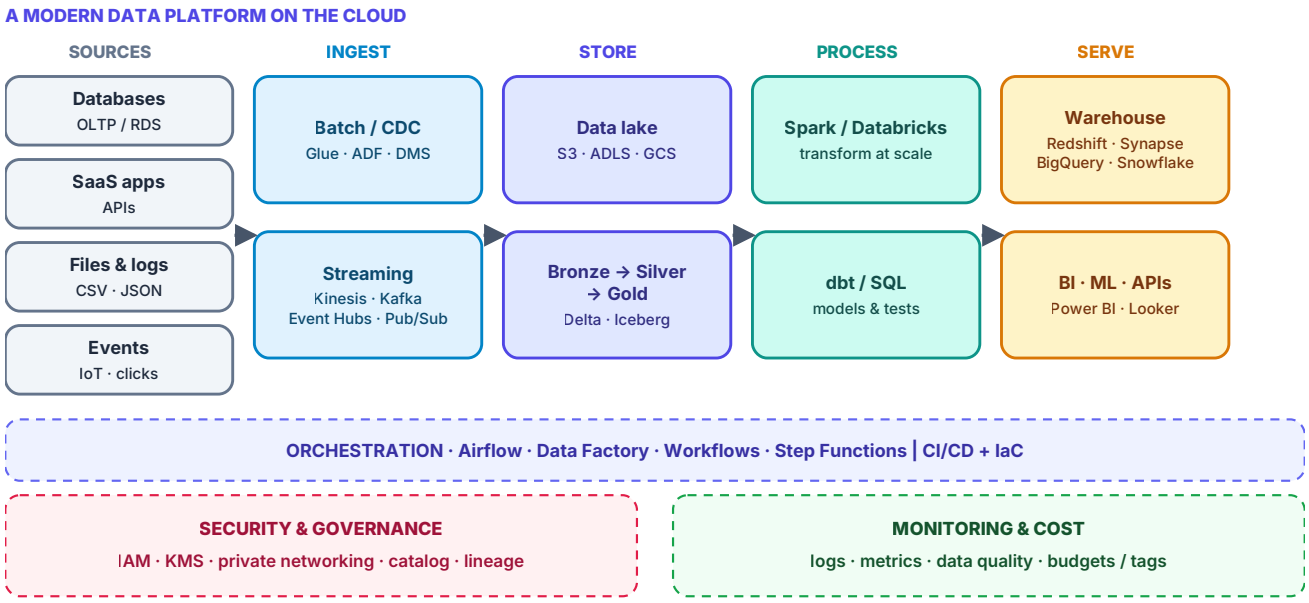


Figure 8.5 — Reference architecture for data engineering on the cloud

Stage	AWS	Azure	Google Cloud
Object storage (lake)	S3	ADLS Gen2 / Blob	Cloud Storage
Batch ETL / integration	Glue, DMS	Data Factory, Synapse pipelines	Dataflow, Data Fusion
Streaming	Kinesis, MSK (Kafka)	Event Hubs	Pub/Sub, Dataflow
Spark / big-data compute	EMR, Glue, Databricks	Databricks, Synapse Spark, HDInsight	Dataproc, Databricks
Warehouse	Redshift	Synapse / Fabric	BigQuery
Orchestration	MWAA (Airflow), Step Functions	Data Factory, Airflow	Cloud Composer (Airflow)
Catalog / governance	Glue Data Catalog, Lake Formation	Purview, Unity Catalog	Dataplex, Data Catalog
BI	QuickSight	Power BI	Looker

INTERVIEW ANGLE

Prepare a 60-second story: *"Data lands from sources into object storage (Bronze), Spark/dbt clean and model it (Silver/Gold) under IAM and encryption, a warehouse or SQL engine serves BI, Airflow orchestrates, Terraform + CI/CD deploy, monitoring and budgets keep it healthy."*

8.6 Multi-Cloud Service Cheat Sheet

★★★

IN ONE LINE

AWS, Azure and Google Cloud equivalents side by side.

The same concepts under different brand names. Learn the **concept** first, then use this table to translate between providers in an interview or on the job.

Concept	AWS	Azure	Google Cloud
Virtual machine	EC2	Virtual Machines	Compute Engine
Auto scaling	Auto Scaling groups	VM Scale Sets	Managed Instance Groups
Containers (managed K8s)	EKS / ECS	AKS	GKE
Serverless functions	Lambda	Azure Functions	Cloud Functions / Cloud Run
Object storage	S3	Blob Storage	Cloud Storage
Block storage	EBS	Managed Disks	Persistent Disk
File storage	EFS / FSx	Azure Files	Filestore
Archive storage	S3 Glacier	Blob Archive tier	Archive storage class
Relational DB (managed)	RDS / Aurora	Azure SQL / Database for PostgreSQL	Cloud SQL / AlloyDB
NoSQL	DynamoDB	Cosmos DB	Firestore / Bigtable
Data warehouse	Redshift	Synapse / Fabric	BigQuery
Virtual network	VPC	VNet	VPC
Firewall rules	Security Groups / NACL	NSG / Azure Firewall	VPC firewall rules
Load balancer	ELB (ALB / NLB)	Load Balancer / App Gateway	Cloud Load Balancing
DNS	Route 53	Azure DNS	Cloud DNS
CDN	CloudFront	Front Door / CDN	Cloud CDN
Private connectivity	Direct Connect	ExpressRoute	Cloud Interconnect
Identity & access	IAM	Entra ID + RBAC	Cloud IAM
Key management	KMS	Key Vault	Cloud KMS
Secrets	Secrets Manager	Key Vault secrets	Secret Manager
Messaging / queues	SQS / SNS	Service Bus / Event Grid	Pub/Sub
Monitoring & logging	CloudWatch / CloudTrail	Azure Monitor	Cloud Monitoring / Logging
IaC (native)	CloudFormation / CDK	Bicep / ARM	Deployment Manager (Terraform common)
CI/CD	CodePipeline / CodeBuild	Azure DevOps / Pipelines	Cloud Build
API management	API Gateway	API Management	API Gateway / Apigee

INTERVIEW ANGLE

Names and bundles change over time (for example Azure Active Directory is now **Microsoft Entra ID**), so confirm exact names in the provider's documentation before quoting them in writing.

8.7 How to Answer a Cloud Architecture Question

★★★

IN ONE LINE

A repeatable structure for designing a hosted application in an interview.

Fresher and junior interviews often end with *"How would you host a website / application on the cloud?"*. Use a structure and tie each choice to a concept from this book.

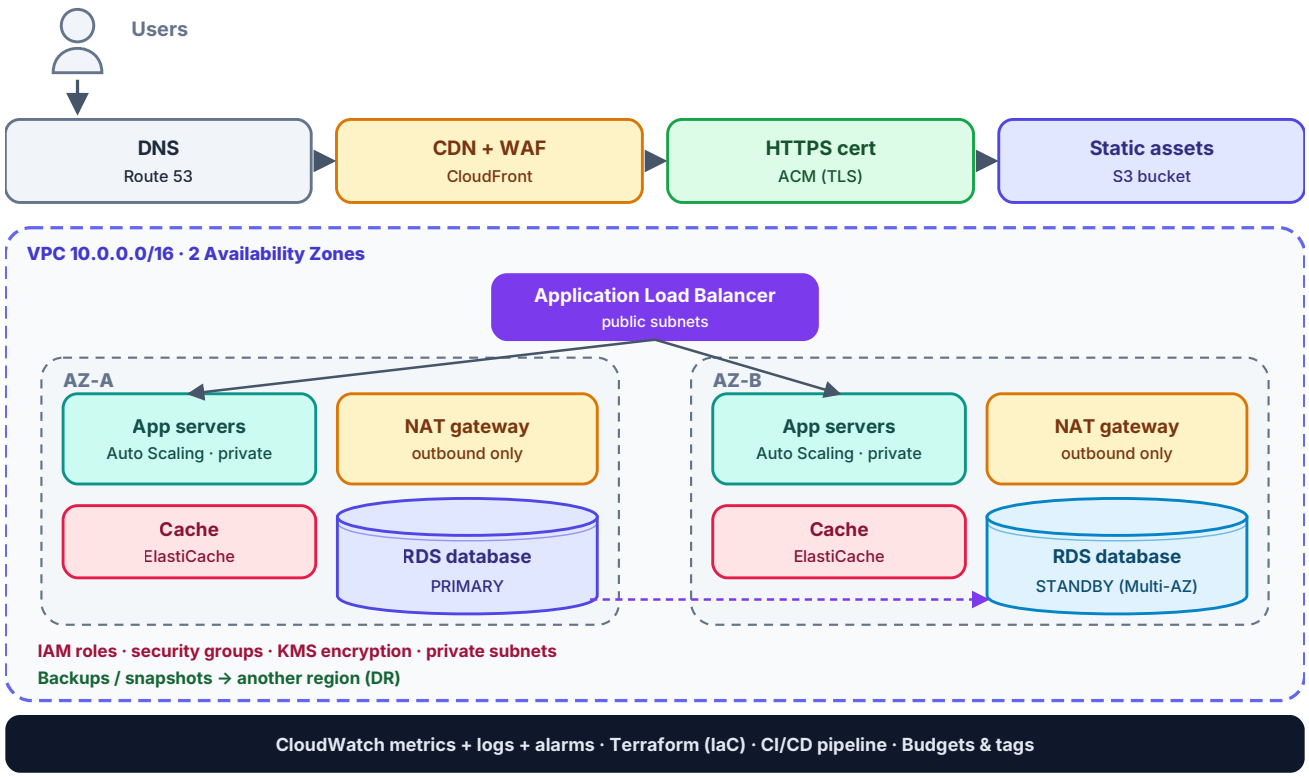


Figure 8.7 — A secure, scalable, highly available three-tier web application

Step	What to say	Concepts
1 Clarify	Users, traffic, latency, data sensitivity, budget, availability target	SLA 5.5 · RTO/RPO 5.7
2 Entry	DNS → CDN + WAF → HTTPS → load balancer	4.2 · 7.8 · 4.3 · 4.8
3 Compute	Stateless app servers in an auto scaling group across 2 AZs (or containers / serverless)	2.9 · 5.1 · 2.4 · 2.6
4 Data	Managed relational DB Multi-AZ, cache for hot reads, object storage for files	3.8 · 3.1 · 5.2
5 Network & security	Public subnet for LB only; private for app + DB; security groups; IAM roles; encryption	4.6 · 4.5 · 6.1 · 6.4
6 Resilience	Backups, cross-region copy, tested recovery plan	3.7 · 5.6
7 Operate	Monitoring, alerts, logs; IaC and CI/CD; tags, budgets, right-sizing	7.4 – 7.6 · 8.1

INTERVIEW ANGLE

Finish with **trade-offs**: "For a tiny site I would use static hosting + serverless to save cost; for global users add multi-region; if the team is small I prefer managed services over self-run VMs." Saying *why* beats naming more services.

8.8 Rapid-Fire Interview Q&A

★★★

IN ONE LINE

Thirty fresher-level questions with answers short enough to say out loud.

Q What is cloud computing?

On-demand delivery of computing resources (servers, storage, databases, networking) over the internet with pay-as-you-go pricing, instead of owning hardware.

Q IaaS vs PaaS vs SaaS?

IaaS gives raw infrastructure (VMs) you manage up from the OS; PaaS gives a platform where you manage only code and data; SaaS is a finished application. More abstraction means less control and less work.

Q Public vs private vs hybrid cloud?

Public: shared provider infrastructure. Private: dedicated to one organisation. Hybrid: on-prem/private connected to public cloud and used together.

Q CapEx vs OpEx?

CapEx is a big upfront purchase of assets; OpEx is recurring spend as you use services. Cloud shifts IT spending from CapEx to OpEx.

Q Explain the shared responsibility model.

The provider secures the cloud infrastructure (security OF the cloud); the customer secures their data, identities, configuration and, depending on the model, OS and applications (security IN the cloud).

Q Virtual machine vs container?

A VM virtualises hardware and includes a full OS (heavy, strong isolation). A container shares the host OS kernel and packages only the app and libraries (lightweight, fast start).

Q What is serverless? Is there really no server?

You run code without managing servers; the provider provisions, scales and bills per execution. Servers still exist but are abstracted away.

Q Scalability vs elasticity?

Scalability: ability to handle growth by adding resources. Elasticity: automatically adding and removing resources as demand changes.

Q Vertical vs horizontal scaling?

Vertical makes one machine bigger (limited, may need downtime). Horizontal adds more machines behind a load balancer (nearly unlimited, resilient, needs stateless design).

Q Object vs block vs file storage?

Object: flat, HTTP-accessed, cheap and scalable (S3). Block: raw fast disk for one server (EBS). File: shared hierarchical network file system (EFS).

Q Durability vs availability?

Durability: data is not lost or corrupted (11 nines for object storage). Availability: the service can be accessed right now (e.g. 99.99%).

Q Is replication a backup?

No. Replication copies changes (including deletions and corruption) almost instantly for availability. Backups keep point-in-time copies you can restore from.

Q Public vs private IP?

Public IPs are internet-routable and globally unique; private IPs are used inside a network and are not routable on the internet. NAT lets private hosts reach out.

Q How does DNS work?

A resolver walks root → TLD → authoritative servers to translate a domain name into an IP, then caches the answer for its TTL.

Q HTTP vs HTTPS?

HTTPS is HTTP over TLS: encrypted, tamper-proof and with server identity via certificates (port 443 vs 80).

Q Security group vs NACL?

Security group: stateful, instance-level, allow rules only. NACL: stateless, subnet-level, allow and deny rules in order.

Q Public vs private subnet?

A public subnet routes to an internet gateway (load balancers). A private subnet has no direct internet route (app servers, databases), using NAT for outbound only.

Q Region vs Availability Zone?

A region is a geographic area; AZs are isolated data-centre groups inside it. Multi-AZ gives high availability; multi-region gives disaster recovery.

Q HA vs fault tolerance?

HA minimises downtime through redundancy and fast failover; fault tolerance continues with no interruption at all and costs more.

Q Define RTO and RPO.

RTO: maximum acceptable downtime. RPO: maximum acceptable data loss, measured as time back from the failure.

Q Authentication vs authorization?

Authentication verifies who you are (401 on failure); authorization decides what you may do (403 on failure).

Q What is least privilege?

Grant only the minimum permissions needed for the shortest time, so mistakes or stolen credentials cause limited damage.

Q Encryption at rest vs in transit?

At rest protects stored data (AES-256, KMS keys); in transit protects data moving over networks (TLS/HTTPS, VPN).

Q Monolith vs microservices?

A monolith is one deployable unit (simple, tightly coupled); microservices are small independently deployable services (scalable, flexible, more operational complexity).

Q Monitoring vs logging?

Monitoring watches metrics and alerts on health; logging records detailed events to explain what happened. Tracing follows a request across services.

Q What is Infrastructure as Code?

Defining infrastructure in version-controlled files (Terraform, CloudFormation) so environments are repeatable, reviewable and automated.

Q CI vs CD?

CI builds and tests every change automatically; continuous delivery keeps software releasable with a manual approval; continuous deployment releases automatically.

Q CDN vs edge computing?

A CDN caches and delivers content from edge locations near users; edge computing runs processing close to devices or users to cut latency.

Q How do you reduce cloud costs?

Right-size, auto-scale, schedule non-prod, use reserved/spot where suitable, tier storage, remove idle resources, tag and set budgets and alerts.

Q What is vendor lock-in and how do you reduce it?

Dependence on proprietary provider services that makes switching hard. Reduce with open standards, containers, Terraform, open data formats and an exit plan, accepting some lock-in for productivity.

8.9 Glossary: Every Term in One Line

★★☆

IN ONE LINE

A quick-revision index of all the concepts in this handbook.

Every term from your list plus the extras, in one place. The last column points to the section with the full explanation and diagram.

Term	One-line definition	§
API	Contract letting software request services from other software.	7.2
Authentication	Proving who you are.	6.2
Authorization	Deciding what an identity may do.	6.2
Auto Scaling	Automatic adding/removing of instances based on demand.	2.9
Availability Zone	Isolated data-centre group within a region.	5.1
Backup	Point-in-time copy for recovery.	3.7
Block storage	Raw fixed-size blocks presented as a disk.	3.2
Bucket	Top-level container for objects in object storage.	3.5
CapEx / OpEx	Upfront asset purchase vs recurring operating cost.	1.4
CDN	Edge cache network delivering content near users.	7.8
CI/CD	Automated build, test and release pipeline.	7.6
Client / Server	Requester and always-on provider of a service.	1.1
Cloud computing	On-demand, metered computing resources over the internet.	1.2
Container	Lightweight packaged app sharing the host OS kernel.	2.4
CSP	Cloud Service Provider (AWS, Azure, GCP).	1.8
Disaster recovery	Restoring systems after a major failure.	5.6
DNS	Translates domain names to IP addresses.	4.2
Durability	Probability data is not lost.	3.6
Edge computing	Processing close to the data source.	7.7
Elasticity	Automatic scale up and down with demand.	2.7
Encryption at rest / in transit	Protecting stored vs moving data.	6.4
Fault tolerance	Continuing with no interruption despite failure.	5.3
File storage	Shared hierarchical network file system.	3.3
Firewall	Allows or blocks traffic by rule.	4.5
High availability	Minimal downtime through redundancy.	5.2
HTTP / HTTPS	Web protocol and its TLS-encrypted version.	4.3
Hybrid cloud	On-prem/private plus public cloud, connected.	1.6
Hypervisor	Software that creates and runs VMs.	2.2

Term	One-line definition	§
IaaS / PaaS / SaaS	Service models by how much the provider manages.	1.7
IaC	Infrastructure defined in code.	7.5
IAM	Controls who can do what on which resources.	6.1
Kubernetes	Container orchestrator.	2.5
Least privilege	Minimum permissions for minimum time.	6.3
Load balancer	Distributes traffic across healthy servers.	4.8
Microservices	Many small independently deployable services.	7.1
Monitoring / Logging	Metrics and alerts vs detailed event records.	7.4
Object storage	Flat, HTTP-accessed storage of objects.	3.1
On-premises	Hardware owned and run by the organisation.	1.3
Pay-as-you-go	Metered billing by actual usage.	1.5
Port / Protocol	Program identifier on a host; rules for communication.	4.4
Private cloud	Infrastructure dedicated to one organisation.	1.6
Public / Private IP	Internet-routable vs internal-only addresses.	4.1
Public cloud	Shared provider-run infrastructure.	1.6
Region	Geographic area containing multiple AZs.	5.1
Replication	Live copy for availability and failover.	3.7
RTO / RPO	Max downtime / max data loss tolerated.	5.7
Scalability	Handling growth by adding resources.	2.7
Serverless	Run code on demand without managing servers.	2.6
Shared responsibility	Provider secures cloud; customer secures use of it.	1.9
SLA / SLO / SLI	Reliability contract, objective and indicator.	5.5
Subnet / VPC	Slice of a private cloud network / the network.	4.6
Vendor lock-in	Costly dependence on one provider.	8.2
Vertical / Horizontal scaling	Bigger machine vs more machines.	2.8
Virtual machine	Software-defined computer with its own OS.	2.3
Virtualization	Splitting one physical machine into many virtual ones.	2.1